

FINAL TERM PREPARATION

Lectures 19 – 45 • Complete Notes

L19 Degenerate BST & AVL Trees

L20 AVL Tree Insertion & Rotation

L21 AVL Tree Building & Rotation Cases

L22 Double Rotation & AVL Insert Code

L23 Rotation Code & Deletion in AVL

L24 Deletion Cases & Expression Trees

L25 Expression Trees & Huffman Encoding

L26 Huffman Encoding & Tree Properties

L27 Threaded Binary Trees

L28 Threaded Inorder & Complete Binary Tree

L29 Heap Data Structure & Insertion

L30 Heap Insert/Delete & BuildHeap

L31 BuildHeap, Heap Methods & C++ Code

L32 percolateDown, buildHeap & Linear Time Proof

L33 Priority Queue, Selection, Heap Sort & Disjoint...

L34 Disjoint Sets & Equivalence Relations

L35 Disjoint Sets Implementation (Parent Array)

L36 Disjoint Sets Optimizations

L37 Applications of Disjoint Sets

L38 Table ADT & Binary Search

L39 Binary Search & Skip Lists

L40 Skip List Operations

L41 Skip List Implementation & Hashing Intro

L42 Collision Resolution — Linear Probing

L43 Hashing Applications & Sorting Intro

L44 Elementary Sorting Algorithms

L45 Merge Sort, Quick Sort & Course Overview

Prepared by: \$ilent \$park

Join WhatsApp Group:

<https://chat.whatsapp.com/KJpwuFbEM9GHpjX71IC3hW>

Lesson 19: Degenerate BSTs & the Birth of AVL Trees

When sorted data breaks your tree — and how AVL fixes it

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi baat hai jaise agar aapke paas 11 dost hain aur unhein height ke hisaab se ek seedhi line mein khara kar diya jaye — to kisi ko dhoondna mushkil ho jata hai, kyunki aapko har ek ko ek ek karke dekhna padta hai. Isi tarah agar BST mein sorted data (jaise 3,4,5,7,9...) insert kiya jaye, to tree linked-list jaisi shape ban jaata hai aur search $O(n)$ ho jaati hai, $O(\log n)$ nahi. AVL tree isi problem ka solution hai — jaise agar hum dostoon ko is tarah arrange karein ke har shakhs ke aas-paas logon ki tadad barabar ho, to kisi ko bhi dhoondna asaan ho jaata hai.

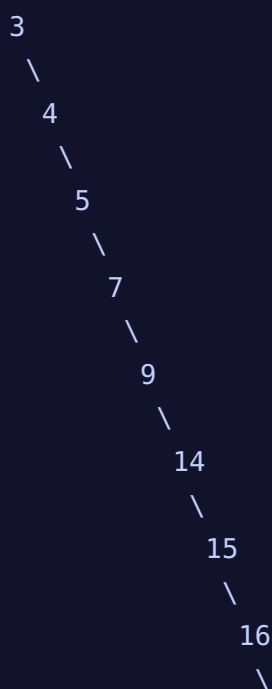
KEY TERMS

- 1 Degenerate Tree** — Sorted data insert karne se banne wala BST jo linked-list jaisa lagta hai — sirf left ya sirf right children hote hain, search $O(n)$ ho jaati hai.
- 2 Balance Factor** — Left subtree height minus right subtree height — AVL mein har node ka balance factor -1, 0, ya 1 hona chahiye.

- 3 **AVL Tree** — Special BST jisme har node ka left-right height farq sirf -1, 0, ya 1 ho sakta hai — tree hamesha balanced rehta hai, search $O(\log n)$ guarantee hai.
- 4 **const Reference Parameter** — Function ko reference bhejna without copy (`const SomeClass ¶m`) — const lagane se function us object ko change nahi kar sakta.
- 5 **const Member Function** — Function signature ke end mein const (e.g. `findMin() const`) — yeh member variables ko change nahi kar sakti, sirf read karegi.
- 6 **const Return Type** — Return type se pehle const (e.g. `const EType& findMin() const`) — jo reference return ho, usay caller change nahi kar sakta.
- 7 **Height of Empty Tree** — AVL definition mein empty tree ki height -1 mani jaati hai — is se balance factor calculation consistent rehti hai.

DIAGRAM TIME

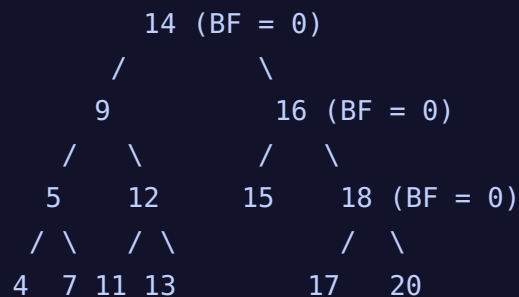
► DEGENERATE TREE (SORTED INSERTION: 3,4,5,7,9,14,15,16,17,18,20)





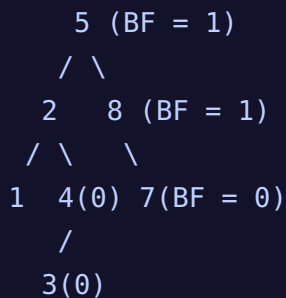
↗ Sorted data insert karne se tree ek seedhi line (linked list) ban jaata hai — search $O(n)$ ho jaati hai, $O(\log n)$ nahi.

► BALANCED AVL TREE (SAME DATA, BALANCED STRUCTURE)



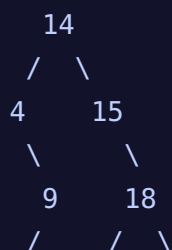
↗ Har node ka balance factor -1, 0, ya 1 hai — is liye yeh valid AVL tree hai, search $O(\log n)$ mein ho jaati hai.

► BALANCE FACTOR EXAMPLE (FROM LECTURE)



↗ Har node par left-right height difference -1, 0, ya 1 — AVL condition satisfied hai.

► BST FOR UNSORTED DATA (14,15,4,9,7,18,3,5,16,20,17)



```

      7    16  20
     /      \
    3        17
     \
      5

```

↗ *Jab data unsorted hota hai, tree balanced dikhta hai aur search efficient rehti hai.*

AVL Connection: Sorted data se degenerate tree banti hai (linked-list jaisi), AVL tree isi problem ko fix karta hai by forcing balance factor -1,0,1 at every node. Chahe data kisi bhi order mein aaye, AVL tree balanced rehta hai aur search $O(\log n)$ guaranteed hai.

CODE EXPLANATION

```

void someFunction(const SomeClass &param) {
    // param cannot be modified inside this function
}

EType& findMin() const;
const EType& findMin() const;

```

- 1 **const SomeClass ¶m** — Reference parameter hai jisay const lagaya gaya — copy nahi banti (speed/memory bachti hai) aur function us object ko change nahi kar sakta.
- 2 **EType& findMin() const;** — const at the end ka matlab function class ki member variables change nahi kar sakta, sirf read access hai — is liye getter functions mein use hota hai.
- 3 **const EType& findMin() const;** — Return type se pehle const ka matlab calling function returned reference ko modify nahi kar sakta — object safe rehta hai.

EXAM FOCUS POINTS

MCQ

const ka triple use: (1) reference parameter (2) member function end mein (3) return type se pehle — teeno ka alag purpose hai, exam mein frequently puchha jaata hai.

CONFUSION

Degenerate tree vs balanced tree ka farq — sorted data se BST linked-list jaisa ban jaata hai, is liye search $O(n)$ ho jaati hai. Yeh bhoolna nahi chahiye.

FORMULA

Balance Factor = $\text{Height}(\text{left}) - \text{Height}(\text{right})$. Empty tree ki height = -1. AVL condition: $-1 \leq \text{balance factor} \leq 1$ for every node.

MCQ

AVL condition har node par check hoti hai, sirf root par nahi. Agar kisi bhi node par balance factor -2 ya 2 ho to tree AVL nahi hai.

PRACTICE

Sorted data (e.g. 1,2,3,4,5,6,7) ko BST mein insert karke degenerate tree draw karein, phir usay AVL tree mein convert karein — yeh practice question exam mein aata hai.

MIND MAP

Degenerate BST → AVL Tree

The Problem

- Sorted data → degenerate BST
- Looks like a linked list
- Search becomes $O(n)$ instead of $O(\log n)$

The Goal

- Keep left/right heights close
- Search stays $O(\log n)$

- Tree should be balanced at all times

The Solution: AVL

- Balance factor = -1, 0, or 1
- Empty tree height = -1
- Named after Adelson-Velskii & Landis
- Every node must satisfy condition

const Usage

- Reference parameter: const & avoids copy
- Member function: const at end — read only
- Return type: const & protects returned reference

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Why does inserting sorted data into a BST create a problem?

A: Sorted data insert karne se har naya node sirf ek side (right ya left) jaata hai, kyunki data already sorted hai. Is liye tree ek linked list jaisi shape ban jaati hai aur search slow ho jaati hai — $O(n)$ instead of $O(\log n)$.

Q: What is the main difference between a BST and an AVL tree?

A: AVL tree ek special BST hai jisme extra condition hoti hai — har node ka balance factor (-1, 0, ya 1) maintain karna zaroori hai. BST mein yeh condition nahi hoti, is liye BST degenerate ho sakta hai.

Q: Why do we use const with reference parameters?

A: Const reference parameter copy nahi banata (is liye speed aur memory bachti hai) aur function us object ko change nahi kar sakta — yeh discipline aur safety dono deta hai.

Q: What does a const member function guarantee?

A: Const member function (signature ke end mein const) class ki member variables ko change nahi kar sakti — sirf read access hai. Is liye getter functions ke liye perfect hai.

Q: Why is the height of an empty tree defined as -1 in AVL trees?

A: Empty tree ki height -1 mani jaati hai taake balance factor calculation consistent rahe. Agar leaf node ka balance factor calculate karein, to left/right empty subtrees ki height -1 hoti hai, is se balance factor 0 aata hai (jo leaf node ke liye sahi hai).

► QUICK QUIZ (10 QUESTIONS)

1. What does inserting sorted data into a BST produce? **(Degenerate tree)**
2. What is the height of an empty tree in AVL definition? **(-1)**
3. Allowed range for an AVL node's balance factor? **(-1, 0, or 1)**
4. What does const at the end of a member function do? **(Prevents changing member variables)**
5. What does const before a return type do? **(Returned reference is read-only)**
6. Why use reference parameters instead of value? **(Avoid copying objects)**
7. What is the balance factor of a leaf node? **(0)**
8. Which two people is AVL named after? **(Adelson-Velskii and Landis)**
9. What happens if balance factor is 2 or -2? **(Tree is not AVL, needs rotation)**
10. Search in a degenerate tree is O(?) **(O(n))**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Degenerate Tree:** A BST that looks like a linked list, usually from inserting sorted data — search $O(n)$.
- **Balance Factor:** $\text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree})$.
- **AVL Tree:** BST where every node's balance factor is -1, 0, or 1.
- **const Reference:** Reference parameter that cannot modify the object — avoids copy.
- **Height of Empty Tree:** Defined as -1 in AVL context.

IMPORTANT CONCEPTS

- Sorted data $\rightarrow$ degenerate tree $\rightarrow$ search $O(n)$ instead of $O(\log n)$.
- AVL tree keeps tree balanced by enforcing balance factor at every node.
- `const` keyword protects objects from unintended modification — used with references, member functions, and return types.
- Empty tree height = -1 (used in balance factor calculation).
- AVL condition must hold at every node, not just the root.

△ IMPORTANT FORMULAS

- **Balance factor** = $\text{Height}(\text{left}) - \text{Height}(\text{right})$
- **Height of empty tree** = -1
- **AVL condition:** $-1 \leq \text{balance factor} \leq 1$ for every node
- **Search complexity in balanced BST:** $O(\log n)$
- **Search complexity in degenerate tree:** $O(n)$

FREQUENTLY ASKED TOPICS

- Degenerate tree vs AVL tree — difference and why it matters.
- `const` with reference parameters, member functions, and return types.
- Balance factor calculation and AVL condition.
- Why sorted data is problematic for BST.
- Height of empty tree = -1 — why this definition is used.

COMMON MISTAKES

- Assuming AVL condition only needs to hold at the root — it must hold at every node.
- Returning reference to a local variable (undefined behavior).
- Forgetting that const member function cannot change member variables.
- Thinking degenerate tree is still efficient — it's not, search becomes $O(n)$.
- Confusing height of empty tree as 0 instead of -1 in AVL context.

One-Day-Before-Exam Summary: Sorted data BST mein insert karne se woh linked-list jaisa ban jaata hai aur search slow ho jaati hai ($O(n)$). AVL tree isi problem ka solution hai — har node ka balance factor -1, 0, ya 1 hona chahiye, aur empty tree ki height -1 mani jaati hai. const keyword teen jagah use hota hai: (1) reference parameter (copy avoid + read-only), (2) member function end mein (read-only access), aur (3) return type se pehle (returned reference read-only). AVL tree ensures search always $O(\log n)$ regardless of data order.

Lesson 20: AVL Tree Insertion & Rotation

Building balanced trees — step by step insertion with rotations

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap apne dostoon ko ek party mein arrange kar rahe hain — agar koi naya dost aata hai, to aapko sab ko is tarah rearrange karna padta hai ke har shakhs ke dono taraf logon ki tadad almost barabar rahe. Agar aap sab ko seedhi line mein khara kar denge, to kisi ko dhoondna mushkil ho jayega. Isi tarah AVL tree mein jab naya node insert hota hai, agar tree unbalanced ho jata hai, toh hum rotation (rearrangement) karte hain taake tree dobara balanced ho jaye — aur search $O(\log n)$ hi rahe.

KEY TERMS

- 1 AVL Tree** — BST jisme har node ka balance factor -1, 0, ya 1 ho. Named after Adelson-Velskii & Landis (1962).
- 2 Balance Factor** — $\text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree})$. AVL condition: -1, 0, ya 1.
- 3 Height of Tree** — Maximum level of its leaves (deepest node ka level). Empty tree ki height = -1.

- 4 **Rotation** — Tree ko rearrange karne ka process jab AVL condition violate ho — nodes ko left ya right rotate karte hain, inorder traversal same rehta hai.
- 5 **Left Rotation** — Jab node ka balance factor -2 ho (right subtree heavy) — nodes ko left side rotate karte hain.
- 6 **Right Rotation** — Jab node ka balance factor 2 ho (left subtree heavy) — nodes ko right side rotate karte hain.
- 7 **Inorder Traversal** — Rotation ke baad bhi inorder traversal same rehta hai — yeh BST property ko maintain karta hai.

DIAGRAM TIME

► AVL TREE WITH BALANCE FACTORS (FROM LECTURE)

```
      5 (BF = 1)
     /  \
    2    8 (BF = 1)
   / \   \
  1  4(0) 7(BF = 0)
     /
    3(0)
```

↗ Har node ka balance factor -1, 0, ya 1 hai — yeh ek valid AVL tree hai.

► NOT AN AVL TREE (VIOLATION AT ROOT 6, BF = 2)

```
      6 (BF = 2) ← violation
     /  \
    1    8
   / \
  4  5
 /
3
```

➤ Root node 6 ka balance factor 2 hai (left subtree height 3, right subtree height 1) — AVL condition violate, rotation required.

► SINGLE LEFT ROTATION (INSERT 1,2,3 — BEFORE ROTATION)

```
1 (BF = -2) ← violation
 \
  2 (BF = -1)
   \
    3 (BF = 0)
```

➤ Insert 1,2,3 — root 1 ka balance factor -2 ho gaya (right heavy). Left rotation required.

► AFTER LEFT ROTATION (BALANCED)

```
2 (BF = 0)
 /  \
1    3 (BF = 0)
```

➤ Left rotation ke baad tree balanced ho gaya — 2 root ban gaya, 1 left, 3 right. Inorder traversal same: 1,2,3.

► GENERAL ROTATION PATTERN (NODE A WITH BF=1, INSERT IN T1)

Before Rotation:

```
A (BF=1)
 /
B (BF=0)
 / \
T1  T2
     \
     T3
```

After Rotation (Right Rotation):

```
B (BF=0)
 /  \
T1   A (BF=0)
     /  \
    T2   T3
```

➤ Jab A ka BF=1 hai aur T1 mein insertion hota hai, to right rotation karte hain — B root ban jaata hai, inorder traversal same rehta hai.

AVL Connection: Insertion ke baad agar kisi node ka balance factor 2 ya -2 ho jaye, to rotation zaroori hai. Rotation ke baad tree balanced

ho jaati hai aur inorder traversal same rehti hai — is liye BST property maintain rehti hai.

CODE EXPLANATION

» Is lecture mein actual C++ code nahi diya gaya — yeh concept-based lecture hai jo AVL insertion aur rotation ke mechanism ko explain karta hai. Code implementation next lectures mein cover ki jayegi.

EXAM FOCUS POINTS

MCQ AVL tree kisne propose ki? — Adelson-Velskii and Landis (1962). AVL = unke names ke acronym.

FORMULA Balance = Height(left) – Height(right). Empty tree height = -1.
AVL condition: $-1 \leq \text{balance} \leq 1$.

CONFUSION Height of tree = maximum level of leaves (deepest node ka level). Level 0 se start hoti hai root ki.

MCQ AVL condition har node par check hoti hai — sirf root par nahi. Agar kisi bhi node par BF = 2 ya -2 ho to tree AVL nahi.

PRACTICE Insert 1,2,3,4,5 in AVL tree — har step par balance factor calculate karein aur rotations identify karein.

PRACTICE Right rotation aur left rotation ke examples draw karein — A, B, T1, T2, T3 notation use karte hue.

AVL Tree Insertion & Rotation

AVL Definition

- BST + balance condition
- $BF = -1, 0, \text{ or } 1$
- Empty tree height = -1
- Every node must satisfy

Insertion Process

- Insert as leaf node (BST rule)
- Check balance factors
- If $BF = 2$ or $-2 \rightarrow$ rotation
- Inorder traversal unchanged

Rotation Types

- Left rotation ($BF = -2$)
- Right rotation ($BF = 2$)
- Maintains BST property
- Inorder traversal same

Key Values

- Height = max level
- Balance = left – right
- AVL condition: -1 to 1
- Search: $O(\log n)$

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the key difference between a BST and an AVL tree?

A: AVL tree ek BST hai jisme extra condition hoti hai — har node ka balance factor -1 , 0 , ya 1 hona chahiye. BST mein yeh condition nahi hoti, is liye BST degenerate ho sakta hai.

Q: Why do we need rotations in AVL trees?

A: Jab insertion ke baad kisi node ka balance factor 2 ya -2 ho jata hai (AVL condition violate), to hum rotation karte hain taake tree dobara balanced ho jaye. Rotation ke baad inorder traversal same rehta hai.

Q: What is the height of an empty tree in AVL definition?

A: Empty tree ki height -1 mani jaati hai. Is se balance factor calculation consistent rehti hai — leaf node ke liye balance factor 0 aata hai.

Q: When does a right rotation occur?

A: Right rotation tab hoti hai jab kisi node ka balance factor 2 ho (left subtree heavy) aur left child ka balance factor 0 ya 1 ho. Example: A (BF= 1), uski left subtree mein insertion.

Q: Does rotation change the inorder traversal of the tree?

A: Nahi, rotation ke baad inorder traversal wahi rehta hai. Is liye BST property maintain rehti hai — data sorted order mein hi aata hai.

► QUICK QUIZ (10 QUESTIONS)

1. What does AVL stand for? **(Adelson-Velskii and Landis)**
2. What is the height of an empty tree? **(-1)**
3. Allowed range for AVL balance factor? **(-1 , 0 , or 1)**
4. When is left rotation performed? **(When BF = -2)**
5. When is right rotation performed? **(When BF = 2)**

- | | | |
|-----|--|---------------------------|
| 6. | Does rotation change inorder traversal? | (No) |
| 7. | In AVL tree, search complexity is $O(?)$ | ($O(\log n)$) |
| 8. | What is the balance factor of a leaf node? | (0) |
| 9. | What happens if $BF = 2$ or -2 ? | (Rotation required) |
| 10. | What is the height of a tree? | (Maximum level of leaves) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **AVL Tree:** BST jisme har node ka balance factor -1, 0, ya 1 ho.
- **Balance Factor:** $\text{Height}(\text{left}) - \text{Height}(\text{right})$.
- **Height:** Maximum level of leaves (deepest node ka level).
- **Empty Tree Height:** -1 (in AVL definition).
- **Rotation:** Tree ko rearrange karne ka process jab AVL condition violate ho.

IMPORTANT CONCEPTS

- AVL tree ensures search $O(\log n)$ by keeping tree balanced.
- Insertion: new node as leaf, then check balance factors.
- If $BF = 2 \rightarrow$ right rotation (left subtree heavy).
- If $BF = -2 \rightarrow$ left rotation (right subtree heavy).
- Rotation maintains inorder traversal (BST property).
- AVL condition must hold at every node, not just root.

△ IMPORTANT FORMULAS

- **Balance Factor** = $\text{Height}(\text{left}) - \text{Height}(\text{right})$
- **Empty tree height** = -1
- **AVL condition:** $-1 \leq BF \leq 1$
- **Height of tree** = maximum level of leaves
- **Search complexity:** $O(\log n)$

FREQUENTLY ASKED TOPICS

- AVL tree definition and balance factor calculation.
- Insertion process and when rotation is needed.
- Left rotation vs right rotation — when each is used.
- Effect of rotation on inorder traversal.
- Height of empty tree = -1.
- Example: insert 1,2,3 and show rotation.

COMMON MISTAKES

- Assuming AVL condition only at root — it must be at every node.
- Forgetting empty tree height = -1 (not 0).
- Confusing left rotation with right rotation.
- Thinking rotation changes inorder traversal — it doesn't.
- Not checking balance factors after every insertion.

One-Day-Before-Exam Summary: AVL tree ek balanced BST hai jisme har node ka balance factor -1, 0, ya 1 hona chahiye. Empty tree ki height -1 mani jaati hai. Jab insertion ke baad kisi node ka $BF = 2$ ho to right rotation hoti hai, aur $BF = -2$ ho to left rotation. Rotation ke baad inorder traversal same rehta hai, is liye BST property maintain rehti hai. AVL tree search $O(\log n)$ guarantee karta hai.

Lesson 21: AVL Tree Building & Rotation Cases

Step-by-step AVL construction — single rotation vs double rotation

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap apne ghar mein samaan arrange kar rahe hain — agar aap naya samaan le kar aate hain aur ek side bohat bharr jaati hai, to aapko samaan ko dobara arrange karna padta hai (rotation). Kabhi ek simple rearrangement (single rotation) kaafi hoti hai, aur kabhi aapko do baar rearrange karna padta hai (double rotation) — jaise jab aap almari mein kapde rakhte hain aur ek side heavy ho jaati hai, to aap ko pehle kuch hata kar phir dobara arrange karna padta hai. AVL tree mein bhi yahi hota hai — kabhi single rotation kaam karti hai, kabhi double rotation ki zaroorat padti hai.

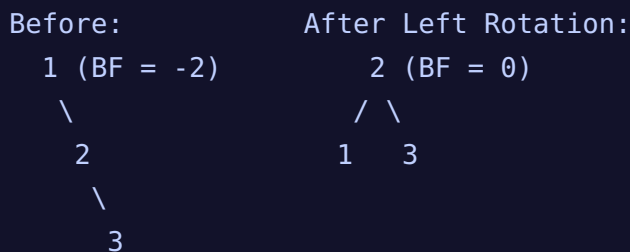
KEY TERMS

- 1 Single Rotation** — Ek hi rotation (left ya right) jo AVL tree ko balance kar deti hai. Cases 1 aur 4 mein kaam karti hai (outside insertion).
- 2 Double Rotation** — Do rotations (left-right ya right-left) jo AVL tree ko balance karti hai. Cases 2 aur 3 mein zaroori hoti hai (inside insertion).

- 3 **Outside Insertion (Case 1 & 4)** — New node left-left ya right-right position mein insert hota hai — single rotation se fix ho jata hai.
- 4 **Inside Insertion (Case 2 & 3)** — New node left-right ya right-left position mein insert hota hai — single rotation fail hoti hai, double rotation required.
- 5 **α (Alpha) Node** — Woh node jisay rotate karna hai — jiska balance factor 2 ya -2 ho jata hai (violation node).
- 6 **Inorder Traversal Preservation** — Rotation ke baad bhi inorder traversal same rehta hai — yeh BST property maintain karta hai.
- 7 **Maximum AVL Height** — $1.44 \log_2 n$ — AVL tree ki maximum height, BST degenerate (n) se bohat behtar.

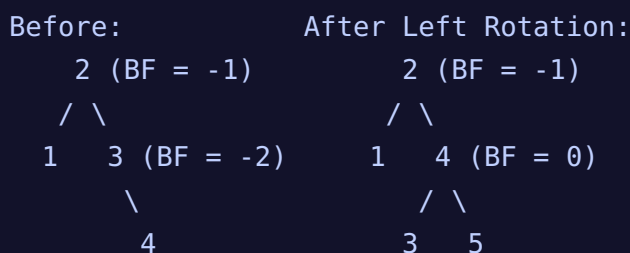
DIAGRAM TIME

► STEP 1: INSERT 1,2,3 — LEFT ROTATION



↗ Insert 1,2,3 — root 1 ka BF -2 ho gaya (right heavy). Left rotation se tree balance ho gaya.

► STEP 2: INSERT 4,5 — LEFT ROTATION ON 3



\

5

↗ Insert 4,5 — node 3 ka BF -2 ho gaya, left rotation on 3. Tree balanced ho gaya.

► STEP 3: INSERT 6 — LEFT ROTATION ON 2

Before:	After Left Rotation:
2 (BF = -2)	4 (BF = 0)
/ \	/ \
1 4 (BF = -1)	2 5 (BF = 0)
/ \	/ \ \
3 5 (BF = -1)	1 3 6
\	
6	

↗ Insert 6 — root 2 ka BF -2 ho gaya, left rotation on 2. Node 4 root ban gaya.

► STEP 4: INSERT 7 — LEFT ROTATION ON 5

Before:	After Left Rotation:
4 (BF = -1)	4 (BF = 0)
/ \	/ \
2 5 (BF = -2)	2 6 (BF = 0)
/ \ \	/ \ / \
1 3 6 (BF = -1)	1 3 5 7
\	
7	

↗ Insert 7 — node 5 ka BF -2 ho gaya, left rotation on 5. Perfect binary tree ban gaya.

► CASE 1: SINGLE RIGHT ROTATION (OUTSIDE INSERTION — LEFT-LEFT)

Before (BF=2):	After Right Rotation:
k2 (BF=2)	k1 (BF=0)
/	/ \
k1 (BF=1)	X k2 (BF=0)
/ \	/ \
X Y	Y Z

\

Z

➤ Case 1: insertion in left subtree of left child (X) — single right rotation fixes it.

► CASE 4: SINGLE LEFT ROTATION (OUTSIDE INSERTION – RIGHT-RIGHT)

Before (BF=-2): <pre> k1 (BF=-2) \ k2 (BF=-1) / \ Y Z / X </pre>	After Left Rotation: <pre> k2 (BF=0) / \ k1 Z (BF=0) / \ X Y </pre>
--	---

➤ Case 4: insertion in right subtree of right child (Z) — single left rotation fixes it.

► CASE 2: SINGLE RIGHT ROTATION FAILS (INSIDE INSERTION – LEFT-RIGHT)

Before (BF=2): <pre> k2 (BF=2) / k1 (BF=-1) / \ X Y (new) \ Z </pre>	After Single Right (still BF=-2): <pre> k1 (BF=-2) / \ X k2 (BF=-1) / \ Y Z </pre>
--	---

➤ Case 2: insertion in right subtree of left child (Y) — single right rotation fails, double rotation required (next lecture).

AVL Connection: AVL tree building mein har insertion ke baad balance factor check karte hain. Agar BF = 2 ya -2 ho to rotation required. Outside insertions (cases 1 & 4) single rotation se fix ho jati hain, inside insertions (cases 2 & 3) mein single rotation fail hoti hai — double rotation ki zaroorat padti hai (next lecture mein cover hogi).

CODE EXPLANATION

» Is lecture mein actual C++ code nahi diya gaya — yeh concept-based lecture hai jo AVL building example aur rotation cases (single vs double) ko explain karta hai. Code implementation agle lectures mein cover ki jayegi.

EXAM FOCUS POINTS

MCQ AVL tree mein root node change hoti hai — BST ke opposite jahan root wahi rehta hai jo pehle insert ho.

FORMULA Maximum height of AVL tree = $1.44 \log_2 n$ — yeh BST degenerate (n) se bohat behtar hai.

CONFUSION Single rotation cases 1 aur 4 (outside insertion) mein kaam karti hai. Cases 2 aur 3 (inside insertion) mein single rotation fail hoti hai.

MCQ Case 1 = left-left (single right rotation). Case 4 = right-right (single left rotation).

PRACTICE Insert 1,2,3,4,5,6,7 step by step — har step par balance factor calculate karein aur rotations identify karein.

PRACTICE Identify whether insertion is case 1,2,3,4 — outside (left-left/right-right) vs inside (left-right/right-left).

MIND MAP

AVL Building & Rotation Cases

Building Steps

- Insert as leaf (BST rule)
- Compute balance factors
- If $BF = 2$ or $-2 \rightarrow$ rotate
- Root changes in AVL

Case 1 & 4 (Outside)

- Case 1: left-left
- Case 4: right-right
- Single rotation works
- Right rotation (case 1)
- Left rotation (case 4)

Case 2 & 3 (Inside)

- Case 2: left-right
- Case 3: right-left
- Single rotation fails
- Double rotation needed
- Next lecture topic

Key Values

- Max AVL height: $1.44 \log_2 n$
- BST degenerate: $O(n)$
- Inorder preserved
- α = violation node

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Why does the root node change in an AVL tree but not in a BST?

A: BST mein root wahi rehta hai jo pehle insert hota hai. AVL mein jab rotation hoti hai, to root change ho sakta hai — rotation ke baad naya node root ban jaata hai jo tree ko balance karta hai.

Q: What is the difference between outside and inside insertion?

A: Outside insertion (cases 1 & 4) mein new node left-left ya right-right position mein insert hota hai — single rotation kaam karti hai. Inside insertion (cases 2 & 3) mein new node left-right ya right-left position mein insert hota hai — single rotation fail hoti hai, double rotation zaroori hai.

Q: What is the maximum height of an AVL tree?

A: AVL tree ki maximum height $1.44 \log_2 n$ hoti hai. Iska matlab hai ki 100,000 items ke liye maximum 24-25 levels — BST degenerate (n) se bohat behtar.

Q: What happens if we apply a single rotation to case 2 (inside insertion)?

A: Single rotation case 2 mein fail ho jaati hai. Example mein humne dekha ke k2 par right rotation karne ke baad bhi k1 ka BF -2 reh gaya — tree still unbalanced hai. Is liye double rotation required hai.

Q: Does rotation change the inorder traversal of the tree?

A: Nahi, rotation ke baad inorder traversal wahi rehta hai. Is liye BST property maintain rehti hai — data sorted order mein hi aata hai. Yehi AVL rotation ka fundamental principle hai.

► QUICK QUIZ (10 QUESTIONS)

- | | | |
|----|--|-------------------------|
| 1. | What is the maximum height of an AVL tree? | ($1.44 \log_2 n$) |
| 2. | Which rotation fixes case 1 (left-left)? | (Single right rotation) |
| 3. | Which rotation fixes case 4 (right-right)? | (Single left rotation) |
| 4. | What happens in case 2 (left-right)? | (Single rotation fails) |

- | | |
|--|-------------------------|
| 5. What happens in case 3 (right-left)? | (Single rotation fails) |
| 6. Does root change in AVL tree? | (Yes) |
| 7. Does rotation change inorder traversal? | (No) |
| 8. What is α (alpha) in rotation context? | (Node to be rotated) |
| 9. Case 1 is also called ___ insertion? | (Outside) |
| 10. Case 2 is also called ___ insertion? | (Inside) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Single Rotation:** Ek hi rotation (left ya right) — cases 1 & 4 (outside insertion) mein kaam karti hai.
- **Double Rotation:** Do rotations (left-right ya right-left) — cases 2 & 3 (inside insertion) mein zaroori.
- **Outside Insertion:** left-left (case 1) ya right-right (case 4) — single rotation works.
- **Inside Insertion:** left-right (case 2) ya right-left (case 3) — single rotation fails.
- **α (Alpha):** Woh node jisay rotate karna hai — $BF = 2$ ya -2 .

IMPORTANT CONCEPTS

- AVL tree building: insert $\rightarrow$ check balance $\rightarrow$ rotate if needed.
- Root changes in AVL (unlike BST where root stays same).
- Inorder traversal preserved after rotation — BST property maintained.
- Maximum AVL height: $1.44 \log_2 n$ — much better than BST degenerate (n).
- Outside insertion (cases 1,4): single rotation sufficient.
- Inside insertion (cases 2,3): single rotation fails, double rotation required.

△ IMPORTANT FORMULAS

- **Maximum AVL height** = $1.44 \log_2 n$
- **Balance Factor** = $\text{Height}(\text{left}) - \text{Height}(\text{right})$

- **AVL condition:** $-1 \leq BF \leq 1$
- **Case 1:** left-left → single right rotation
- **Case 4:** right-right → single left rotation
- **Case 2:** left-right → double rotation (left + right)
- **Case 3:** right-left → double rotation (right + left)

FREQUENTLY ASKED TOPICS

- AVL tree building example (insert 1-7 step by step).
- Single rotation vs double rotation — when each is used.
- Outside vs inside insertion — identify cases 1-4.
- Maximum height of AVL tree = $1.44 \log_2 n$.
- Root change in AVL vs BST.
- Inorder traversal preservation during rotation.

COMMON MISTAKES

- Using single rotation for inside insertion (cases 2,3) — yeh kaam nahi karti.
- Forgetting that AVL root can change after rotation.
- Thinking rotation changes inorder traversal — it doesn't.
- Confusing case 1 (left-left) with case 2 (left-right).
- Not checking balance factors after every insertion

Lesson 22: Double Rotation & AVL Insert Code

Left-right & right-left double rotations — complete AVL insertion implementation

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap apne ghar mein samaan arrange kar rahe hain — agar aap naya samaan le kar aate hain aur ek side bohat bharr jaati hai, to aapko samaan ko dobara arrange karna padta hai (rotation). Kabhi ek simple rearrangement (single rotation) kaafi hoti hai, aur kabhi aapko do baar rearrange karna padta hai (double rotation) — jaise jab aap almari mein kapde rakhte hain aur ek side heavy ho jaati hai, to aap ko pehle kuch hata kar phir dobara arrange karna padta hai. AVL tree mein bhi yahi hota hai — kabhi single rotation kaam karti hai, kabhi double rotation ki zaroorat padti hai.

KEY TERMS

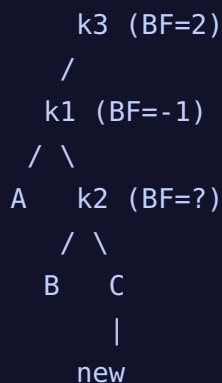
- 1 Left-Right Double Rotation** — Case 2 (inside insertion — left-right) ko fix karne ke liye pehle left rotation phir right rotation.
- 2 Right-Left Double Rotation** — Case 3 (inside insertion — right-left) ko fix karne ke liye pehle right rotation phir left rotation.

- 3 **avlInsert Function** — C++ function jo AVL tree mein node insert karta hai aur balance maintain karta hai — recursive implementation.
- 4 **Height Difference (htdiff)** — $\text{height}(\text{left}) - \text{height}(\text{right})$ — agar 2 ho to rotation required.
- 5 **Outside Insertion Detection** — $\text{info} < \text{root} \rightarrow \text{getLeft}() \rightarrow \text{getInfo}()$ → case 1 (single rotation).
- 6 **Inside Insertion Detection** — $\text{info} > \text{root} \rightarrow \text{getLeft}() \rightarrow \text{getInfo}()$ → case 2 (double rotation).
- 7 **Height Update** — After insertion: $\text{root} \rightarrow \text{setHeight}(\max(\text{leftHt}, \text{rightHt}) + 1)$.

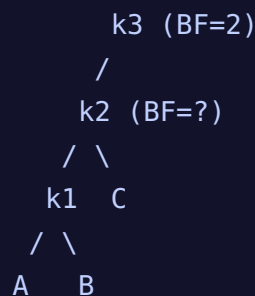
DIAGRAM TIME

► CASE 2: LEFT-RIGHT DOUBLE ROTATION (STEP 1 – LEFT ROTATION)

Before:



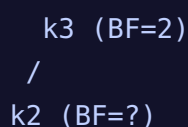
After Left Rotation:



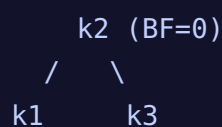
↗ Step 1: Left rotation between k1 and k2 — k2 up, k1 down. Inorder preserved: A,k1,B,k2,C,k3,D.

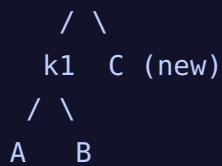
► CASE 2: LEFT-RIGHT DOUBLE ROTATION (STEP 2 – RIGHT ROTATION)

After Left Rotation:



After Right Rotation (Final):

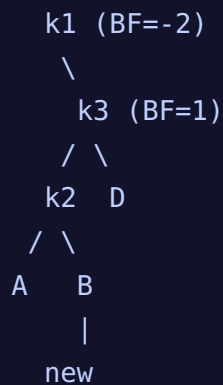




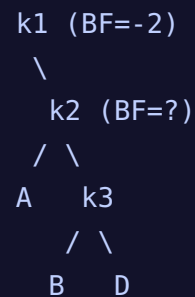
↗ Step 2: Right rotation between k_2 and k_3 — k_2 becomes root. Tree balanced, inorder still $A, k_1, B, k_2, C, k_3, D$.

► CASE 3: RIGHT-LEFT DOUBLE ROTATION (STEP 1 — RIGHT ROTATION)

Before:



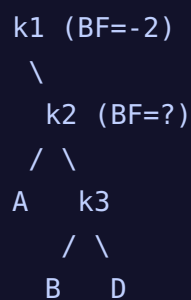
After Right Rotation:



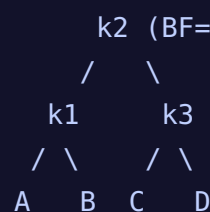
↗ Step 1: Right rotation between k_2 and k_3 — k_2 up, k_3 down. Inorder preserved.

► CASE 3: RIGHT-LEFT DOUBLE ROTATION (STEP 2 — LEFT ROTATION)

After Right Rotation:

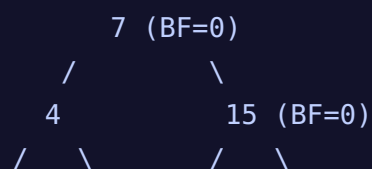


After Left Rotation (Final):



↗ Step 2: Left rotation between k_1 and k_2 — k_2 becomes root. Tree balanced.

► FINAL AVL TREE AFTER INSERTING 1-16 (BALANCED)



```

      2      6      14      16 (BF=0)
    /  \    /    /
   1   3  5   13

```

➤ Sorted numbers 1-16 insert karne ke baad final AVL tree — balanced with max height ~3.

AVL Connection: Double rotation do single rotations ka combination hai. Case 2 (left-right) mein pehle left rotation phir right rotation. Case 3 (right-left) mein pehle right rotation phir left rotation. Dono cases mein k2 (inner node) root ban jaata hai aur tree balanced ho jaati hai. Inorder traversal hamesha preserve rehti hai.

CODE EXPLANATION

```

/* AVL Insert function */
TreeNode<int>* avlInsert(TreeNode<int>* root, int info)
{
    if (info < root->getInfo()) {
        root->setLeft(avlInsert(root->getLeft(), info));
        int htdiff = height(root->getLeft()) - height(root->getRight());
        if (htdiff == 2)
            if (info < root->getLeft()->getInfo()) // outside: case 1
                root = singleRightRotation(root);
            else // inside: case 2
                root = doubleLeftRightRotation(root);
    }
    else if (info > root->getInfo()) {
        root->setRight(avlInsert(root->getRight(), info));
        int htdiff = height(root->getRight()) - height(root->getLeft());
        if (htdiff == 2)
            if (info > root->getRight()->getInfo()) // outside: case 4
                root = singleLeftRotation(root);
            else // inside: case 3
                root = doubleRightLeftRotation(root);
    }
    // else info already exists (duplicate)
    int ht = max(height(root->getLeft()), height(root->getRight()));
    root->setHeight(ht + 1);
}

```

```
return root;  
}
```

- 1 **Recursive Insertion** — avlInsert recursively call hoti hai left ya right subtree mein. Return value se root update hota hai.
- 2 **Height Difference Check** — $htdiff = height(left) - height(right)$. Agar 2 ho to rotation required (left subtree heavy).
- 3 **Case Detection (Left Subtree)** — $info < root->getLeft()->getInfo()$ → outside (case 1) → singleRightRotation. Else → inside (case 2) → doubleLeftRightRotation.
- 4 **Case Detection (Right Subtree)** — $info > root->getRight()->getInfo()$ → outside (case 4) → singleLeftRotation. Else → inside (case 3) → doubleRightLeftRotation.
- 5 **Height Update** — $\max(height(left), height(right)) + 1$ — har node ki height update hoti hai after insertion.
- 6 **Root Return** — function root return karta hai (jo rotation ke baad change ho sakta hai).

EXAM FOCUS POINTS

MCQ Left-right double rotation = case 2 (inside insertion — left-right).
Pehle left rotation phir right rotation.

MCQ Right-left double rotation = case 3 (inside insertion — right-left).
Pehle right rotation phir left rotation.

FORMULA Double rotation mein k2 (inner node) root ban jaata hai — k1 aur k3 uske left aur right children.

CONFUSION Single rotation cases 1 & 4 mein kaam karti hai. Cases 2 & 3 mein double rotation zaroori hai.

PRACTICE

avlInsert code ko trace karein — insert 15 in tree with 1-7,16 — identify left-right double rotation.

PRACTICE

Insert 14 in tree — right-left double rotation identify karein aur diagram draw karein.

MIND MAP**Double Rotation & AVL Insert Code****Double Rotation Cases**

- Case 2: left-right
- Case 3: right-left
- Two single rotations
- k2 becomes root
- Inorder preserved

avlInsert Algorithm

- Recursive insertion
- Compute htdiff
- If htdiff == 2 → rotate
- Update height
- Return root

Case Detection

- Left subtree: info < left child → single
- Left subtree: info > left child → double
- Right subtree: info > right child → single
- Right subtree: info < right child → double

Height Management

- $\text{height}(\text{node}) = \max(\text{leftHt}, \text{rightHt}) + 1$
- Empty node height = -1
- $\text{htdiff} = \text{left} - \text{right}$
- $\text{htdiff} = 2 \rightarrow \text{rotation}$

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Why does single rotation fail in cases 2 and 3?

A: Cases 2 aur 3 mein new node inside (left-right ya right-left) insert hota hai. Single rotation ke baad bhi ek subtree (Y) apni jagah par rehti hai, jis ki wajah se tree still unbalanced rehti hai. Double rotation se Y ko properly distribute kiya ja sakta hai.

Q: What is the order of rotations in left-right double rotation?

A: Pehle left rotation between $k1$ and $k2$ (inner node up), phir right rotation between $k2$ and $k3$ ($k2$ root). Final mein $k2$ root ban jaata hai aur tree balanced ho jaati hai.

Q: What is the order of rotations in right-left double rotation?

A: Pehle right rotation between $k2$ and $k3$ (inner node up), phir left rotation between $k1$ and $k2$ ($k2$ root). Final mein $k2$ root ban jaata hai.

Q: How does avlInsert detect which rotation to perform?

A: avlInsert info value ko compare karta hai. Left subtree insertion mein: agar $\text{info} < \text{left child ka info} \rightarrow \text{outside (single)}$. Agar $\text{info} > \text{left child ka info} \rightarrow \text{inside (double)}$. Right subtree mein opposite.

Q: Why do we update height after insertion in avlInsert?

A: Height update zaroori hai kyunki new node insertion se subtree ki height change ho jaati hai. Height values balance factor calculation ke liye use hoti hain. $root \rightarrow setHeight(\max(leftHt, rightHt) + 1)$.

► QUICK QUIZ (10 QUESTIONS)

1. Left-right double rotation fixes which case? **(Case 2)**
2. Right-left double rotation fixes which case? **(Case 3)**
3. Which node becomes root in double rotation? **(k2 - inner node)**
4. First rotation in left-right double rotation? **(Left rotation)**
5. First rotation in right-left double rotation? **(Right rotation)**
6. What is htdiff in avlInsert? **(height(left) – height(right))**
7. When is single rotation performed? **(Cases 1 and 4)**
8. When is double rotation performed? **(Cases 2 and 3)**
9. How is height updated after insertion? **(max(left,right) + 1)**
10. What does avlInsert return? **(Root of the tree)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Left-Right Double Rotation:** Case 2 fix — pehle left rotation phir right rotation. k2 becomes root.
- **Right-Left Double Rotation:** Case 3 fix — pehle right rotation phir left rotation. k2 becomes root.
- **avlInsert:** Recursive function jo AVL tree mein node insert karta hai aur balance maintain karta hai.
- **htdiff:** height(left) – height(right) — agar 2 ho to rotation required.

IMPORTANT CONCEPTS

- Double rotation = two single rotations in specific order.
- Case 2 (left-right): left rotation on k1-k2, then right rotation on k2-k3.

- Case 3 (right-left): right rotation on k2-k3, then left rotation on k1-k2.
- Inorder traversal preserved after double rotation.
- avlInsert uses info comparison to detect outside vs inside insertion.
- Height updated after every insertion: $\max(\text{left}, \text{right}) + 1$.
- Final AVL tree with 1-16 is perfectly balanced.

△ IMPORTANT FORMULAS

- **htdiff** = $\text{height}(\text{left}) - \text{height}(\text{right})$
- **Case 2 (left-right)**: Left rotation + Right rotation
- **Case 3 (right-left)**: Right rotation + Left rotation
- **Height update** = $\max(\text{height}(\text{left}), \text{height}(\text{right})) + 1$
- **AVL condition**: $-1 \leq \text{BF} \leq 1$
- **Max AVL height**: $1.44 \log_2 n$

FREQUENTLY ASKED TOPICS

- Left-right double rotation (case 2) — step by step.
- Right-left double rotation (case 3) — step by step.

Lesson 23: Rotation Code & Deletion in AVL Tree

Single & double rotation implementations — deletion cases in AVL

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap apne ghar mein samaan arrange kar rahe hain — agar aap kuch samaan hataate hain (deletion) to ek side khali ho jaati hai aur balance bigad jaata hai. Aapko samaan ko dobara arrange karna padta hai (rotation). Deletion mein yeh process insertion se zyada complicated hai — kyunki ek node hataane ke baad aapko har level par balance check karna padta hai aur kabhi kabhi multiple rotations ki zaroorat padti hai. Jaise agar aap almari se ek kapda nikaalte hain to poori almari ka balance bigad sakta hai.

KEY TERMS

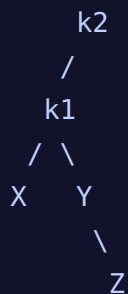
- 1 singleRightRotation** — Function jo k2 node par right rotation perform karta hai. k1 (left child) becomes root, Y moves to k2's left.
- 2 singleLeftRotation** — Function jo k1 node par left rotation perform karta hai. k2 (right child) becomes root, Y moves to k1's right.
- 3 doubleRightLeftRotation** — Case 3 fix: pehle right rotation (k2-k3), phir left rotation (k1-k2). k2 becomes root.

- 4 **doubleLeftRightRotation** — Case 2 fix: pehle left rotation (k1-k2), phir right rotation (k2-k3). k2 becomes root.
- 5 **Deletion in AVL** — Node delete karna (BST style) phir balance check karna up to root — multiple rotations ho sakti hain.
- 6 **Case 1a/1b** — Parent balance 0 tha, deletion left/right subtree mein — balance change karo, stop. No rotation needed.
- 7 **Case 2a/2b** — Parent balance 1/-1 tha, deletion left/right mein — balance zero ho jati hai, continue up to root.
- 8 **height Function** — Node ki height return karta hai. NULL node ke liye -1 return (empty tree definition).

DIAGRAM TIME

► SINGLE RIGHT ROTATION (CODE VISUALIZATION)

Before:



After:



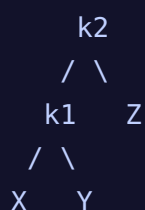
➤ k1 root ban jaata hai, Y (k1 ka right) k2 ka left ban jaata hai. Inorder: X,k1,Y,k2,Z — same.

► SINGLE LEFT ROTATION (CODE VISUALIZATION)

Before:



After:

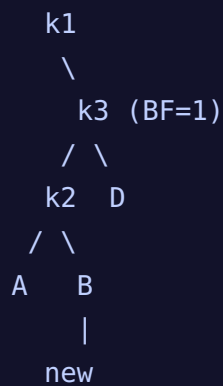


/
X

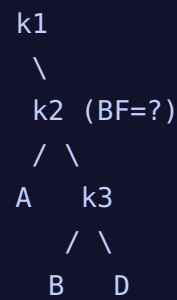
➤ *k2 root ban jaata hai, Y (k2 ka left) k1 ka right ban jaata hai. Inorder same.*

► DOUBLE RIGHT-LEFT ROTATION (STEP 1 – RIGHT ROTATION)

Before:



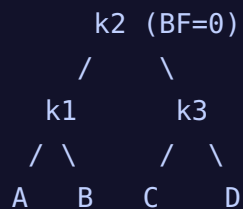
After Right:



➤ *Step 1: Right rotation between k2 and k3 — k2 up, k3 down.*

► DOUBLE RIGHT-LEFT ROTATION (STEP 2 – LEFT ROTATION)

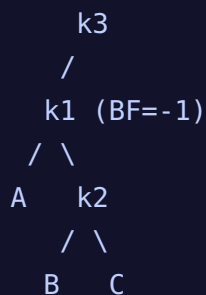
After Left Rotation (Final):



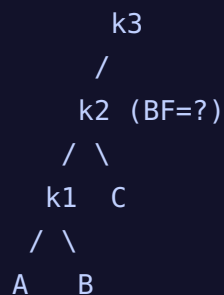
➤ *Step 2: Left rotation between k1 and k2 — k2 becomes root. Tree balanced.*

► DOUBLE LEFT-RIGHT ROTATION (STEP 1 – LEFT ROTATION)

Before:



After Left:

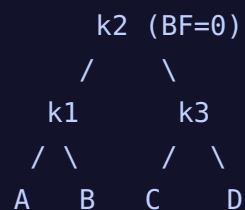


|
new

➤ Step 1: Left rotation between k_1 and k_2 — k_2 up, k_1 down.

► DOUBLE LEFT-RIGHT ROTATION (STEP 2 — RIGHT ROTATION)

After Right Rotation (Final):



➤ Step 2: Right rotation between k_2 and k_3 — k_2 becomes root. Tree balanced.

► DELETION WORST CASE — MULTIPLE ROTATIONS



➤ Node A delete karne par har level par rotation ho sakti hai — deletion mein up to $\log_2 N$ rotations.

AVL Connection: Insertion mein at most one single ya double rotation hoti hai. Deletion mein multiple rotations ho sakti hain — up to $\log_2 N$. Is liye deletion more complex hai. Case 1a/1b mein no rotation, case 2a/2b mein balance change and continue.

CODE EXPLANATION

► SINGLE RIGHT ROTATION CODE


```

TreeNode<int>* singleRightRotation(TreeNode<int>* k2) {
    if (k2 == NULL) return NULL;
    TreeNode<int>* k1 = k2->getLeft();
    k2->setLeft(k1->getRight()); // Y moves to k2's left
    k1->setRight(k2);           // k2 becomes k1's right
    int h = Max(height(k2->getLeft()), height(k2->getRight()));
    k2->setHeight(h + 1);
    h = Max(height(k1->getLeft()), k2->getHeight());
    k1->setHeight(h + 1);
    return k1;                 // new root
}

```

- 1 **k1 = k2->getLeft()** — k1 (left child) new root banega.
- 2 **k2->setLeft(k1->getRight())** — Y (k1 ka right) k2 ka left ban jaata hai.
- 3 **k1->setRight(k2)** — k2 k1 ka right child ban jaata hai.
- 4 **Height updates** — Pehle k2 ki height update, phir k1 ki. Order important hai.

► SINGLE LEFT ROTATION CODE

```

TreeNode<int>* singleLeftRotation(TreeNode<int>* k1) {
    if (k1 == NULL) return NULL;
    TreeNode<int>* k2 = k1->getRight();
    k1->setRight(k2->getLeft()); // Y moves to k1's right
    k2->setLeft(k1);           // k1 becomes k2's left
    int h = Max(height(k1->getLeft()), height(k1->getRight()));
    k1->setHeight(h + 1);
    h = Max(height(k2->getRight()), k1->getHeight());
    k2->setHeight(h + 1);
    return k2;                 // new root
}

```

- 1 **k2 = k1->getRight()** — k2 (right child) new root banega.
- 2 **k1->setRight(k2->getLeft())** — Y (k2 ka left) k1 ka right ban jaata hai.
- 3 **k2->setLeft(k1)** — k1 k2 ka left child ban jaata hai.

► DOUBLE RIGHT-LEFT ROTATION

```
TreeNode<int>* doubleRightLeftRotation(TreeNode<int>* k1) {
    if (k1 == NULL) return NULL;
    k1->setRight(singleRightRotation(k1->getRight())); // Step 1
    return singleLeftRotation(k1); // Step 2
}
```

1 **singleRightRotation on k1->getRight()** — Pehle right rotation between k2 and k3.

2 **singleLeftRotation(k1)** — Phir left rotation between k1 and k2. k2 becomes root.

► DOUBLE LEFT-RIGHT ROTATION

```
TreeNode<int>* doubleLeftRightRotation(TreeNode<int>* k3) {
    if (k3 == NULL) return NULL;
    k3->setLeft(singleLeftRotation(k3->getLeft())); // Step 1
    return singleRightRotation(k3); // Step 2
}
```

1 **singleLeftRotation on k3->getLeft()** — Pehle left rotation between k1 and k2.

2 **singleRightRotation(k3)** — Phir right rotation between k2 and k3. k2 becomes root.

► HEIGHT FUNCTION

```
int height(TreeNode<int>* node) {
    if (node != NULL) return node->getHeight();
    return -1; // empty tree height = -1
}
```

1 **NULL check** — Empty tree ki height -1 (AVL definition).

EXAM FOCUS POINTS

MCQ singleRightRotation — k2 as argument, k1 becomes root. Y moves to k2's left.

MCQ singleLeftRotation — k1 as argument, k2 becomes root. Y moves to k1's right.

MCQ doubleRightLeftRotation — pehle singleRightRotation, phir singleLeftRotation.

MCQ doubleLeftRightRotation — pehle singleLeftRotation, phir singleRightRotation.

FORMULA Deletion worst case: up to $\log_2 N$ rotations (multiple rotations possible).

CONFUSION Insertion: at most one single OR one double rotation. Deletion: multiple rotations possible.

PRACTICE Trace singleRightRotation on a sample tree — identify X, Y, Z subtrees.

PRACTICE Deletion cases 1a, 1b, 2a, 2b — identify which rotation is needed.

MIND MAP

Rotation Code & Deletion in AVL

Single Rotations

- singleRightRotation(k2)
- k1 becomes root
- $Y \rightarrow k2$'s left
- $k2 \rightarrow k1$'s right

- Inorder preserved

Double Rotations

- Right-Left: right + left
- Left-Right: left + right
- k2 becomes root
- Two single rotations
- Inorder preserved

Deletion Cases

- Case 1a: parent BF=0, delete left → change balance, stop
- Case 1b: parent BF=0, delete right → change balance, stop
- Case 2a: parent BF=1, delete left → balance becomes 0, continue
- Case 2b: parent BF=-1, delete right → balance becomes 0, continue

Key Facts

- Insertion: ≤ 1 rotation
- Deletion: $\leq \log_2 N$ rotations
- $\text{height}(\text{NULL}) = -1$
- Inorder always preserved

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the difference between insertion and deletion rotations in AVL tree?

A: Insertion mein at most one single ya double rotation hoti hai. Deletion mein multiple rotations ho sakti hain — worst case mein $\log_2 N$ rotations tak. Is liye deletion more complex hai.

Q: What does singleRightRotation do with subtree Y?

A: singleRightRotation mein Y (k1 ka right subtree) k2 ka left subtree ban jaata hai. Inorder traversal X, k1, Y, k2, Z preserve rehti hai.

Q: What does singleLeftRotation do with subtree Y?

A: singleLeftRotation mein Y (k2 ka left subtree) k1 ka right subtree ban jaata hai. Inorder traversal X, k1, Y, k2, Z preserve rehti hai.

Q: Why is height of empty tree defined as -1?

A: Height of empty tree -1 hai taake balance factor calculation consistent rahe. Leaf node ke liye left aur right empty subtrees ki height -1 hoti hai, is se balance factor 0 aata hai.

Q: What happens in deletion case 1a?

A: Case 1a mein parent node ka balance 0 tha aur node left subtree mein delete hui. Parent ka balance change ho kar -1 ho jaata hai aur stop — no rotation needed. Higher nodes affected nahi hote.

► QUICK QUIZ (10 QUESTIONS)

- | | | |
|-------|---|------|
| 1. | What is the argument of singleRightRotation? | (k2) |
| <hr/> | | |
| 2. | What is the argument of singleLeftRotation? | (k1) |
| <hr/> | | |
| 3. | Which node becomes root in singleRightRotation? | (k1) |

<

Lesson 24: Deletion Cases & Expression Trees

AVL deletion — 5 cases + binary trees in compilers & databases

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap apne ghar mein samaan arrange kar rahe hain — agar aap kuch samaan hataate hain (deletion) to ek side khali ho jaati hai aur balance bigad jaata hai. Kabhi aapko bas ek chhota sa adjustment karna padta hai (case 1a/1b), kabhi aapko upar tak check karna padta hai (case 2a/2b), kabhi single rotation kaafi hoti hai (case 3a/5a), aur kabhi double rotation ki zaroorat padti hai (case 4a). Deletion insertion se zyada complicated hai kyunki aapko har level par balance check karna padta hai.

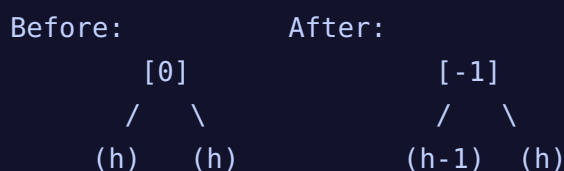
KEY TERMS

- Case 1a** — Parent $BF=0$, deletion in left subtree → change balance to -1, stop. No rotation needed.
- Case 1b** — Parent $BF=0$, deletion in right subtree → change balance to 1, stop. No rotation needed.
- Case 2a** — Parent $BF=1$, deletion in left subtree → balance becomes 0, continue up the tree.

- 4 **Case 2b** — Parent $BF=-1$, deletion in right subtree $\rightarrow$ balance becomes 0, continue up the tree.
- 5 **Case 3a** — Parent $BF=-1$, delete left, right subtree balanced $\rightarrow$ single rotation, stop.
- 6 **Case 4a** — Parent $BF=-1$, delete left, right subtree unbalanced $\rightarrow$ double rotation, continue up.
- 7 **Case 5a** — Parent $BF=-1$, delete left, right subtree unbalanced (different) $\rightarrow$ single rotation, continue up.
- 8 **Expression Tree** — Binary tree jo arithmetic expressions ko represent karta hai — leaves operands, internal nodes operators.
- 9 **Parse Tree** — Compiler/database query ka tree representation — grammar rules ke mutabiq structure dikhata hai.
- 10 **Common Subexpression Elimination** — Compiler optimization technique jo duplicate subtrees ko identify karke graph mein convert karti hai.

DIAGRAM TIME

► CASE 1A: PARENT $BF=0$, DELETE LEFT $\rightarrow$ NO ROTATION



↗ Balance 0 se -1 ho jaati hai — AVL condition still satisfied, no rotation needed. Stop.

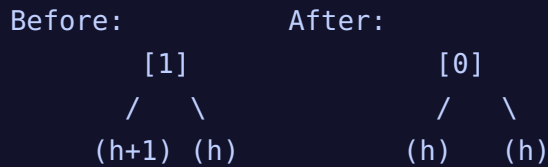
► CASE 1B: PARENT $BF=0$, DELETE RIGHT $\rightarrow$ NO ROTATION





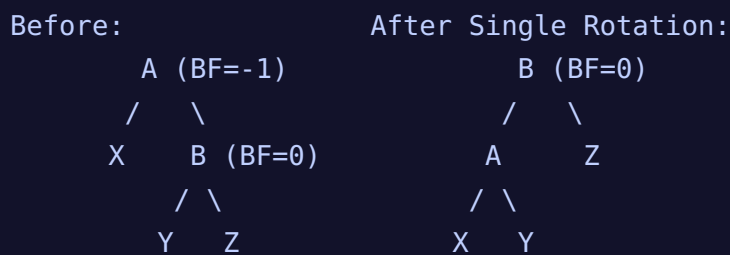
↗ Balance 0 se 1 ho jaati hai — no rotation needed. Stop.

► CASE 2A: PARENT BF=1, DELETE LEFT → CONTINUE UP



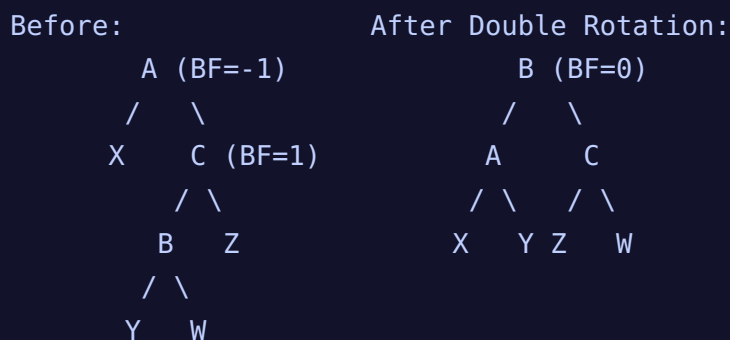
↗ Balance 1 se 0 ho jaati hai — continue up the tree, check higher nodes.

► CASE 3A: PARENT BF=-1, DELETE LEFT, RIGHT BALANCED → SINGLE ROTATION, STOP



↗ Single rotation at B — tree balanced, stop. No effect on higher nodes.

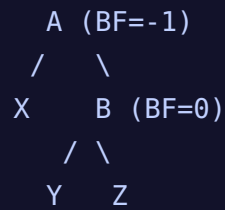
► CASE 4A: PARENT BF=-1, DELETE LEFT, RIGHT UNBALANCED → DOUBLE ROTATION, CONTINUE



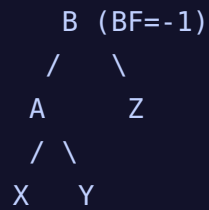
↗ Double rotation at B — k2 becomes root. May affect higher nodes, so continue up.

► CASE 5A: PARENT BF=-1, DELETE LEFT, RIGHT UNBALANCED (DIFFERENT) → SINGLE ROTATION, CONTINUE

Before:



After Single Rotation:



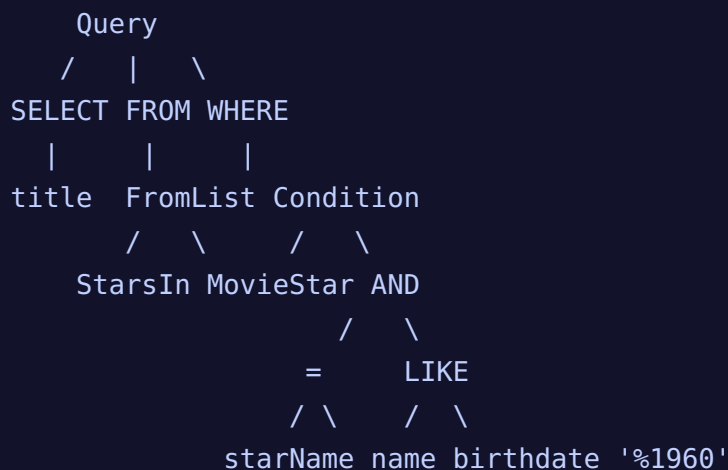
↗ Single rotation — may affect higher nodes, so continue up.

► EXPRESSION TREE: $(A + B * C) + ((D * E + F) * G)$



↗ Expression tree — leaves are operands (a,b,c...), internal nodes are operators (+,*).

► PARSE TREE FOR SQL QUERY



↗ SQL query parse tree — database engine query processing ke liye use karta hai.

► COMMON SUBEXPRESSION ELIMINATION (OPTIMIZATION)

Before (Tree):



After (Graph):



➤ $d * e + f$ common subexpression ko ek baar calculate karke reuse kiya gaya — tree graph mein convert ho gaya.

AVL Connection: Deletion in AVL tree mein 5 cases hain. Case 1a/1b mein no rotation. Case 2a/2b mein balance change and continue. Case 3a mein single rotation and stop. Case 4a mein double rotation and continue. Case 5a mein single rotation and continue. Deletion insertion se zyada complex hai — multiple rotations ho sakti hain.

CODE EXPLANATION

» Is lecture mein actual C++ code nahi diya gaya — yeh concept-based lecture hai jo AVL deletion cases aur expression trees ko explain karta hai. Deletion code implementation text book mein available hai aur aap khud practice kar sakte hain.

EXAM FOCUS POINTS

MCQ Case 1a: Parent BF=0, delete left → change balance to -1, stop. No rotation.

MCQ Case 3a: Parent BF=-1, delete left, right balanced → single rotation, stop.

MCQ

Case 4a: Parent $BF=-1$, delete left, right unbalanced $\rightarrow$ double rotation, continue.

CONFUSION

Case 2a/2b mein rotation nahi hoti — sirf balance change hota hai aur upar continue karte hain.

FORMULA

Deletion worst case: up to $\log_2 N$ rotations (multiple rotations possible at each level).

MCQ

Expression trees — leaves are operands, internal nodes are operators. Compilers mein use hoti hain.

MCQ

Parse trees — compilers, databases (SQL query processing), speech recognition mein use hoti hain.

PRACTICE

Given a tree and a deletion, identify which case applies and what rotation is needed.

MIND MAP

Deletion Cases & Expression Trees

Deletion Cases Summary

- Case 1a/1b: $BF=0 \rightarrow$ change balance, stop
- Case 2a/2b: $BF=\pm 1 \rightarrow$ balance becomes 0, continue
- Case 3a: single rotation, stop
- Case 4a: double rotation, continue
- Case 5a: single rotation, continue

Key Insights

- Deletion $>$ insertion complexity
- Multiple rotations possible
- Up to $\log_2 N$ rotations
- Recursion used for backtracking

Expression Trees

- Arithmetic expressions
- Leaves = operands
- Internal nodes = operators
- Used in compilers

Parse Trees

- Compilers (C++, Java)
- SQL query processing
- Speech recognition
- Common subexpression elimination

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the main difference between insertion and deletion in AVL trees?

A: Insertion mein at most one single ya double rotation hoti hai. Deletion mein multiple rotations ho sakti hain — worst case mein $\log_2 N$ rotations tak. Is liye deletion more complex hai aur cases 1a se 5a tak define kiye gaye hain.

Q: What happens in case 1a of deletion?

A: Case 1a mein parent node ka balance 0 tha aur node left subtree mein delete hui. Parent ka balance -1 ho jaata hai aur stop — no rotation needed. Higher nodes affected nahi hote.

Q: When does case 3a occur and what action is taken?

A: Case 3a mein parent $BF=-1$, deletion left subtree mein, right subtree balanced hoti hai. Single rotation perform karte hain aur stop — no effect on higher nodes.

Q: When does case 4a occur and what action is taken?

A: Case 4a mein parent $BF=-1$, deletion left subtree mein, right subtree unbalanced hoti hai. Double rotation perform karte hain aur continue up the tree — higher nodes bhi check karte hain.

Q: What are expression trees and where are they used?

A: Expression trees arithmetic expressions ko represent karte hain — leaves operands hain, internal nodes operators hain. Compilers, spreadsheets, aur parse trees ki form mein databases mein use hoti hain.

Q: What is common subexpression elimination?

A: Compiler optimization technique hai jo duplicate subtrees ko identify karta hai. Common subexpression ko ek baar calculate karke reuse kiya jaata hai — tree ko graph mein convert kar deta hai efficiency ke liye.

► QUICK QUIZ (10 QUESTIONS)

- | | | |
|----|--|-----------------------------|
| 1. | Case 1a: Parent $BF=0$, delete left → what action? | (Change balance, stop) |
| 2. | Case 2a: Parent $BF=1$, delete left → what action? | (Change balance, continue) |
| 3. | Case 3a: Parent $BF=-1$, delete left, right balanced → what action? | (Single rotation, stop) |
| 4. | Case 4a: Parent $BF=-1$, delete left, right unbalanced → what action? | (Double rotation, continue) |
| 5. | Case 5a: Parent $BF=-1$, delete left, right unbalanced → what action? | (Single rotation, continue) |
| 6. | In expression trees, what are leaves? | (Operands) |
| 7. | In expression trees, what are internal nodes? | (Operators) |
| 8. | What does common subexpression elimination create? | (Graph) |

9. Deletion worst case rotations = ?

($\log_2 N$)

10. Parse trees are used in which optimization?

(Common subexpression elimination)

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Case 1a/1b:** Parent BF=0, delete left/right → change balance, stop. No rotation.
- **Case 2a/2b:** Parent BF= ± 1 , delete left/right → balance becomes 0, continue up.
- **Case 3a:** Parent BF=-1, delete left, right balanced → single rotation, stop.
- **Case 4a:** Parent BF=-1, delete left, right unbalanced → double rotation, continue.
- **Case 5a:** Parent BF=-1, delete left, right unbalanced → single rotation, continue.
- **Expression Tree:** Binary tree for arithmetic expressions — leaves operands, internal nodes operators.
- **Parse Tree:** Tree representation of grammar rules — used in compilers, databases, speech recognition.

IMPORTANT CONCEPTS

- Deletion in AVL is more complex than insertion — multiple rotations possible.
- Recursion used for backtracking — work pushed on stack, done while returning.
- Expression trees used in compilers for parsing and optimization.
- Common subexpression elimination converts tree to graph for efficiency.
- Parse trees used in SQL query processing and speech recognition.
- Symmetrical cases exist (3b, 4b, 5b) for deletion in right subtree.

△ IMPORTANT FORMULAS

- **Deletion worst case** = up to $\log_2 N$ rotations

- **Insertion rotations** = at most 1 single or 1 double
- **AVL condition**

Lesson 25: Expression Trees & Huffman Encoding

Building expression trees from postfix — Huffman compression using binary trees

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap ek complex mathematical expression ko samajhna chahte hain — expression tree us expression ko visual form mein represent karta hai, jaise aap algebra ka sawaal tree ki shakal mein likhte hain. Aur Huffman Encoding jaise aap apni kitaab ko compress karke chhota packet bana kar bhejte hain — jis mein zyada aane wale alphabets ko chhota code milta hai aur kam aane walon ko bara code. Is tarah total size kam ho jaata hai aur transmission fast ho jaati hai.

KEY TERMS

- 1 Expression Tree** — Binary tree jo arithmetic expressions ko represent karta hai — leaves operands, internal nodes operators.
- 2 Postfix to Expression Tree** — Algorithm jo postfix expression ko tree mein convert karta hai — stack use karta hai.
- 3 Inorder with Parenthesis** — inorder traversal jo parenthesis add karta hai — expression precedence clear hoti hai.

- 4 **Huffman Encoding** — Data compression algorithm jo binary tree use karta hai — frequent characters ko short codes, rare ko long codes.
- 5 **Frequency Table** — Har character ki occurrence count — Huffman tree building ka first step.
- 6 **Huffman Tree** — Binary tree jisme leaves characters hain aur internal nodes frequency sums hain — root total frequency.
- 7 **Data Compression** — Data size kam karne ka process — transmission faster, storage less.
- 8 **ASCII vs Huffman** — ASCII har character ke liye 8 bits fixed; Huffman variable length codes — efficient.

DIAGRAM TIME

► EXPRESSION TREE: $(A + B * C) + ((D * E + F) * G)$



➤ Inner nodes operators (+, *), leaves operands (a,b,c,d,e,f,g). Binary tree hai, BST nahi.

► INORDER WITH PARENTHESIS – CODE VISUALIZATION

```
void inorder(TreeNode<int>* treeNode) {
    if (treeNode != NULL) {
        cout << "(";
        inorder(treeNode->getLeft());
        cout << ")";
        cout << *(treeNode->getInfo());
        cout << "(";
```

```

        inorder(treeNode->getRight());
        cout << ")";
    }
}

```

↗ *Parenthesis wala inorder — output: $(a + (b * c)) + (((d * e) + f) * g)$*

► POSTFIX TO EXPRESSION TREE – STEP 1 (PUSH OPERANDS)

Postfix: $a\ b\ +\ c\ d\ e\ +\ *\ *$

Stack after a,b:

[a] [b]

top→

Stack after c,d,e:

[a] [b] [c] [d] [e]

top→

↗ *Operands ko stack mein push karte hain — har operand ka tree node banata hain.*

► POSTFIX TO EXPRESSION TREE – STEP 2 (OPERATOR +)

Before:

[a] [b] [c] [d] [e]

After:

[a] [b] [c] [+]

$$\begin{array}{c} / \quad \backslash \\ d \quad e \end{array}$$

Operator + → pop d,e → make parent +

↗ *Operator aane par do operands pop karte hain — unhe parent operator node ke children banate hain.*

► POSTFIX TO EXPRESSION TREE – STEP 3 (OPERATOR *)

Before:

[a] [b] [c] [+]

After:

[a] [b] [*]

$$\begin{array}{c} / \quad \backslash \\ c \quad + \\ / \quad \backslash \\ d \quad e \end{array}$$

↗ *Operator * → pop c aur + subtree — unhe * ke children banate hain.*

► POSTFIX TO EXPRESSION TREE – STEP 4 (OPERATOR *)

Before:

[a] [b] [*]

After (Final):

[*]



➤ Final operator $*$ $\rightarrow$ pop + subtree (a+b) aur $*$ subtree (c*(d+e)) — final expression tree.

► HUFFMAN TREE — FREQUENCY TABLE

Phrase: "traversing threaded binary trees"

Letters: NL:1, SP:3, a:3, b:1, d:2, e:5, g:1, h:1, i:2, n:2, r:5, s:2

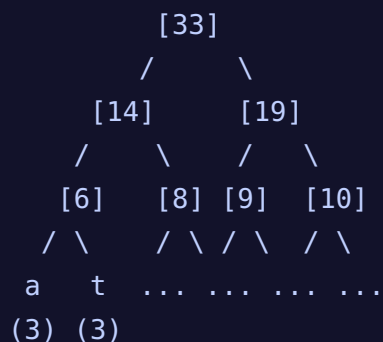
➤ Total 33 characters — 264 bits ASCII vs ~116 bits Huffman (40% compression).

► HUFFMAN TREE BUILDING (COMBINE LOWEST FREQUENCIES)

Step 1: v(1)+y(1)=2 Step 2: g(1)+h(1)=2
 Step 3: NL(1)+b(1)=2 Step 4: d(2)+i(2)=4
 Step 5: s(2)+n(2)=4 Step 6: a(3)+t(3)=6
 ... continue combining until root = 33

➤ Lowest frequency nodes ko combine karte hain — parent frequency = sum of children frequencies.

► FINAL HUFFMAN TREE STRUCTURE



➤ Root frequency 33 — leaves are characters, internal nodes are frequency sums.

Huffman Connection: Huffman Encoding binary tree use karta hai — frequent characters ko short codes (tree mein closer to root), rare characters ko long codes (deeper). Is tarah total message size kam ho jaata hai. ASCII 264 bits ki jagah ~116 bits mein message send ho sakta hai — 40% compression.

CODE EXPLANATION

► INORDER WITH PARENTHESIS

```
void inorder(TreeNode<int>* treeNode) {
    if (treeNode != NULL) {
        cout << "(";
        inorder(treeNode->getLeft());
        cout << ")";
        cout << *(treeNode->getInfo());
        cout << "(";
        inorder(treeNode->getRight());
        cout << ")";
    }
}
```

- 1 **cout << "("** — Left subtree se pehle opening parenthesis.
- 2 **inorder(treeNode->getLeft())** — Left subtree traverse.
- 3 **cout << ")"** — Left subtree ke baad closing parenthesis.
- 4 **cout << *(treeNode->getInfo())** — Current node print.
- 5 **cout << "("** — Right subtree se pehle opening parenthesis.
- 6 **inorder(treeNode->getRight())** — Right subtree traverse.
- 7 **cout << ")"** — Right subtree ke baad closing parenthesis.

► POSTFIX TO EXPRESSION TREE ALGORITHM

Algorithm:

1. Read symbol from postfix expression (left to right)

2. If symbol is operand:
Create tree node, push on stack
3. If symbol is operator:
Pop T1 (right child) and T2 (left child)
Create new node with operator
Set left = T2, right = T1
Push new node on stack
4. At end, stack has one tree – the expression tree

- 1 **Operand → Push** — Operand ka tree node banate hain aur stack mein push karte hain.
- 2 **Operator → Pop twice** — Do operands/subtrees pop karte hain.
- 3 **Make parent node** — Operator root, popped nodes left aur right children.
- 4 **Push subtree** — Naya subtree stack mein push.

EXAM FOCUS POINTS

- MCQ** Expression tree mein leaves operands hain, internal nodes operators hain — BST nahi hai.
- MCQ** Postfix to expression tree — stack use karta hai. Operand → push, Operator → pop two, make parent.
- MCQ** Inorder traversal with parenthesis — output: $(a + (b * c)) + (((d * e) + f) * g)$.
- MCQ** Huffman Encoding — frequent characters ko short codes, rare ko long codes — variable length coding.
- FORMULA** Huffman compression: $33 \text{ chars} \times 8 \text{ bits} = 264 \text{ bits (ASCII)} \rightarrow \sim 116 \text{ bits (Huffman)} \rightarrow \sim 40\% \text{ saving}$.
- CONFUSION** Expression tree $\neq$ Binary Search Tree — BST mein ordering hoti hai, expression tree mein nahi.

PRACTICE

Postfix "a b + c d e + * *" ko expression tree mein convert karein — step by step stack trace.

PRACTICE

Huffman tree build karein given frequencies — combine lowest frequencies till root.

MIND MAP**Expression Trees & Huffman Encoding****Expression Trees**

- Leaves: operands
- Internal: operators
- Binary operators → binary tree
- Inorder → infix expression
- Postorder → postfix expression

Building Expression Tree

- Postfix input
- Stack of tree nodes
- Operand → push
- Operator → pop two, make parent
- One tree remains on stack

Huffman Encoding

- Frequency table
- Combine lowest frequencies
- Binary tree with frequencies
- Frequent = short code
- Rare = long code

Applications

- Compilers (expression trees)
- Spreadsheets
- Data compression (Huffman)
- JPEG, MPEG
- Modems (live compression)

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the difference between an expression tree and a BST?

A: Expression tree mein leaves operands hain, internal nodes operators hain — koi sorting nahi hoti. BST mein left child < parent < right child — ordering hoti hai. Expression tree binary tree hai but BST nahi.

Q: How does postfix to expression tree algorithm work?

A: Postfix expression ko left to right read karte hain. Operand milne par tree node banate hain aur stack mein push karte hain. Operator milne par do nodes pop karte hain, operator ka parent node banate hain, popped nodes ko left aur right children banate hain, aur new subtree stack mein push karte hain.

Q: Why do we add parenthesis in inorder traversal of expression trees?

A: Inorder traversal se infix expression milti hai lekin parenthesis nahi hote — operator precedence ambiguous ho sakti hai. Parenthesis add karne se expression unambiguous ho jaati hai aur evaluation order clear hota hai.

Q: What is the basic idea behind Huffman Encoding?

A: Frequent characters ko short binary codes assign karte hain, rare characters ko longer codes. Is tarah total message size kam ho jaata hai — ASCII fixed 8 bits ki jagah variable length codes use hote hain.

Q: How is a Huffman tree built?

A: Pehle frequency table banate hain. Phir har character ka node (frequency ke saath) list mein daalte hain. Do lowest frequency nodes ko combine karte hain — parent frequency = sum. Is process ko repeat karte hain jab tak ek root na bane (total frequency).

Q: What is the advantage of Huffman Encoding over ASCII?

A: ASCII har character ke liye 8 bits fixed use karta hai. Huffman variable length codes use karta hai — frequent characters ko short codes milte hain. Example: 33 character phrase 264 bits (ASCII) se ~116 bits (Huffman) mein compress ho jata hai — ~40% saving.

► QUICK QUIZ (10 QUESTIONS)

1. In expression trees, leaves contain what? **(Operands)**
2. In expression trees, internal nodes contain what? **(Operators)**
3. Postfix to expression tree uses which data structure? **(Stack)**
4. What happens when operator is encountered in postfix to tree? **(Pop two, make parent)**
5. Huffman Encoding is used for? **(Data compression)**
6. In Huffman, frequent characters get ___ codes? **(Short)**
7. Huffman tree root frequency = ? **(Total characters)**
8. Huffman compression saving for 33 char phrase? **(~40%)**

Lesson 26: Huffman Encoding & Tree Properties

Complete Huffman compression — internal vs external nodes property

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap ek lamba message bhejna chahte hain — agar aap har letter ko poora naam likh kar bhejte hain to time lagta hai. Lekin agar aap zyada aane wale letters ko short codes dein aur kam aane walon ko long codes, to total message chhota ho jaata hai. Huffman Encoding yahi karta hai — jaise aap "a" ko "0", "e" ko "10" aur "z" ko "11001" code de dete hain. Receiver ko sirf tree bhejna padta hai aur phir woh decode kar leta hai. Compression 54% tak ho jaati hai — network transmission fast ho jaati hai.

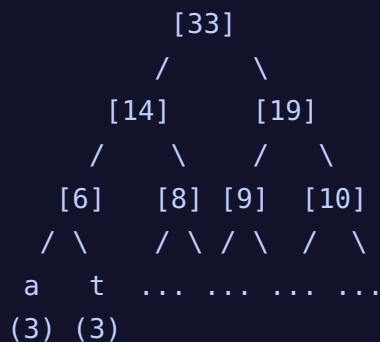
KEY TERMS

- 1 Huffman Code Table** — Har character ka binary code — frequent characters short codes, rare long codes.
- 2 Encoded Message** — Original text ko Huffman codes mein convert karna — only 0s and 1s.
- 3 Decoding Process** — Receiver tree use karta hai — bits follow karte hain root se leaf tak — character milta hai.

- 4 **Priority Queue** — Huffman tree building ke liye — lowest frequency nodes pehle.
- 5 **Compression Ratio** — ASCII 264 bits vs Huffman 120 bits — 54% compression.
- 6 **Internal Node** — Binary tree mein node with data — includes root and leaves.
- 7 **External Node** — NULL pointers — square nodes in diagram — no data.
- 8 **Property: $N_{\text{internal}} = N_{\text{external}} + 1$** — Binary tree theorem — important for proofs.

DIAGRAM TIME

► HUFFMAN TREE WITH PATH VALUES (0=LEFT, 1=RIGHT)



↗ Root se leaf tak path 0/1 — code for 'a' = 000, 't' = 001, 'r' = 110, 'e' = 101.

► HUFFMAN CODE TABLE (FROM TREE)

Character	Code	Character	Code
NL	10000	i	0101
SP	1111	n	0110
a	000	r	110
b	10001	s	0111
d	0100	t	001
e	101	v	11100

g	10010	y	11101
h	10011		

↗ Frequent letters (a,e,r,t) short codes (3 bits). Rare letters (NL,b,g,h,v,y) long codes (5 bits).

► ENCODED MESSAGE (120 BITS)

Original: traversing threaded binary trees

Encoded:

```
00111000011100101110011101010110100101111001
100111101010000100101010011111000101010110000
110111011111001110101101011110000
```

↗ 33 characters → 120 bits (vs 264 bits ASCII) — 54% compression.

► DECODING PROCESS (RECEIVER SIDE)

Bitstream: 001110000...

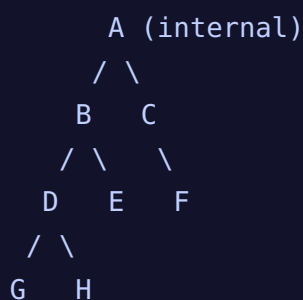
Start at root: 0→left, 0→left, 1→right → leaf 't'

Reset to root, next bit: 1→right, 1→right, 0→left → leaf 'r'

Continue until all bits decoded.

↗ Receiver tree use karta hai — bits follow karte hain root se leaf tak — character decode ho jaata hai.

► INTERNAL VS EXTERNAL NODES PROPERTY



↗ Internal nodes = 7 (A,B,C,D,E,F,G,H?) Let's count: A,B,C,D,E,F,G,H = 8?
Actually leaf nodes are also internal in this definition. External nodes = NULL
pointers = 9. Property: Internal N = 9, External = 10 → N+1.

► INTERNAL/EXTERNAL NODES VISUALIZATION



➤ Internal nodes (with data): A,B,C,D,E,F,G,H,I,J = 10. External nodes (NULL pointers) = 11. Property: External = Internal + 1.

Huffman Connection: Huffman tree building bottom-up hoti hai — leaf nodes se start karte hain aur combine karte hain. Priority queue use karke lowest frequencies ko pehle process karte hain. Binary tree property: N internal nodes $\rightarrow N+1$ external nodes. Yeh property Huffman tree mein bhi apply hoti hai — agar N characters hain to $N-1$ internal nodes honge.

CODE EXPLANATION

➤ Is lecture mein actual C++ code nahi diya gaya — yeh concept-based lecture hai jo Huffman encoding complete process aur binary tree properties ko explain karta hai. Huffman encoding ka code implementation Algorithms course mein cover kiya jayega.

EXAM FOCUS POINTS

MCQ Huffman code table — frequent characters get short codes, rare get long codes (variable length).

MCQ Huffman compression — 33 char phrase: 264 bits (ASCII) $\rightarrow$ 120 bits (Huffman) = 54% compression.

MCQ

Decoding — receiver tree use karta hai, bits follow karke leaf tak jaata hai, character decode hota hai.

FORMULA

Internal nodes = N , External nodes = $N+1$ — binary tree property.

CONFUSION

External nodes = NULL pointers (square nodes) — not leaf nodes. Leaf nodes are internal nodes.

MCQ

Huffman tree building — priority queue use karta hai — lowest frequency nodes first.

PRACTICE

Given a phrase, build Huffman tree, create code table, calculate compression ratio.

PRACTICE

Given a binary tree, count internal and external nodes — verify $N+1$ property.

MIND MAP

Huffman Encoding & Tree Properties

Huffman Process

- Frequency table
- Build tree (bottom-up)
- Assign 0/1 to branches
- Create code table
- Encode message

Compression

- Variable length codes
- Frequent = short
- Rare = long
- ASCII: 264 bits
- Huffman: 120 bits

- 54% compression

Decoding

- Receiver has tree
- Follow bits root→leaf
- Character decoded
- Reset to root
- Continue

Tree Properties

- Internal nodes: with data
- External nodes: NULL
- Property: $\text{Ext} = \text{Int} + 1$
- $N \text{ internal} \rightarrow N+1 \text{ external}$

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: How does Huffman encoding achieve data compression?

A: Huffman encoding variable length codes use karta hai. Frequent characters ko short codes milte hain, rare characters ko long codes. Is tarah total message size kam ho jaata hai. Example: 33 char phrase 264 bits (ASCII) se 120 bits (Huffman) mein compress ho gayi — 54% saving.

Q: How does the receiver decode a Huffman-encoded message?

A: Receiver ko sender tree bhejta hai. Receiver tree ke root se start karta hai. Har bit ke hisaab se left (0) ya right (1) move karta hai. Leaf node tak pahunch kar character decode ho jaata hai. Phir root par reset karta hai aur next bit se process repeat karta hai.

Q: What is the role of priority queue in Huffman encoding?

A: Priority queue Huffman tree building mein use hoti hai — lowest frequency nodes ko pehle process karte hain. Two lowest frequency nodes ko queue se remove karte hain, unka parent banate hain (frequency = sum), aur parent ko queue mein insert karte hain. Process tab tak repeat karte hain jab tak queue mein ek node na bache — woh root hai.

Q: What is the difference between internal and external nodes in a binary tree?

A: Internal nodes woh nodes hain jin mein data hai — root, inner nodes, aur leaves sab internal nodes hain. External nodes NULL pointers hain — square nodes diagram mein. Property: External nodes = Internal nodes + 1.

Q: Why does the property "External = Internal + 1" hold for binary trees?

A: Yeh binary tree ka fundamental property hai. Har internal node ke 2 children positions hain (left/right). Leaf nodes ke NULL pointers external nodes hain. Total NULL pointers = total children positions - actual children. Is se External = Internal + 1 proof hota hai.

Q: How do modems use Huffman encoding in real-time?

A: Modems live compression karte hain. Sender modem incoming data ka Huffman tree banata hai, tree ko receiver modem ko bhejta hai, phir compressed data bhejta hai. Receiver modem tree use karke decode karta hai. Modems periodically tree ko revise karte hain based on recent data patterns.

► QUICK QUIZ (10 QUESTIONS)

1. Huffman encoding uses ___ length codes? **(Variable)**
2. Frequent characters get ___ codes? **(Short)**
3. Rare characters get ___ codes? **(Long)**
4. 33 char phrase ASCII bits = ? **(264)**
5. 33 char phrase Huffman bits = ? **(120)**

- | | |
|---|------------------|
| 6. Compression percentage for 264→120? | (54%) |
| 7. In binary tree, External = Internal + ? | (1) |
| 8. External nodes are also called ____ nodes? | (NULL) |
| 9. Huffman tree building uses which data structure? | (Priority queue) |
| 10. Huffman encoding is used in which image format? | (JPEG) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Huffman Code Table:** Character to binary code mapping — frequent = short, rare = long.
- **Encoded Message:** Original text converted to Huffman codes (0s and 1s).
- **Decoding:** Receiver tree use karta hai — bits follow kar ke leaf tak jaate hain.
- **Internal Node:** Node with data (root, inner, leaves all are internal).
- **External Node:** NULL pointer (square node) — represents missing child.
- **Binary Tree Property:** External nodes = Internal nodes + 1.

IMPORTANT CONCEPTS

- Huffman encoding = variable length coding based on frequency.
- ASCII fixed 8 bits vs Huffman variable bits (3-5 bits per character).
- Compression ratio: 33 chars 264 bits → 120 bits = 54% saving.
- Priority queue used for tree building — lowest frequency first.
- Decoding: root to leaf path — 0=left, 1=right.
- Modems use Huffman for live compression — periodic tree revision.
- Binary tree property: External = Internal + 1.
- External nodes = NULL pointers (not leaf nodes).

△ IMPORTANT FORMULAS

- **ASCII bits** = Characters × 8

- **Huffman bits** = Sum over all chars (frequency × code length)
- **Compression ratio** = $(\text{ASCII} - \text{Huffman}) / \text{ASCII} \times 100\%$
- **Binary tree property**: External = Internal + 1
- **Huffman tree nodes**: N leaves + (N-1) internal nodes
- **Total nodes** = $2N - 1$ (for N characters)

FREQUENTLY ASKED TOPICS

- Build Huffman tree from frequency table.
- Create code table from Huffman tree.
- Encode a message using Huffman codes.
- Decode a message using Huffman tree.
- Calculate compression ratio.
- Count internal and external nodes in a binary tree.
- Verify External = Internal + 1 property.

COMMON MISTAKES

- Confusing external nodes with leaf nodes — external nodes are NULL pointers, leaves are internal nodes.
- Thinking Huffman codes are fixed length — they are variable length.
- Forgetting that receiver needs the tree to decode.
- Not resetting to root after each character during decoding.
- Assuming all Huffman trees for same text are identical — different choices possible.

One-Day-Before-Exam Summary: Huffman encoding data compression technique hai — frequent characters ko short codes, rare ko long codes. 33 character phrase 264 bits (ASCII) se 120 bits (Huffman) mein compress ho gayi — 54% saving. Decoding ke liye receiver ko tree chahiye — bits follow karke root se leaf tak jaate hain. Binary tree property: External nodes = Internal nodes + 1 (NULL pointers + data nodes)

Lesson 27: Threaded Binary Trees

Stack-free inorder traversal using threads — properties & insertion

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap kisi kitaab mein chapters search kar rahe hain — normally aapko har chapter ke end par wapas index par jaana padta hai (stack use karna). Lekin agar aap har chapter ke end mein next chapter ka page number likh den (threads), to aap seedha next chapter par ja sakte hain bina wapas index par gaye. Threaded binary tree bhi yahi karta hai — NULL pointers ko inorder successor/predecessor ki taraf point karwa deta hai, taake traversal stack-free ho jaaye.

KEY TERMS

- 1 Binary Tree Property (Links)** — N internal nodes $\rightarrow 2N$ links total, $N-1$ internal links, $N+1$ external links (NULL pointers).
- 2 External Link** — NULL pointer — $N+1$ in a binary tree with N internal nodes.
- 3 Threaded Binary Tree** — NULL pointers ko inorder successor/predecessor se replace karna — stack-free traversal.

- 4 **LTH (Left Thread Flag)** — Flag jo batata hai left pointer thread hai ya child pointer.
- 5 **RTH (Right Thread Flag)** — Flag jo batata hai right pointer thread hai ya child pointer.
- 6 **Inorder Successor** — Inorder traversal mein next node — left-most node in right subtree.
- 7 **nextInorder Function** — Routine jo node ka inorder successor return karti hai — threads use karti hai.
- 8 **Stack-free Traversal** — Recursion ya explicit stack ke bina inorder traversal — threads se possible.

DIAGRAM TIME

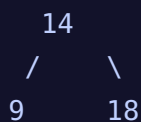
► INTERNAL VS EXTERNAL LINKS (N=9 INTERNAL NODES)



Internal links: 8 (N-1)
External links: 10 (N+1)
Total links: 18 = 2N

↗ 9 internal nodes → 8 internal links + 10 external links (NULL pointers) = 18 = 2×9.

► THREADED BINARY TREE – NULLS REPLACED WITH THREADS



```

      / \   / \
     4  12 16 20
    / \   / \ / \
   3  7  11 15 17 21

```

↗ NULL pointers ko inorder successor/predecessor threads se replace kiya gaya
— dotted lines indicate threads.

► THREAD INSERTION – STEP BY STEP

Before: 14→15→18→20 (inorder)
Inserting 16 as left child of 18:

1. t->L = p->L (copy thread)
2. t->LTH = thread
3. t->R = p (successor)
4. t->RTH = thread
5. p->L = t (attach)
6. p->LTH = child

↗ New node 16 insert karne ke liye predecessor/successor threads update karte hain.

► NEXTINORDER FUNCTION VISUALIZATION

```

TreeNode* nextInorder(TreeNode* p) {
    if (p->RTH == thread)
        return(p->R);
    else {
        p = p->R;
        while(p->LTH == child)
            p = p->L;
        return p;
    }
}

```

↗ If RTH==thread → return right (successor). Else → go right, then left-most node.

► FINDING INORDER SUCCESSOR OF NODE 4

```

      14
     /  \
    9    18
   / \  / \

```

```

    4  12  16  20
   / \  / \  / \
  3  7 11 15 17 21

```

➤ Node 4 ka inorder successor = left-most node in right subtree = 7. Right subtree of 4: 9→12→7.

► PROBLEM: NEXTINORDER WITH ROOT (14)

```

Calling nextInorder(root=14):
→ RTH != thread (has right child)
→ go to right child = 18
→ then left-most = 16
→ returns 16 (WRONG! Should be 3 for inorder)

```

➤ Root se start karne par nextInorder fail ho jaata hai — inorder first node should be 3, not 16.

Threaded Tree Connection: Threaded binary trees NULL pointers ka use karte hain — stack-free inorder traversal possible. N internal nodes → N+1 NULL pointers available. Flags (LTH, RTH) distinguish threads from child pointers. nextInorder routine gives successor. Root se start karne mein problem hai — lecture mein fix discuss hoga.

CODE EXPLANATION

► THREAD INSERTION CODE

```

// Inserting new node t as left child of p
t->L = p->L;           // copy the thread (predecessor)
t->LTH = thread;       // left is a thread
t->R = p;              // *p is successor of *t
t->RTH = thread;       // right is a thread
p->L = t;              // attach the new leaf
p->LTH = child;        // left is now a child pointer

```

- 1 **t->L = p->L** — New node ka left pointer predecessor ki taraf (p ka purana left thread).

- 2 **t->LTH = thread** — Left pointer thread hai flag set.
- 3 **t->R = p** — New node ka right pointer successor (p) ki taraf.
- 4 **t->RTH = thread** — Right pointer thread hai flag set.
- 5 **p->L = t** — New node ko p ke left child ke roop mein attach.
- 6 **p->LTH = child** — p ka left pointer ab child hai flag update.

► NEXTINORDER FUNCTION

```
TreeNode* nextInorder(TreeNode* p) {  
    if (p->RTH == thread)  
        return(p->R);           // right thread → successor  
    else {  
        p = p->R;                // go to right child  
        while(p->LTH == child)   // go left until thread  
            p = p->L;  
        return p;               // left-most node is successor  
    }  
}
```

- 1 **if (p->RTH == thread)** — Right pointer thread hai to directly successor.
- 2 **p = p->R** — Right child par jaao (normal link).
- 3 **while(p->LTH == child)** — Left child link hai to left jaate raho.
- 4 **return p** — Left-most node (inorder successor) return.
- 5 **Problem with root** — Root se call karne par first node nahi milta — fix needed.

EXAM FOCUS POINTS

MCQ

N internal nodes → 2N links, N-1 internal links, N+1 external links (NULL pointers).

MCQ

Threaded binary tree — NULL pointers ko inorder successor/ predecessor se replace karte hain.

MCQ

LTH and RTH flags — distinguish threads from child pointers.

MCQ

Inorder successor of a node = left-most node in its right subtree.

CONFUSION

nextInorder(root) doesn't work directly — root ka inorder predecessor/successor handle karna hai.

FORMULA

External links = $N+1$, Internal links = $N-1$, Total = $2N$.

PRACTICE

Given a BST, convert to threaded binary tree — identify all NULL pointers and replace with threads.

PRACTICE

Trace nextInorder for different nodes in a threaded tree.

MIND MAP

Threaded Binary Trees

Tree Properties

- N internal nodes
- $2N$ total links
- $N-1$ internal links
- $N+1$ external links
- External = NULL pointers

Threaded Tree

- Replace NULL pointers
- With inorder successor
- With inorder predecessor
- LTH/RTH flags
- Stack-free traversal

Thread Insertion

- Copy predecessor thread
- Set LTH = thread
- Set RTH = thread
- Attach new node
- Update parent flag

nextInorder

- If RTH=thread → successor
- Else go right
- Then left-most node
- Root problem exists
- Fix in next lecture

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the link property of binary trees with N internal nodes?

A: Binary tree with N internal nodes has $2N$ total links. $N-1$ links connect internal nodes (each node except root has one parent). $N+1$ links are external (NULL pointers). Total = $(N-1) + (N+1) = 2N$.

Q: What is a threaded binary tree and why is it useful?

A: Threaded binary tree NULL pointers ko inorder successor/predecessor se replace karta hai. Is se inorder traversal stack-free ho jaati hai — recursion ya explicit stack ki zaroorat nahi. Performance improve hoti hai repeated traversals mein.

Q: What are LTH and RTH flags used for?

A: LTH (Left Thread Flag) batata hai left pointer thread hai ya child pointer. RTH (Right Thread Flag) batata hai right pointer thread hai ya child pointer. Flags distinguish normal child links from threads — leaf node identification bhi change ho jaati hai.

Q: How does nextInorder find the inorder successor of a node?

A: Agar RTH=thread hai to directly right pointer return karta hai (successor). Agar RTH=child hai to right child par jaata hai, phir left links follow karta hai jab tak LTH=child hai. Jab thread aata hai to left-most node return karta hai — woh inorder successor hai.

Q: What is the problem with calling nextInorder on the root?

A: Root se call karne par nextInorder right subtree mein chala jaata hai — inorder first node (left-most node of tree) nahi milta. Example: root 14 → nextInorder returns 16 instead of 3. Lecture mein fix discuss ho ga.

Q: How do we insert a node in a threaded binary tree?

A: New node t ko p ke left child ke roop mein insert karte hain. t ka left pointer p ke purane left thread ko copy karta hai (predecessor), t ka right pointer p ko point karta hai (successor). Flags update karte hain — t ka LTH/RTH = thread, p ka LTH = child.

► QUICK QUIZ (10 QUESTIONS)

1. N internal nodes → total links = ? **(2N)**
2. N internal nodes → internal links = ? **(N-1)**
3. N internal nodes → external links = ? **(N+1)**
4. Threaded tree replaces which pointers? **(NULL pointers)**
5. LTH flag indicates what? **(Left pointer is thread)**
6. RTH flag indicates what? **(Right pointer is thread)**
7. Inorder successor of a node = ? **(Left-most node in right subtree)**
8. nextInorder returns what? **(Inorder successor)**

9. nextInorder(root) problem — returns wrong node?

(Yes)

10. Threaded tree traversal is ___ free?

(Stack)

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Internal Link:** Link between two internal nodes — $N-1$ in a tree with N internal nodes.
- **External Link:** NULL pointer — $N+1$ in a tree with N internal nodes.
- **Threaded Binary Tree:** NULL pointers replaced with inorder successor/predecessor.
- **LTH (Left Thread Flag):** Indicates left pointer is thread (1) or child (0).
- **RTH (Right Thread Flag):** Indicates right pointer is thread (1) or child (0).
- **Inorder Successor:** Next node in inorder traversal — left-most node in right subtree.

Lesson 28: Threaded Inorder & Complete Binary Tree

Dummy node trick for stack-free traversal — array representation of complete binary trees

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap kisi kitaab mein chapters search kar rahe hain — agar aap har chapter ke end mein next chapter ka page number likh den (threads), to aap seedha next chapter par ja sakte hain. Lekin agar aap bilkul shuru se start karte hain to aapko pehle chapter ka pata nahi hota — is liye hum ek "dummy" page (0) insert kar dete hain jo first chapter ki taraf point karta hai. Is tarah traversal stack-free ho jaati hai. Complete binary tree mein hum pointers ki jagah array use kar sakte hain — jaise aap kitaab ke pages ko index numbers se refer karte hain.

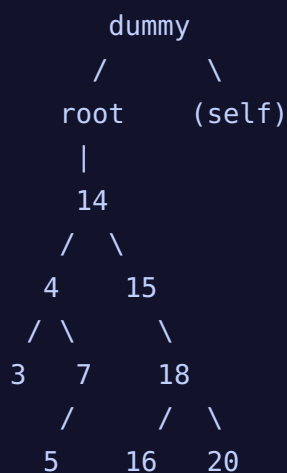
KEY TERMS

- 1 **Dummy Node** — Extra node inserted in threaded tree — left points to root, right points to itself — solves root problem in nextInorder.
- 2 **fastInorder** — Public routine jo dummy node se start karta hai — calls nextInorder in loop — stack-free traversal.

- 3 **Complete Binary Tree** — Tree completely filled except possibly bottom level — bottom level filled left to right.
- 4 **Array Representation** — Complete binary tree ko array mein store karna — no pointers needed.
- 5 **Index Formula** — Node at position i : left child = $2i$, right child = $2i+1$, parent = $\text{floor}(i/2)$.
- 6 **Level-Order Traversal** — Tree nodes ko level by level visit karna — array index directly gives level-order position.
- 7 **Height Property** — Complete binary tree of height h has between 2^h and $2^{(h+1)}-1$ nodes.

DIAGRAM TIME

► THREADED TREE WITH DUMMY NODE



↗ Dummy node left → root, right → self. Left-most node's left thread → dummy, right-most node's right thread → dummy.

► FASTINORDER — START WITH DUMMY

```
void fastInorder(TreeNode* p) {
    while ((p = nextInorder(p)) != dummy)
```

```

    cout << p->getInfo();
}

```

↗ *fastInorder(dummy) call — dummy se start, left-most node (3) tak jaata hai phir threads follow karta hai.*

► NEXTINORDER WITH DUMMY – FIRST STEP

```
nextInorder(dummy):
```

- RTH != thread (has right to self)
- p = p->R (go to self? Actually left pointer to root)
- while(LTH == child) go left
- reaches 3 (left-most node)
- returns 3

↗ *Dummy se start karne par correct first node (3) milta hai — root problem solved.*

► COMPLETE BINARY TREE (ARRAY REPRESENTATION)

```

      A (1)
     /  \
    B (2) C (3)
   /  \ /  \
  D(4) E(5) F(6) G(7)
 /  \ /
H(8) I(9) J(10)

```

↗ *Level-order numbers match array indices. Node at i: left=2i, right=2i+1, parent=floor(i/2).*

► ARRAY STORAGE OF COMPLETE BINARY TREE

```

Index: 0  1  2  3  4  5  6  7  8  9  10 11 12 13 14
Value: -  A  B  C  D  E  F  G  H  I  J  -  -  -  -
        (unused) (root at 1, not 0)

```

↗ *Index 1 se start — A at 1, B at 2, C at 3, D at 4, E at 5, F at 6, G at 7, H at 8, I at 9, J at 10.*

► LEVEL-ORDER TRAVERSAL WITH INDEX NUMBERS

```

      A (1)
     /   \
    B (2)  C (3)
   /  \  /  \
  D(4) E(5) F(6) G(7)
 / \   /
H(8) I(9) J(10)

```

➤ Level-order traversal directly gives array indices — queue se level-order karke array mein store kar sakte hain.

Threaded Tree Connection: Dummy node trick se nextInorder root problem solve hoti hai — fastInorder stack-free traversal provide karta hai. Complete binary tree array mein store ho sakta hai — $2i$, $2i+1$ formula se children access. Arrays pointers se faster hain — compiler supported, paging issues less.

CODE EXPLANATION

► NEXTINORDER (THREADED TREE)

```

TreeNode* nextInorder(TreeNode* p) {
    if (p->RTH == thread)
        return(p->R);           // right thread → successor
    else {
        p = p->R;                // go to right child
        while(p->LTH == child)   // go left until thread
            p = p->L;
        return p;               // left-most node = successor
    }
}

```

- 1 **if (p->RTH == thread)** — Right pointer thread hai to directly successor return.
- 2 **p = p->R** — Right child par jaao (normal link).
- 3 **while(p->LTH == child)** — Left child link hai to left jaate raho.

4 **return p** — Left-most node (inorder successor).

► FASTINORDER (STACK-FREE TRAVERSAL)

```
void fastInorder(TreeNode* p) {  
    while ((p = nextInorder(p)) != dummy)  
        cout << p->getInfo();  
}
```

1 **fastInorder(dummy)** — Dummy node se start karte hain.

2 **(p = nextInorder(p)) != dummy** — nextInorder call, return value p mein save, dummy check.

3 **cout << p->getInfo()** — Print node value.

4 **No stack used** — Pure threads based traversal, very fast.

► COMPLETE BINARY TREE – ARRAY FORMULAS

```
// Node at position i in array (1-based indexing)  
int leftChild(int i)    { return 2*i; }  
int rightChild(int i)   { return 2*i + 1; }  
int parent(int i)       { return i/2; } // integer division (floor)
```

1 **leftChild = 2i** — Left child position in array.

2 **rightChild = 2i+1** — Right child position in array.

3 **parent = floor(i/2)** — Parent position in array.

4 **Index 0 unused** — 2i formula ke liye 0 se start nahi karte.

EXAM FOCUS POINTS

MCQ

Dummy node — left points to root, right points to itself — solves nextInorder root problem.

MCQ fastInorder(dummy) — stack-free inorder traversal using threads.

MCQ Complete binary tree — completely filled except possibly bottom level — bottom filled left to right.

FORMULA Node at i : left= $2i$, right= $2i+1$, parent= $\text{floor}(i/2)$.

FORMULA Complete binary tree height $h \rightarrow$ between 2^h and $2^{(h+1)}-1$ nodes.

CONFUSION Complete binary tree $\neq$ Perfect binary tree — perfect has all leaves at same level, complete allows missing nodes at bottom right.

PRACTICE Given a complete binary tree, store in array and find children of node at position 5.

PRACTICE Trace fastInorder on threaded tree with dummy node.

MIND MAP

Threaded Inorder & Complete Binary Tree

Threaded Traversal

- Dummy node trick
- fastInorder(dummy)
- Stack-free
- No recursion
- Very fast

Complete Binary Tree

- Filled except bottom
- Bottom left to right
- Height h : 2^h to $2^{(h+1)}-1$

- Height = $\text{floor}(\log_2 N)$

Array Storage

- No pointers needed
- Index 1-based
- Left = $2i$
- Right = $2i+1$
- Parent = $\text{floor}(i/2)$

Efficiency

- Arrays are faster
- Compiler supported
- Less paging overhead
- Level-order = array order

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the dummy node trick in threaded binary trees?

A: Dummy node extra node hai — left pointer root ki taraf, right pointer khud ki taraf. Left-most node ka left thread dummy ko point karta hai, right-most node ka right thread dummy ko. fastInorder(dummy) call karne se correct inorder traversal start hota hai — root problem solve ho jaati hai.

Q: How does fastInorder work without a stack?

A: fastInorder dummy node se start karta hai. nextInorder repeatedly call hota hai — threads follow karte hain (right thread → successor) ya right child par ja kar left-most node find karte hain. Loop tab tak chalta hai jab tak dummy node wapas na aa jaye. Koi recursion ya stack use nahi hota.

Q: What is a complete binary tree?

A: Complete binary tree woh tree hai jo completely filled hai except possibly bottom level. Bottom level left to right filled hai. Is liye array mein store kiya ja sakta hai without pointers. Height h ke liye nodes between 2^h and $2^{(h+1)}-1$ hote hain.

Q: How can a complete binary tree be stored in an array?

A: Node at position i (1-based indexing) — left child at $2i$, right child at $2i+1$, parent at $\text{floor}(i/2)$. Index 0 unused. Level-order traversal directly gives array order. No pointers needed — implicit pointers through formulas.

Q: Why are arrays faster than pointers for tree implementation?

A: Arrays are language built-in — compiler directly supports them. Pointers can cause paging issues in virtual memory — code pages load/unload. Arrays use sequential memory access — faster. Complete binary trees array mein store ho sakte hain — pointer overhead avoid ho jaata hai.

Q: What is the height property of complete binary trees?

A: Complete binary tree of height h has between 2^h and $2^{(h+1)}-1$ nodes. Height = $\text{floor}(\log_2 N)$ where N is number of nodes. This makes search/operations $O(\log n)$ efficient.

► QUICK QUIZ (10 QUESTIONS)

- | | |
|--|----------------------------|
| 1. Dummy node's left points to what? | (Root) |
| 2. Dummy node's right points to what? | (Itself) |
| 3. fastInorder is called with which node? | (Dummy) |
| 4. Complete binary tree bottom level is filled from? | (Left to right) |
| 5. Node at i : left child = ? | ($2i$) |
| 6. Node at i : right child = ? | ($2i+1$) |
| 7. Node at i : parent = ? | ($\text{floor}(i/2)$) |
| 8. Array index 0 is used for complete binary tree? | (No) |
| 9. Height h complete tree nodes range? | (2^h to $2^{(h+1)}-1$) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Dummy Node:** Extra node in threaded tree — left→root, right→self — enables correct stack-free traversal.
- **fastInorder:** Public routine — calls nextInorder in loop until dummy — prints inorder without stack.
- **Complete Binary Tree:** Tree filled except bottom level — bottom left to right.
- **Array Representation:** Complete tree stored in array — index formulas for children/parent.

IMPORTANT CONCEPTS

- Threaded tree traversal is stack-free — no recursion, no explicit stack.
- Dummy node solves the "root problem" in nextInorder.
- Complete binary tree can be stored without pointers — array indexing.
- Level-order traversal order = array indices (1-based).
- Arrays are faster than pointers — compiler supported, less paging.
- Height of complete tree with N nodes = $\text{floor}(\log_2 N)$.

△ IMPORTANT FORMULAS

- **Node at i** (1-based): left = $2i$, right = $2i+1$, parent = $\text{floor}(i/2)$
- **Complete tree height h :** $2^h \leq N \leq 2^{(h+1)}-1$
- **Height from N :** $h = \text{floor}(\log_2 N)$
- **Index 0:** Always unused (to make $2i$ work)

FREQUENTLY ASKED TOPICS

- Trace fastInorder on threaded tree with dummy node.
- Store complete binary tree in array — find children/parent.
- Compare threaded traversal vs recursive traversal.
- Level-order traversal and array index relationship.

- Complete binary tree height calculation.

COMMON MISTAKES

- Forgetting dummy node's right points to itself (not NULL).
- Using index 0 in array representation — should start from 1.
- Confusing complete binary tree with perfect binary tree.
- Thinking threaded traversal uses stack — it doesn't.
- Not resetting to root after each character in decoding (previous lecture).

One-Day-Before-Exam Summary: Threaded binary trees NULL pointers ko threads se replace karte hain — inorder traversal stack-free ho jaati hai. Dummy node trick root problem solve karta hai — `fastInorder(dummy)` call karte hain. Complete binary tree completely filled except bottom level — array mein store ho sakta hai (index 1 se start, $\text{left}=2i$, $\text{right}=2i+1$, $\text{parent}=\text{floor}(i/2)$). Arrays pointers se faster hain — compiler supported, paging issues less. Level-order traversal directly gives array indices.

Lesson 29: Heap Data Structure & Insertion

Complete binary tree + heap order — min heap, max heap, and insertion with percolate-up

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap ek company mein employees ki list manage kar rahe hain — jis employee ki salary sab se kam hai, woh top par aata hai (min heap). Agar koi naya employee aata hai jiski salary aur bhi kam hai, to woh upar chala jaata hai aur baqi neeche shift ho jaate hain — jaise heap insertion mein new element apni jagah paane ke liye upar (percolate-up) hota hai. Heap priority queue ko efficiently implement karta hai — sorting, operating systems, aur routers mein use hota hai.

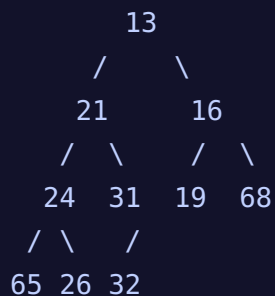
KEY TERMS

- 1 Heap** — Complete binary tree that conforms to heap order — used for priority queue.
- 2 Min Heap** — Parent node value $\leq$ children values — smallest value at root.
- 3 Max Heap** — Parent node value $\geq$ children values — largest value at root.

- 4 **Heap Order Property** — For every node X, parent key $\leq$ child key (min heap) or parent key $\geq$ child key (max heap).
- 5 **Percolate Up (Sift Up)** — Heap insertion mein new element ko upar move karna jab tak heap order satisfied na ho.
- 6 **Complete Binary Tree Property** — Heap uses complete binary tree — array storage with $2i$, $2i+1$ formulas.
- 7 **Priority Queue** — Data structure where elements have priority — heap is efficient implementation.
- 8 **Array Storage of Heap** — Heap stored in array — no pointers needed, index 1-based.

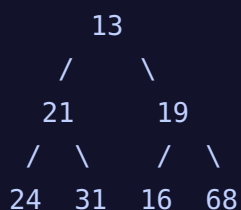
DIAGRAM TIME

► MIN HEAP (COMPLETE BINARY TREE + HEAP ORDER)



↗ Min heap: parent $\leq$ children. Root 13 smallest. Complete binary tree — array mein store ho sakta hai.

► NOT A HEAP (VIOLATION AT NODE 19)



```

    / \ /
   65 26 32

```

↗ Node 19 ka left child 16 hai — $16 < 19$, heap order violated. Not a valid min heap.

► MAX HEAP EXAMPLE

```

      73
     /  \
    52   63
   /  \  /  \
  31  26 40  25
 /  \  /
13 27 57

```

↗ Max heap: parent $\geq$ children. Root 73 largest. Used for priority where higher value = higher priority.

► HEAP INSERTION – STEP 1: ADD AT NEXT POSITION (14 AS RIGHT CHILD OF 31)

```

      13
     /  \
    21   16
   /  \  /  \
  24  31 19  68
 /  \  /  \
65 26 32 14

```

↗ New node 14 added at position 11 (right child of 31). Tree remains complete but heap order violated.

► HEAP INSERTION – STEP 2: PERCOLATE UP (14 VS 31 → SWAP)

```

      13
     /  \
    21   16
   /  \  /  \
  24  14 19  68

```

```

    / \   / \
   65 26 32 31

```

↗ $14 < 31 \rightarrow \text{swap}$. 14 now at position 5. Still heap order violated ($14 < 21$).

► HEAP INSERTION – STEP 3: PERCOLATE UP (14 VS 21 → SWAP)

```

      13
     /  \
    14   16
   /  \  /  \
  24 21 19 68
 /  \  /  \
65 26 32 31

```

↗ $14 < 21 \rightarrow \text{swap}$. 14 now at position 2. $13 < 14 \rightarrow \text{heap order satisfied}$.
Insertion complete.

► HEAP INSERTION – STEP 4: INSERT 15 (LEFT CHILD OF 19)

```

      13
     /  \
    14   16
   /  \  /  \
  24 21 19 68
 /  \  /  \ /
65 26 32 31 15

```

↗ New node 15 added at position 12 (left child of 19). $15 < 19 \rightarrow \text{swap}$. Then $15 < 16 \rightarrow \text{swap}$. Final position 3.

► ARRAY REPRESENTATION OF HEAP

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12
Value:	-	13	14	16	24	21	19	68	65	26	32	31	15

↗ Index 1 = root. Node at i : $\text{left}=2i$, $\text{right}=2i+1$, $\text{parent}=\text{floor}(i/2)$. No pointers needed.

Heap Connection: Heap = complete binary tree + heap order. Min heap mein parent $\leq$ children — root smallest. Insertion: new node at

next position (complete tree), then percolate up (swap with parent) until heap order satisfied. Array storage efficient — index formulas ($2i$, $2i+1$, $\text{floor}(i/2)$). Heap used for priority queue, sorting (heapsort), operating systems, routers.

CODE EXPLANATION

► HEAP INSERTION ALGORITHM (MIN HEAP)

```
void insertMinHeap(int heap[], int &n, int value) {
    n++;                // increase heap size
    heap[n] = value;    // place at end
    int i = n;
    while (i > 1 && heap[i] < heap[i/2]) {
        swap(heap[i], heap[i/2]); // swap with parent
        i = i / 2;                // move up
    }
}
```

- 1 **n++** — Heap size increase — new element at end.
- 2 **heap[n] = value** — New value inserted at last position.
- 3 **while (i > 1 && heap[i] < heap[i/2])** — Jab tak parent bara hai, swap karo.
- 4 **swap(heap[i], heap[i/2])** — Parent aur child ko swap.
- 5 **i = i / 2** — Upar move karo (parent position).
- 6 **Time Complexity** — $O(\log n)$ — height of complete binary tree.

► ARRAY FORMULAS FOR HEAP

```
// Node at position i (1-based indexing)
int leftChild(int i)    { return 2 * i; }
int rightChild(int i)   { return 2 * i + 1; }
int parent(int i)       { return i / 2; }    // integer division
```

- 1 **left = $2i$** — Left child position in array.
- 2 **right = $2i+1$** — Right child position.
- 3 **parent = $\text{floor}(i/2)$** — Parent position.
- 4 **Index 0 unused** — To make $2i$ formula work.

EXAM FOCUS POINTS

MCQ Heap is a complete binary tree that conforms to heap order.

MCQ Min heap: parent $\leq$ children — root has smallest value.

MCQ Max heap: parent $\geq$ children — root has largest value.

MCQ Heap insertion: add at next position $\rightarrow$ percolate up (swap with parent) until heap order satisfied.

FORMULA Node at i : left= $2i$, right= $2i+1$, parent= $\text{floor}(i/2)$.

FORMULA Heap insertion time = $O(\log n)$ — height of complete tree.

CONFUSION Heap $\neq$ Binary Search Tree — BST has left $<$ parent $<$ right, heap has parent $\leq$ children (no ordering between left and right).

PRACTICE Insert values 14, 15 into given min heap — trace percolate-up steps.

MIND MAP

Heap Data Structure & Insertion

Heap Definition

- Complete binary tree
- + Heap order property
- Min heap: parent $\leq$ children
- Max heap: parent $\geq$ children
- Root = min/max value

Insertion Process

- Add at next position
- Percolate up (sift up)
- Compare with parent
- Swap if violated
- Continue until root

Array Storage

- 1-based indexing
- Left = $2i$
- Right = $2i+1$
- Parent = $\text{floor}(i/2)$
- No pointers needed

Applications

- Priority queue
- Heapsort algorithm
- Operating systems
- Network routers
- Event simulation

CONCEPTUAL QUESTIONS & QUIZ

Q: What is a heap and what are its two essential properties?

A: Heap ek complete binary tree hai jo heap order property ko follow karta hai. Two properties: (1) Complete binary tree — sab levels filled except possibly bottom, bottom left to right. (2) Heap order — min heap mein parent $\leq$ children, max heap mein parent $\geq$ children.

Q: What is the difference between a min heap and a max heap?

A: Min heap mein parent node ki value children se chhoti ya barabar hoti hai — root smallest value hoti hai. Max heap mein parent node ki value children se badi ya barabar hoti hai — root largest value hoti hai.

Q: How does insertion work in a heap?

A: New value ko heap ke end mein add karte hain (complete tree property maintain). Phir percolate up (sift up) karte hain — parent se compare karte hain, agar parent bara hai to swap karte hain. Process tab tak repeat karte hain jab tak heap order satisfied na ho ya root tak pahunch jayein.

Q: What is the time complexity of heap insertion?

A: Heap insertion $O(\log n)$ time leta hai — kyunki complete binary tree ki height $\text{floor}(\log_2 n)$ hoti hai. Worst case mein new element root tak percolate up ho sakta hai.

Q: How is a heap stored in an array?

A: Heap complete binary tree hai, is liye array mein store kiya ja sakta hai — 1-based indexing. Node at position i : left child = $2i$, right child = $2i+1$, parent = $\text{floor}(i/2)$. Index 0 unused. No pointers needed — implicit links through formulas.

Q: What are the applications of heap data structure?

A: Heap used in priority queue (operating systems, routers, event simulation), heapsort algorithm, graph algorithms (Dijkstra, Prim), and finding k th smallest/largest elements.

► QUICK QUIZ (10 QUESTIONS)

1. Heap is a complete binary tree + ____ order?

(Heap)

- | | |
|--|-------------------------|
| 2. Min heap: parent $\leq$ ___ ? | (Children) |
| 3. Max heap: parent $\geq$ ___ ? | (Children) |
| 4. In min heap, root has ___ value? | (Smallest) |
| 5. Heap insertion time complexity? | ($O(\log n)$) |
| 6. Node at i: left child = ? | ($2i$) |
| 7. Node at i: parent = ? | ($\text{floor}(i/2)$) |
| 8. Heap insertion uses which operation? | (Percolate up) |
| 9. Heap is used for which data structure? | (Priority queue) |
| 10. Array index 0 is used in heap storage? | (No) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Heap:** Complete binary tree + heap order property.
- **Min Heap:** Parent $\leq$ children — root = smallest value.
- **Max Heap:** Parent $\geq$ children — root = largest value.
- **Heap Order Property:** For every node, parent key $\leq$ child key (min) or parent key $\geq$ child key (max).
- **Percolate Up (Sift Up):** New element ko upar move karna during insertion.

IMPORTANT CONCEPTS

- Heap is a complete binary tree — array storage possible.
- Insertion: add at end $\rightarrow$ percolate up until heap order satisfied.
- Time complexity: $O(\log n)$ for insertion.
- Heap used for priority queue — efficient than array-based sorting.
- Min heap root gives minimum element — useful for extracting min.
- Max heap root gives maximum element — useful for extracting max.

△ IMPORTANT FORMULAS

- **Node at i** (1-based): left = $2i$, right = $2i+1$, parent = $\text{floor}(i/2)$
- **Heap insertion:** $O(\log n)$
- **Heap height:** $\text{floor}(\log_2 n)$
- **Complete tree nodes:** 2^h to $2^{(h+1)}-1$

FREQUENTLY ASKED TOPICS

- Define heap and explain min heap vs max heap.
- Trace heap insertion with array representation.
- Show percolate-up steps with example.
- Store a heap in array using $2i$, $2i+1$ formulas.
- Applications of heap — priority queue, heapsort.
- Compare heap with BST.

COMMON MISTAKES

- Confusing heap with BST — heap has no left/right ordering.
- Forgetting that heap is a complete binary tree.
- Using index 0 in heap array — should start from 1.
- Not percolating up enough during insertion.
- Thinking heap insertion is $O(1)$ — it's $O(\log n)$.

One-Day-Before-Exam Summary: Heap complete binary tree hai jo heap order ko follow karta hai. Min heap mein parent $\leq$ children — root smallest. Max heap mein parent $\geq$ children — root largest. Insertion: new value at end, then percolate up (swap with parent) until heap order satisfied — $O(\log n)$. Array storage: 1-based indexing, left= $2i$, right= $2i+1$, parent= $\text{floor}(i/2)$. Heap used for priority queue, heapsort, operating systems, routers.

Lesson 30: Heap Insert/Delete & BuildHeap

Percolate up, percolate down, and linear-time heap construction

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap ek company mein employees ki list manage kar rahe hain — sab se chhoti salary wala top par aata hai (min heap). Jab koi naya employee aata hai (insertion), woh neeche se upar move karta hai (percolate up) jab tak uski sahi jagah na mil jaye. Jab top employee nikalta hai (deleteMin), ek "hole" upar se neeche move karta hai (percolate down) — chhote children upar aate hain. Agar aapko sab employees ek saath milte hain, to unhein $N\log N$ time mein heap nahi banate — balke $N/2$ se start karke percolate down karte hain, jo $O(N)$ time mein heap bana deta hai.

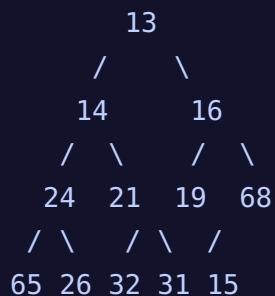
KEY TERMS

- 1 Percolate Up (Sift Up)** — Insertion mein new element ko upar move karna — parent se compare, swap if smaller (min heap).
- 2 Percolate Down (Sift Down)** — Deletion/buildHeap mein element ko neeche move karna — children se compare, swap with smaller child (min heap).

- 3 **deleteMin** — Min heap se root (minimum) delete karna — hole percolates down, last element fills hole.
- 4 **buildHeap** — N keys ko $O(N)$ time mein heap mein convert karna — start from $N/2$, percolate down each.
- 5 **Hole** — Deletion ke baad vacant position — percolate down se hole bottom tak move hota hai.
- 6 **Heap Sort** — buildHeap + repeated deleteMin — sorted data obtained in $O(N \log N)$.
- 7 **Linear Time Build** — buildHeap $O(N)$ time mein hota hai — $N/2$ se start, not N.

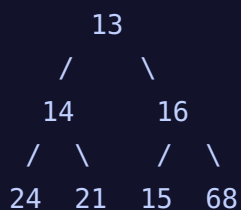
DIAGRAM TIME

► INSERT 15 IN MIN HEAP — STEP 1: ADD AT END (POSITION 12)



↗ 15 added as left child of 19 (position 12). $15 < 19 \rightarrow$ swap needed.

► INSERT 15 — STEP 2: PERCOLATE UP (SWAP WITH 19)




```

    / \   / \   /
   65 26 32 31 19

```

↗ 15 and 19 swapped. 15 now at position 6. $15 < 16 \rightarrow$ another swap needed.

► INSERT 15 – STEP 3: PERCOLATE UP (SWAP WITH 16) – FINAL

```

      13
     /  \
    14   15
   /  \  /  \
  24 21 16 68
 /  \  /  \  /
65 26 32 31 19

```

↗ 15 and 16 swapped. 15 at position 3. $13 < 15 \rightarrow$ heap order satisfied. Insertion complete.

► DELETEMIN – STEP 1: REMOVE ROOT (13), HOLE AT ROOT

```

      [hole]
     /      \
    14       15
   /  \    /  \
  24 21  16 68
 /  \  /  \  /
65 26 32 31 19

```

↗ Root 13 removed. Hole at root. Compare children 14 and 15 $\rightarrow$ 14 moves up.

► DELETEMIN – STEP 2: 14 MOVES UP, HOLE MOVES DOWN

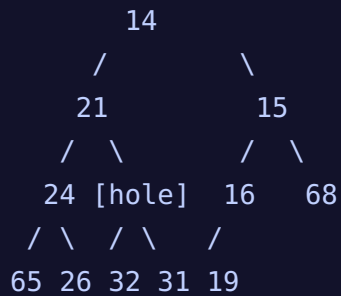
```

      14
     /  \
    [hole] 15
   /  \    /  \
  24 21  16 68
 /  \  /  \  /
65 26 32 31 19

```

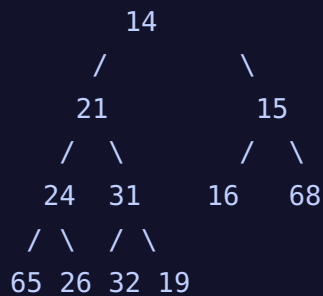
↗ 14 moved up. Hole at position 2. Children 24 and 21 $\rightarrow$ 21 moves up.

► DELETEMIN – STEP 3: 21 MOVES UP, HOLE CONTINUES DOWN



↗ 21 moved up. Hole at position 5. Children 32 and 31 → 31 moves up.

► DELETEMIN – STEP 4: 31 MOVES UP, HOLE AT LEAF – DELETE



↗ 31 moved up. Hole at leaf position. Remove last element (19) → heap size reduced.

► BUILDHEAP – INITIAL UNORDERED ARRAY (N=15)

```

Index: 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
Value: 65 31 32 26 21 19 68 13 24 15 14 16  5 70 12
    
```

↗ Unordered complete binary tree. Need to convert to min heap.

► BUILDHEAP – START FROM $I = N/2 = 7$ (VALUE 68)

```

percolateDown(7): 68 has children 70, 12 → swap 68 with 12
percolateDown(6): 19 has children 16, 5 → swap 19 with 5
percolateDown(5): 21 has children 15, 14 → swap 21 with 14
percolateDown(4): 26 has children 13, 24 → swap 26 with 13
percolateDown(3): 32 has children 5, 12 → swap 32 with 5 (then 5 has
    
```

↗ Continue until $i=1$. Result: min heap in $O(N)$ time.

Heap Connection: Insertion = percolate up ($O(\log N)$). Deletion = percolate down ($O(\log N)$). buildHeap = percolate down from $N/2$ down to 1 ($O(N)$ time). This is more efficient than N successive inserts ($O(N \log N)$). Heap used for priority queue and heapsort.

CODE EXPLANATION

► HEAP INSERTION (MIN HEAP)

```
void insertMinHeap(int heap[], int &n, int value) {
    n++;
    heap[n] = value;
    int i = n;
    while (i > 1 && heap[i] < heap[i/2]) {
        swap(heap[i], heap[i/2]);
        i = i / 2;
    }
}
```

- 1 **n++** — Heap size increase.
- 2 **heap[n] = value** — New value at end.
- 3 **while (i > 1 && heap[i] < heap[i/2])** — Percolate up.
- 4 **swap(heap[i], heap[i/2])** — Swap with parent.
- 5 **i = i / 2** — Move up.

► DELETEMIN (MIN HEAP)

```
int deleteMin(int heap[], int &n) {
    int min = heap[1];
    heap[1] = heap[n];    // move last to root
    n--;
    percolateDown(heap, n, 1);
    return min;
}
```

- 1 **min = heap[1]** — Root (minimum) save.
- 2 **heap[1] = heap[n]** — Last element to root.
- 3 **n--** — Reduce size.
- 4 **percolateDown(heap, n, 1)** — Root ko correct position par le jao.

► PERCOLATEDOWN (SIFT DOWN)

```
void percolateDown(int heap[], int n, int i) {
    while (2*i <= n) {
        int j = 2*i;                // left child
        if (j+1 <= n && heap[j+1] < heap[j])
            j = j+1;                // smaller child
        if (heap[i] <= heap[j]) break;
        swap(heap[i], heap[j]);
        i = j;
    }
}
```

- 1 **j = 2*i** — Left child.
- 2 **if (j+1 <= n && heap[j+1] < heap[j])** — Right child smaller.
- 3 **if (heap[i] <= heap[j]) break** — Heap order satisfied.
- 4 **swap(heap[i], heap[j])** — Swap with smaller child.
- 5 **i = j** — Move down.

► BUILDHEAP (LINEAR TIME)

```
void buildHeap(int heap[], int n) {
    for (int i = n/2; i >= 1; i--)
        percolateDown(heap, n, i);
}
```

- 1 **i = n/2** — Start from last internal node.
- 2 **percolateDown(heap, n, i)** — Each node ko correct position par.

3 Time Complexity — $O(N)$ — linear time.

EXAM FOCUS POINTS

MCQ Insertion in heap: add at end → percolate up ($O(\log N)$).

MCQ deleteMin: root removed → last element to root → percolate down ($O(\log N)$).

MCQ buildHeap: start from $N/2$ down to 1 → percolate down each ($O(N)$ time).

CONFUSION N successive inserts = $O(N \log N)$. buildHeap = $O(N)$ — much faster.

FORMULA $N/2$ is first internal node (parent of last leaf).

PRACTICE Trace insert(15) in given min heap — show percolate up steps.

PRACTICE Trace deleteMin on a min heap — show percolate down steps.

PRACTICE Build heap from array
[65,31,32,26,21,19,68,13,24,15,14,16,5,70,12] using
buildHeap.

MIND MAP

Heap Operations & BuildHeap

Insertion

- Add at end
- Percolate up
- Compare with parent

- Swap if violated
- $O(\log N)$

Deletion (deleteMin)

- Remove root
- Last element to root
- Percolate down
- Compare with children
- Swap with smaller
- $O(\log N)$

buildHeap

- Start from $N/2$
- Down to 1
- Percolate down each
- $O(N)$ time
- Linear time!

Applications

- Priority queue
- Heap sort
- $O(N)$ build
- $O(\log N)$ insert/delete

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: What is the difference between percolate up and percolate down?

A: Percolate up insertion mein use hota hai — new element ko upar move karte hain parent se compare kar ke. Percolate down deletion/buildHeap mein use hota hai — element ko neeche move karte hain children se compare kar ke. Min heap mein percolate up smaller element upar laata hai, percolate down larger element neeche bhejta hai.

Q: How does deleteMin work in a heap?

A: Root (minimum) ko save karte hain. Last element ko root par laate hain. Heap size reduce karte hain. Phir percolate down karte hain — root ko children se compare, smaller child se swap, tab tak jab tak heap order satisfied na ho. Root return karte hain (minimum value).

Q: Why does buildHeap start from $N/2$ instead of N ?

A: $N/2$ se neeche wale nodes leaves hain (no children). Leaves already satisfy heap property. Is liye hum first internal node ($N/2$) se start karte hain. This is why buildHeap $O(N)$ time mein complete ho jaata hai — hum sirf internal nodes par percolate down karte hain.

Q: What is the time complexity of buildHeap?

A: buildHeap $O(N)$ time mein hota hai — linear time. N successive inserts $O(N \log N)$ lete hain. buildHeap faster hai kyunki hum internal nodes ($N/2$) par percolate down karte hain aur har node ki height ke hisaab se kaam hota hai.

Q: What is the hole in heap deletion and how does it move?

A: deleteMin mein root remove karne par hole (vacant position) root par banti hai. Percolate down mein hole apne children ke saath compare hota hai — smaller child hole mein upar move hota hai, hole neeche move hota hai. Hole tab tak move karta hai jab tak leaf level par na pahunch jaye, phir last element ko remove kar dete hain.

Q: What is heap sort and how does it work?

A: Heap sort ek sorting algorithm hai — pehle buildHeap se min heap banate hain ($O(N)$), phir repeatedly deleteMin karte hain ($O(\log N)$ each). Total $O(N \log N)$ time. Data sorted order mein nikalta hai — har baar root (minimum) nikalta hai.

► QUICK QUIZ (10 QUESTIONS)

1. Insertion in heap uses which operation? **(Percolate up)**
2. Deletion in heap uses which operation? **(Percolate down)**
3. deleteMin removes which element? **(Root/Minimum)**
4. buildHeap starts from which index? **(N/2)**
5. buildHeap time complexity? **(O(N))**
6. Insertion time complexity? **(O(log N))**
7. deleteMin time complexity? **(O(log N))**
8. N successive inserts time complexity? **(O(N log N))**
9. In percolate down, compare with which children? **(Both left and right)**
10. Heap sort uses which operations? **(buildHeap + deleteMin)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Percolate Up:** Insertion mein element ko upar move karna — parent se compare.
- **Percolate Down:** Deletion/buildHeap mein element ko neeche move karna — children se compare.
- **deleteMin:** Min heap se root (minimum) delete karna — percolate down.
- **buildHeap:** N keys ko O(N) time mein heap mein convert karna — start from N/2.
- **Hole:** Deletion ke baad vacant position — percolate down se bottom tak move.

IMPORTANT CONCEPTS

- Insertion: add at end → percolate up until heap order satisfied.
- Deletion: root removed → last to root → percolate down.
- buildHeap: N/2 to 1 → percolate down each → O(N) time.
- N successive inserts = O(N log N) — slower than buildHeap.

- Heap sort: buildHeap + repeated deleteMin $\rightarrow O(N \log N)$.
- Heap used for priority queue — efficient insert/delete.

△ IMPORTANT FORMULAS

- **Insertion:** $O(\log N)$
- **deleteMin:** $O(\log N)$
- **buildHeap:** $O(N)$
- **N successive inserts:** $O(N \log N)$
- **Heap sort:** $O(N \log N)$
- **First internal node:** $N/2$

FREQUENTLY ASKED TOPICS

- Trace insert operation with percolate up.
- Trace deleteMin with percolate down.
- Build heap from array using buildHeap.
- Compare buildHeap vs N successive inserts.
- Why buildHeap starts at $N/2$.
- Heap sort algorithm.

COMMON MISTAKES

- Forgetting that buildHeap starts from $N/2$, not N .
- Thinking buildHeap is $O(N \log N)$ — it's $O(N)$.
- In percolate down, only comparing with left child — should compare with both.
- Not handling case when right child doesn't exist.
- Forgetting to reduce heap size in deleteMin.

One-Day-Before-Exam Summary: Heap operations: insert (percolate up, $O(\log N)$), deleteMin (percolate down, $O(\log N)$), buildHeap (percolate down from $N/2$, $O(N)$). buildHeap N successive inserts se faster hai. Percolate up: new element upar move karta hai. Percolate down: element neeche move karta hai, smaller child upar. Heap sort: buildHeap + repeated deleteMin. Heap priority queue ke liye best data structure hai.

Lesson 31: BuildHeap, Heap Methods & C++ Code

$O(N)$ heap construction, decreaseKey/increaseKey/remove, and Heap class implementation

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aapke paas 15 employees ki list hai aur aap unhein salary ke hisaab se arrange karna chahte hain (min heap). Agar aap ek ek karke insert karte hain, to time lagta hai ($N \log N$). Lekin agar aap sab ko ek saath arrange karte hain — sab se neeche walon se start karte hain ($N/2$) aur upar aate hain — to $O(N)$ time mein kaam ho jaata hai. Jaise agar aap sab employees ko pehle ek list mein likh lein, phir har internal manager ko apni team mein se sab se chhoti salary wala upar rakhne ko kahein — yeh buildHeap hai.

KEY TERMS

- 1 **buildHeap** — N keys ko $O(N)$ time mein heap mein convert karna — $N/2$ se start, percolateDown each.
- 2 **decreaseKey(p, delta)** — Node at position p ka value δ se kam karna — percolate up (min heap).

- 3 **increaseKey(p, delta)** — Node at position p ka value delta se badhana — percolate down (min heap).
- 4 **remove(p)** — Node at position p ko delete karna — decreaseKey(p, $-\infty$) + deleteMin().
- 5 **Heap Class (C++)** — Template class with insert, deleteMin, getMin, isEmpty, isFull, getSize.
- 6 **Percolate Up in Insert** — hole moves up, parent moves down — find final position first for efficiency.
- 7 **Priority Queue** — Heap-based implementation — processes, events, scheduling.

DIAGRAM TIME

► BUILDHEAP – STEP BY STEP (N=15, START AT I=7)

```
i=7: 68 with children 70,12 → swap 68↔12 → subtree rooted at 12 is heap
i=6: 19 with children 16,5 → swap 19↔5 → 5 up, 19 down
i=5: 21 with children 15,14 → swap 21↔14 → 14 up
i=4: 26 with children 13,24 → swap 26↔13 → 13 up
i=3: 32 with children 5,12 → swap 32↔5 → then 32 with 19 → swap
```

↗ Har internal node (N/2 se 1 tak) par percolateDown — smaller values up, larger down.

► BUILDHEAP – FINAL MIN HEAP

```
      5
     / \
    13  12
   / \ / \
  24 14 16 21
 / \ / \ /
65 26 31 15 19
```

/ \
70 68

↗ Final min heap — root is smallest. All parent $\leq$ children. $O(N)$ time mein build.

► DECREASEKEY(P, DELTA) — EXAMPLE

Before: node 17 at position p
decreaseKey(p, 10) → node becomes 7
7 < parent 12 → percolate up
After: 7 moves up, heap order restored

↗ Value decrease karne par node upar move karta hai (percolate up in min heap).

► INCREASEKEY(P, DELTA) — EXAMPLE

Before: node 10 at position p
increaseKey(p, 5) → node becomes 15
15 > child 12 → percolate down
After: 15 moves down, smaller child up

↗ Value increase karne par node neeche move karta hai (percolate down in min heap).

► REMOVE(P) — USING DECREASEKEY + DELETEMIN

Step 1: decreaseKey(p, $-\infty$) → node becomes $-\infty$
Step 2: $-\infty$ percolates up to root
Step 3: deleteMin() removes root (which is the node)
Result: Node at position p deleted, heap restored

↗ remove(p): value ko $-\infty$ karo, percolate up, phir deleteMin — $O(\log N)$.

Heap Connection: buildHeap $O(N)$ time mein heap banata hai — $N/2$ se start. decreaseKey/increaseKey priority queue operations ke liye useful hain — process scheduling mein use hoti hain. remove(p) = decreaseKey + deleteMin. Heap class template-based hai — any data type store kar sakte hain.

CODE EXPLANATION

► HEAP CLASS HEADER (HEAP.H)

```
template <class eType>
class Heap {
public:
    Heap(int capacity = 100);
    void insert(const eType &x);
    void deleteMin(eType &minItem);
    const eType &getMin();
    bool isEmpty();
    bool isFull();
    int getSize();
private:
    int currentSize;
    eType* array;
    int capacity;
    void percolateDown(int hole);
};
```

- 1 **template <class eType>** — Any data type ke liye generic heap.
- 2 **Heap(int capacity = 100)** — Constructor with default capacity 100.
- 3 **insert(const eType &x)** — const reference (copy avoid).
- 4 **deleteMin(eType &minItem)** — Min delete, value in minItem.
- 5 **getMin()** — Returns const reference to minimum.
- 6 **array = new eType[capacity + 1]** — Index 0 unused, so capacity+1.

► INSERT METHOD (PERCOLATE UP)

```
template <class eType>
bool Heap<eType>::insert(const eType &x) {
    if (isFull()) return false;
    int hole = ++currentSize;
    for (; hole > 1 && x < array[hole/2]; hole /= 2)
        array[hole] = array[hole/2];
    array[hole] = x;
```

```
    return true;
}
```

- 1 **hole = ++currentSize** — Next position (end of array).
- 2 **for (; hole > 1 && x < array[hole/2]; hole /= 2)** — Percolate up.
- 3 **array[hole] = array[hole/2]** — Parent moves down.
- 4 **array[hole] = x** — Final position mein insert.

► DELETEMIN METHOD

```
template <class eType>
void Heap<eType>::deleteMin(eType &minItem) {
    if (isEmpty()) return;
    minItem = array[1];
    array[1] = array[currentSize--];
    percolateDown(1);
}
```

- 1 **minItem = array[1]** — Save minimum (root).
- 2 **array[1] = array[currentSize--]** — Last element to root, size--.
- 3 **percolateDown(1)** — Restore heap order.

► PERCOLATEDOWN METHOD

```
template <class eType>
void Heap<eType>::percolateDown(int hole) {
    int child;
    eType tmp = array[hole];
    for (; hole*2 <= currentSize; hole = child) {
        child = hole*2;
        if (child != currentSize && array[child+1] < array[child])
            child++;
        if (array[child] < tmp)
            array[hole] = array[child];
        else break;
    }
}
```

```
array[hole] = tmp;  
}
```

- 1 **tmp = array[hole]** — Save value to percolate down.
- 2 **child = hole*2** — Left child.
- 3 **if (array[child+1] < array[child]) child++** — Smaller child.
- 4 **if (array[child] < tmp)** — Swap if child smaller.

EXAM FOCUS POINTS

MCQ buildHeap starts from $N/2$ — leaves already satisfy heap property.

MCQ buildHeap time complexity = $O(N)$ — linear time.

MCQ decreaseKey(p, delta) — value decrease → percolate up (min heap).

MCQ increaseKey(p, delta) — value increase → percolate down (min heap).

MCQ remove(p) = decreaseKey(p, $-\infty$) + deleteMin().

FORMULA N successive inserts = $O(N \log N)$, buildHeap = $O(N)$.

CONFUSION decreaseKey in min heap uses percolate up (not down).
increaseKey uses percolate down.

PRACTICE Trace buildHeap on array
[65,31,32,26,21,19,68,13,24,15,14,16,5,70,12].

BuildHeap, Heap Methods & C++ Code

buildHeap

- Start from $N/2$
- percolateDown each
- $O(N)$ time
- Leaves skipped
- Better than N inserts

Heap Methods

- decreaseKey: percolate up
- increaseKey: percolate down
- remove: $-\infty$ + deleteMin
- Used in priority queues

C++ Class

- Template based
- insert (percolate up)
- deleteMin (percolate down)
- getMin, isEmpty
- isFull, getSize

Applications

- Priority queues
- Process scheduling
- Heap sort
- Event simulation

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Why does buildHeap start from $N/2$ instead of N ?

A: $N/2$ se neeche wale nodes leaves hain — unka koi child nahi, is liye woh already heap property satisfy karte hain. Hum sirf internal nodes ($N/2$ se 1 tak) par percolateDown karte hain. Is liye buildHeap $O(N)$ time mein complete ho jaata hai.

Q: What is the difference between decreaseKey and increaseKey?

A: decreaseKey(p , delta) node ki value kam karta hai — min heap mein value kam hone par node upar move karti hai (percolate up). increaseKey(p , delta) node ki value badhata hai — value badhne par node neeche move karti hai (percolate down).

Q: How does remove(p) work in a heap?

A: remove(p) three steps mein hota hai: (1) decreaseKey(p , $-\infty$) — node ki value ko minimum possible karte hain. (2) Node percolate up ho kar root par aa jaata hai. (3) deleteMin() call karte hain — root delete ho jaata hai (jo humein chahiye tha). Time complexity $O(\log N)$.

Q: Why is buildHeap better than N successive inserts?

A: N successive inserts $O(N \log N)$ time lete hain — har insert $O(\log N)$. buildHeap $O(N)$ time mein kaam karta hai — hum internal nodes ($N/2$) par percolateDown karte hain. Is liye buildHeap faster hai jab saara data ek saath available ho.

Q: What is the purpose of using templates in Heap class?

A: Templates Heap ko generic banate hain — hum any data type (int, float, string, custom objects) store kar sakte hain. eType parameter ki jagah actual type substitute hota hai. Code reuse hota hai — ek baar likho, multiple types ke liye kaam kare.

Q: How does the insert method find the final position efficiently?

A: insert method pehle hole ko end mein rakhta hai. For loop mein parent se compare karta hai — agar x parent se chhota hai to parent neeche move karta hai, hole upar move karta hai. Swapping nahi karta — sirf values shift karta hai. Final position milne par x ko hole mein daal deta hai.

► QUICK QUIZ (10 QUESTIONS)

1. buildHeap starts from which index? **(N/2)**
2. buildHeap time complexity? **(O(N))**
3. N successive inserts time complexity? **(O(N log N))**
4. decreaseKey uses which percolate operation? **(Percolate up)**
5. increaseKey uses which percolate operation? **(Percolate down)**
6. remove(p) = decreaseKey + ___ ? **(deleteMin)**
7. Heap class uses which C++ feature for genericity? **(Templates)**
8. In insert, hole moves in which direction? **(Up)**
9. In deleteMin, percolateDown moves element in which direction? **(Down)**
10. Heap is primarily used for which data structure? **(Priority queue)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **buildHeap:** N keys ko O(N) time mein heap mein convert karna — N/2 se start.
- **decreaseKey(p, delta):** Node value decrease — percolate up (min heap).
- **increaseKey(p, delta):** Node value increase — percolate down (min heap).
- **remove(p):** Node delete — decreaseKey(-∞) + deleteMin.
- **Heap Class:** Template-based implementation with insert, deleteMin, getMin, isEmpty, isFull, getSize.

IMPORTANT CONCEPTS

- **buildHeap** $O(N)$ — faster than N successive inserts ($O(N \log N)$).
- Leaves already satisfy heap property — start from $N/2$.
- **decreaseKey**: smaller value $\rightarrow$ percolate up.
- **increaseKey**: larger value $\rightarrow$ percolate down.
- **remove**: make minimum $\rightarrow$ **deleteMin**.
- Heap used in priority queue, process scheduling, heapsort.
- Templates make heap generic — any data type.

△ IMPORTANT FORMULAS

- **buildHeap**: $O(N)$
- **N successive inserts**: $O(N \log N)$
- **decreaseKey**: $O(\log N)$
- **increaseKey**: $O(\log N)$
- **remove**: $O(\log N)$
- **insert**: $O(\log N)$
- **deleteMin**: $O(\log N)$

FREQUENTLY ASKED TOPICS

- Trace **buildHeap** on given array.
- Explain why **buildHeap** starts from $N/2$.
- Compare **buildHeap** vs N successive inserts.
- Explain **decreaseKey** and **increaseKey** with examples.
- Explain **remove(p)** using **decreaseKey** + **deleteMin**.
- Heap class implementation — constructor, **insert**, **deleteMin**.
- Why templates are used in Heap class.

COMMON MISTAKES

- Thinking **buildHeap** is $O(N \log N)$ — it's $O(N)$.
- Using percolate down for **decreaseKey** — should use percolate up.
- Using percolate up for **increaseKey** — should use percolate down.
- Forgetting that **remove** uses **decreaseKey** + **deleteMin**.
- Not using templates for generic heap.

- Forgetting index 0 is unused in heap array.

One-Day-Before-Exam Summary: buildHeap $O(N)$ time mein heap banata hai — $N/2$ se percolateDown karte hain. Leaves already heap hain. decreaseKey = percolate up, increaseKey = percolate down. remove(p) = decreaseKey(p, $-\infty$) + deleteMin. Heap class template-based hai — insert (percolate up), deleteMin (percolate down), getMin, isEmpty, isFull, getSize. Heap priority queue ke liye best data structure hai — $O(\log N)$ insert/delete, $O(N)$ build.

Lesson 32: percolateDown, buildHeap & Linear Time Proof

Detailed code analysis — mathematical proof that buildHeap is $O(N)$

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aapke paas 31 employees hain aur aap unhein salary ke hisaab se arrange karna chahte hain (min heap). buildHeap $O(N)$ time mein kaam karta hai — jaise aap sab se neeche walon se start karte hain aur upar aate hain. Mathematical proof kehti hai ke sum of heights of all nodes $\approx N$ — is liye buildHeap linear time hai. Jaise agar aap perfect binary tree mein har node ki height count karein, to total height $\approx N, \log N$ insignificant ho jaata hai. Is liye buildHeap fast hai.

KEY TERMS

- 1 **percolateDown** — Hole ko neeche move karna — children se compare, smaller child up, hole down.
- 2 **getMin** — Minimum value return (root) — without deleting — like stack's top.

- 3 **buildHeap** — Array se heap build karna — copy array, then percolateDown from $N/2$ to 1.
- 4 **Sum of Heights (S)** — Perfect binary tree of height h : $S = 2^{(h+1)} - 1 - (h+1) = N - h - 1$.
- 5 **Linear Time Proof** — $S \approx N$ — $\log N$ term insignificant for large N — buildHeap is $O(N)$.
- 6 **Perfect Binary Tree** — All levels completely filled — height h has $2^{(h+1)} - 1$ nodes.
- 7 **Marked Edges Proof** — Visual proof: mark left edge then right edges — dark edges count = S .

DIAGRAM TIME

► PERFECT BINARY TREE — NODES BY HEIGHT

Height h : 1 node (root)
Height $h-1$: 2 nodes
Height $h-2$: 4 nodes
Height $h-3$: 8 nodes
...
Height 0: 2^h nodes (leaves)

↗ Perfect binary tree mein height h ke nodes ki count $2^{(h-i)}$ hai — sum of heights proof ke liye use.

► SUM OF HEIGHTS FORMULA

$S = \sum 2^i (h - i), i = 0 \text{ to } h-1$
 $S = h + 2(h-1) + 4(h-2) + 8(h-3) + \dots + 2^{(h-1)}(1)$
Solving: $S = 2^{(h+1)} - 1 - (h+1)$
 $S = N - h - 1$ (where $N = 2^{(h+1)} - 1$)

↗ Sum of heights (S) = $N - h - 1$ — for large N , $S \approx N$, so buildHeap is $O(N)$.

► VISUAL PROOF – STEP 1: MARK LEFT EDGES FOR HEIGHT 1 NODES



↗ Height 1 nodes ki left edges mark karein (gray). Leaves height 0 — no marking.

► VISUAL PROOF – STEP 2: MARK LEFT + RIGHT EDGES FOR HEIGHT 2 NODES



↗ Height 2 nodes: mark first left edge, then right edge down to leaf.

► VISUAL PROOF – STEP 3: MARK LEFT + 2 RIGHT EDGES FOR HEIGHT 3 NODES



↗ Height 3 nodes: mark first left edge, then 2 right edges. Continue for all heights.

► VISUAL PROOF – ALL MARKED EDGES (N=31, H=4)

Dark edges = 26 (S)
Dotted edges = 4 (height h)

Total edges = 30 (N-1)

$S = N - h - 1 = 31 - 4 - 1 = 26 \checkmark$

↗ $N=31, h=4$. Marked edges (dark) = 26 = sum of heights. Dotted edges = 4 = height. Proves theorem visually.

Heap Connection: buildHeap $O(N)$ time mein hota hai — mathematical proof se proved. Sum of heights $S \approx N$ for large N . is liye percolateDown operations total $O(N)$ hain. N successive inserts $O(N \log N)$ se better. Visual proof: mark left edge + right edges for each node — dark edges count = sum of heights = $N - h - 1$.

CODE EXPLANATION

► PERCOLATEDOWN METHOD (DETAILED)

```
template <class eType>
void Heap<eType>::percolateDown(int hole) {
    int child;
    eType tmp = array[hole];
    for (; hole * 2 <= currentSize; hole = child) {
        child = hole * 2; // left child
        if (child != currentSize && array[child+1] < array[child])
            child++;      // right child is smaller
        if (array[child] < tmp)
            array[hole] = array[child]; // child moves up
        else
            break;          // tmp is in correct position
    }
    array[hole] = tmp; // place tmp at final position
}
```

- 1 **tmp = array[hole]** — Save value to percolate down.
- 2 **for (; hole*2 <= currentSize; hole = child)** — Loop until leaf.
- 3 **child = hole * 2** — Left child position.
- 4 **if (array[child+1] < array[child]) child++** — Right child smaller.

- 5 **if (array[child] < tmp) array[hole] = array[child]** — Child moves up.
- 6 **else break** — tmp smaller, correct position found.
- 7 **array[hole] = tmp** — Place tmp at final position (no swap needed).

► GETMIN METHOD

```
template <class eType>
const eType& Heap<eType>::getMin() {
    if (!isEmpty())
        return array[1];
}
```

- 1 **return array[1]** — Root (minimum) return, without deleting.
- 2 **const reference** — No copy, read-only access.

► BUILDHEAP METHOD

```
template <class eType>
void Heap<eType>::buildHeap(eType* anArray, int n) {
    for (int i = 1; i <= n; i++)
        array[i] = anArray[i-1];    // copy (0-based to 1-based)
    currentSize = n;
    for (int i = currentSize / 2; i > 0; i--)
        percolateDown(i);           // build heap in O(N)
}
```

- 1 **for (int i = 1; i <= n; i++)** — Copy array (anArray 0-based to internal 1-based).
- 2 **currentSize = n** — Set heap size.
- 3 **for (int i = currentSize/2; i > 0; i--)** — Start from last internal node.
- 4 **percolateDown(i)** — Each internal node to correct position par.

► HELPER METHODS

```
template <class eType>
bool Heap<eType>::isEmpty() { return currentSize == 0; }
```

```
template <class eType>
bool Heap<eType>::isFull() { return currentSize == capacity; }

template <class eType>
int Heap<eType>::getSize() { return currentSize; }
```

- 1 **isEmpty** — Check if heap has no elements.
- 2 **isFull** — Check if heap array is full.
- 3 **getSize** — Return current number of elements.

EXAM FOCUS POINTS

MCQ percolateDown — hole moves down, smaller child moves up. No swap, just assignment.

MCQ getMin — returns root (minimum) without deleting — $O(1)$ time.

MCQ buildHeap — copies array (0-based to 1-based), then percolateDown from $N/2$.

FORMULA Sum of heights $S = N - h - 1$ (perfect binary tree).

FORMULA buildHeap time = $O(N)$ — linear time.

CONFUSION percolateDown mein swapping nahi — assignment use karte hain (faster).

PRACTICE Trace percolateDown on a heap — show hole movement.

PRACTICE Prove buildHeap $O(N)$ using sum of heights formula.

percolateDown, buildHeap & Linear Time Proof

percolateDown

- Hole moves down
- Smaller child moves up
- No swap (assignment)
- $O(\log N)$ per call

getMin

- Returns root
- Without deleting
- $O(1)$ time
- Like stack's top

buildHeap

- Copy array (1-based)
- percolateDown from $N/2$
- $O(N)$ time
- Linear time!

Proof: $O(N)$

- $S = \sum \text{heights}$
- $S = N - h - 1$
- For large N , $S \approx N$
- $\text{buildHeap} = O(N)$

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Why does percolateDown use assignment instead of swapping?

A: Swapping uses 3 statements (temp, assign, assign). Assignment uses 1 statement. percolateDown mein hum tmp value save karte hain aur sirf child values ko upar move karte hain. Final position par tmp ko daal dete hain. Is tarah $O(\log N)$ time mein kaam ho jata hai aur faster hai.

Q: What is the difference between getMin and deleteMin?

A: getMin root (minimum) value return karta hai without deleting — $O(1)$ time. deleteMin root delete karta hai, last element root par laata hai, phir percolateDown karta hai — $O(\log N)$ time.

Q: How does buildHeap copy the input array?

A: Input array anArray 0-based hai (index 0 se start). Internal array 1-based hai (index 1 se start). buildHeap for loop mein anArray[i-1] ko array[i] mein copy karta hai. Is tarah $2i$, $2i+1$ formulas easily apply ho jate hain.

Q: What is the sum of heights theorem for perfect binary trees?

A: Perfect binary tree of height h with $N = 2^{(h+1)} - 1$ nodes — sum of heights $S = N - h - 1$. Mathematically proved: $S = h + 2(h-1) + 4(h-2) + \dots + 2^{(h-1)}(1) = 2^{(h+1)} - 1 - (h+1) = N - h - 1$.

Q: How does the sum of heights theorem prove buildHeap is $O(N)$?

A: buildHeap mein percolateDown operations total time = sum of heights of all nodes (S). Theorem says $S = N - h - 1$. For large N , $\log N$ term ($h+1$) becomes insignificant, so $S \approx N$. Is liye buildHeap $O(N)$ time mein hota hai — linear time.

Q: What is the visual proof of the sum of heights theorem?

A: Perfect binary tree mein har node ke liye: left edge mark karo, phir right edges mark karo down to leaf. Dark edges count = sum of heights (S). Dotted edges = h (height). Total edges = N-1. So $S + h = N-1 \rightarrow S = N - h - 1$. Visual proof confirms mathematical proof.

► QUICK QUIZ (10 QUESTIONS)

1. percolateDown moves hole in which direction? **(Down)**
2. getMin returns which element? **(Root/Minimum)**
3. buildHeap copies array to which index? **(1)**
4. buildHeap starts percolateDown from which index? **(N/2)**
5. Sum of heights $S = N - \underline{\quad} - 1$? **(h)**
6. buildHeap time complexity? **(O(N))**
7. percolateDown uses swapping or assignment? **(Assignment)**
8. getMin time complexity? **(O(1))**
9. In visual proof, dark edges count = ? **(Sum of heights)**
10. Dotted edges in visual proof = ? **(Height h)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **percolateDown:** Hole ko neeche move karna — smaller child up, hole down. No swap, assignment only.
- **getMin:** Minimum value return — without deleting — O(1) time.
- **buildHeap:** Array se heap build — copy array, percolateDown from N/2 — O(N).
- **Sum of Heights (S):** Perfect binary tree: $S = N - h - 1$ — proves linear time.
- **Perfect Binary Tree:** All levels filled — height h has $2^{h+1}-1$ nodes.

IMPORTANT CONCEPTS

- `percolateDown` uses assignment (not swap) for efficiency.
- `getMin` = $O(1)$ — root access.
- `buildHeap` = $O(N)$ — linear time construction.
- Sum of heights theorem: $S = N - h - 1$.
- For large N , $S \approx N$ — $\log N$ term insignificant.
- Visual proof: mark left + right edges — dark edges = S .

△ IMPORTANT FORMULAS

- **Sum of heights:** $S = N - h - 1$
- **Perfect tree nodes:** $N = 2^{(h+1)} - 1$
- **buildHeap:** $O(N)$
- **percolateDown:** $O(\log N)$ per call
- **getMin:** $O(1)$
- **N successive inserts:** $O(N \log N)$

FREQUENTLY ASKED TOPICS

- Trace `percolateDown` on a heap.
- Explain `getMin` vs `deleteMin`.
- Trace `buildHeap` on given array.
- Prove `buildHeap` is $O(N)$ using sum of heights.
- Visual proof of sum of heights theorem.
- Why `percolateDown` uses assignment not swap.

COMMON MISTAKES

- Thinking `percolateDown` uses swapping — it uses assignment.
- Confusing `getMin` with `deleteMin` — `getMin` doesn't delete.
- Forgetting `buildHeap` copies from 0-based to 1-based.
- Thinking `buildHeap` is $O(N \log N)$ — it's $O(N)$.
- Not understanding sum of heights proof.

One-Day-Before-Exam Summary: `percolateDown` hole ko neeche move karta hai — smaller child up, assignment use (no swap). `getMin` $O(1)$ root return. `buildHeap` array copy + `percolateDown` from $N/2 =$

$O(N)$. Sum of heights $S = N - h - 1$ proves linear time — for large N , $S \approx N$. Visual proof: mark left edge + right edges — dark edges = S .
buildHeap N successive inserts se better hai — $O(N)$ vs $O(N \log N)$.

Lesson 33: Priority Queue, Selection, Heap Sort & Disjoint Set

Heap-based priority queue — kth smallest element — heapsort — disjoint set ADT & equivalence relations

DEEP EXPLANATION — CHAI-TIME ANALOGY

Yeh bilkul waisi hi baat hai jaise aap ek hospital mein emergency patients ki list manage kar rahe hain — jis patient ki condition zyada serious hai, woh pehle aata hai (priority queue). Heap is priority queue ko efficiently implement karta hai. Agar aapko 1000 patients mein se 10th sab se serious patient find karna hai (selection problem), to heap use kar ke quickly nikal sakte hain. Heap sort jaise aap sab patients ko seriousness ke hisaab se sort kar lein. Disjoint Set ADT jaise aap yeh pata karein ke kaun kis se related hai — equivalence relations (reflexive, symmetric, transitive) use karte hain.

KEY TERMS

- 1 **Priority Queue (Heap-based)** — Heap use karke priority queue implement karna — $O(\log N)$ insert/delete.

- 2 **Selection Problem** — N elements mein se kth smallest element find karna — heap use kar ke efficient.
- 3 **Median** — $k = N/2$ — half elements greater, half smaller — heap se find kar sakte hain.
- 4 **Heap Sort** — buildHeap + repeated deleteMin — $O(N \log N)$ sorting algorithm.
- 5 **Disjoint Set ADT** — Elements ko disjoint groups mein partition karna — equivalence relations based.
- 6 **Equivalence Relation** — Reflexive (xRx), Symmetric ($xRy \rightarrow yRx$), Transitive ($xRy \ \& \ yRz \rightarrow xRz$).
- 7 **Blob Coloring** — Computer vision problem — disjoint sets use karte hain blobs identify karne ke liye.
- 8 **Image Segmentation** — Image ko regions mein divide karna — disjoint set ADT application.

DIAGRAM TIME

► PRIORITY QUEUE USING HEAP – CLASS STRUCTURE

```
PriorityQueue
├─ Heap<Event> *heap
├─ insert(Event *e) → heap->insert(e)
├─ remove() → heap->deleteMin(e)
├─ full() → heap->isFull()
└─ length() → heap->getSize()
```

↗ *PriorityQueue internally Heap use karta hai — all operations delegate to heap methods.*

► SELECTION PROBLEM – KTH SMALLEST USING HEAP

Input: $N=1000$ elements, $k=10$

1. `buildHeap(1000 elements)` $\rightarrow O(N)$
2. for $i = 1$ to k :
 `deleteMin()` $\rightarrow$ removes smallest
3. k th `deleteMin` returns 10th smallest

$\nearrow$ `buildHeap` $O(N)$ + k `deleteMin` $O(k \log N)$ — faster than sorting $O(N \log N)$ for small k .

► HEAP SORT — BUILDHEAP + DELETEMIN ALL

Input: Unsorted array

1. `buildHeap(array)` $\rightarrow O(N)$ min heap
2. for $i = 1$ to N :
 `deleteMin()` $\rightarrow$ store in `array[i]`
3. Result: Sorted array (ascending)

$\nearrow$ Heap Sort: $O(N \log N)$ — `buildHeap` $O(N)$ + N `deleteMin` $O(N \log N)$.

► DISJOINT SET — EQUIVALENCE RELATIONS

Set $S = \{\text{Ali, Saad, Ahmad, Ayesha, Fatima}\}$

Relations:

- Ali $\sim$ Saad (related)
- Saad $\sim$ Ahmad (related)
- Ayesha $\sim$ Fatima (related)

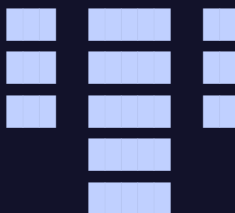
By Transitivity: Ali $\sim$ Ahmad (Ali $\sim$ Saad $\sim$ Ahmad)

Disjoint Sets: {Ali, Saad, Ahmad} and {Ayesha, Fatima}

$\nearrow$ Equivalence relations: reflexive, symmetric, transitive — disjoint sets form equivalence classes.

► BLOB COLORING — DISJOINT SET APPLICATION

Image (5 blobs):



Each blob = one disjoint set
Same label for all pixels in a blob

↗ *Black and white image mein blobs identify karna — each blob ek disjoint set.*

► IMAGE SEGMENTATION – DISJOINT SET

Gray scale image → regions of similar color
Each region = one disjoint set
Used in: MRI, CT Scan, Computer Vision

↗ *Medical imaging mein organs identify karne ke liye disjoint set ADT use hota hai.*

Heap & Disjoint Set Connection: Heap priority queue implement karta hai — used in selection problem (kth smallest), heapsort. Disjoint Set ADT equivalence relations par based hai — reflexive, symmetric, transitive properties. Used in blob coloring, image segmentation, social networks (people related).

CODE EXPLANATION

► PRIORITYQUEUE USING HEAP (IMPLEMENTATION)

```
#include "Event.cpp"
#include "Heap.cpp"
#define PQMAX 30

class PriorityQueue {
public:
    PriorityQueue() {
        heap = new Heap<Event>(PQMAX);
    }
    ~PriorityQueue() {
        delete heap;
    }
    Event* remove() {
        if (!heap->isEmpty()) {
            Event* e;
```

```

        heap->deleteMin(e);
        return e;
    }
    return NULL;
}
int insert(Event* e) {
    if (!heap->isFull()) {
        heap->insert(e);
        return 1;
    }
    return 0;
}
int full() { return heap->isFull(); }
int length() { return heap->getSize(); }
private:
    Heap<Event>* heap;
};

```

- 1 **heap = new Heap<Event>(PQMAX)** — Heap object create (Event type).
- 2 **delete heap** — Destructor — memory cleanup.
- 3 **heap->deleteMin(e)** — Remove minimum event from heap.
- 4 **heap->insert(e)** — Insert event into heap (percolate up).
- 5 **heap->isFull() / isEmpty()** — Delegate to heap methods.

► SELECTION PROBLEM – KTH SMALLEST

```

int findKthSmallest(int arr[], int n, int k) {
    Heap<int> heap(n);
    heap.buildHeap(arr, n);           // O(N)
    int result;
    for (int i = 0; i < k; i++) {
        heap.deleteMin(result);       // O(k log N)
    }
    return result;                    // kth smallest
}

```

- 1 **buildHeap(arr, n)** — O(N) time.

2 for ($i=0$; $i - k$ deleteMin operations.

3 Time Complexity — $O(N + k \log N)$ — faster than $O(N \log N)$ for small k .

► HEAP SORT

```
void heapSort(int arr[], int n) {  
    Heap<int> heap(n);  
    heap.buildHeap(arr, n);           //  $O(N)$   
    for (int i = 0; i < n; i++) {  
        heap.deleteMin(arr[i]);       //  $O(N \log N)$   
    }  
    // arr now sorted in ascending order  
}
```

1 buildHeap(arr, n) — $O(N)$.

2 for ($i=0$; $i - N$ deleteMin operations.

3 Total Time — $O(N \log N)$.

EXAM FOCUS POINTS

MCQ Priority Queue using Heap — insert $O(\log N)$, deleteMin $O(\log N)$, buildHeap $O(N)$.

MCQ Selection Problem — k th smallest using heap: $O(N + k \log N)$.

MCQ Median = $k = N/2$ — find using heap selection.

MCQ Heap Sort — buildHeap + N deleteMin = $O(N \log N)$.

MCQ Disjoint Set ADT — equivalence relations: reflexive, symmetric, transitive.

MCQ Blob Coloring — disjoint sets used to identify connected components.

CONFUSION

Heap Sort vs Selection Problem — heap sort sorts all elements, selection finds kth only.

PRACTICE

Given array, find 5th smallest using heap.

MIND MAP**Priority Queue, Selection, Heap Sort & Disjoint Set****Priority Queue**

- Heap-based implementation
- Insert: $O(\log N)$
- DeleteMin: $O(\log N)$
- getMin: $O(1)$
- Better than array-based

Selection Problem

- kth smallest element
- buildHeap + k deleteMin
- $O(N + k \log N)$
- Median: $k = N/2$

Heap Sort

- buildHeap ($O(N)$)
- N deleteMin ($O(N \log N)$)
- Total: $O(N \log N)$
- In-place possible

Disjoint Set

- Equivalence relations
- Reflexive, Symmetric, Transitive

- Blob coloring
- Image segmentation
- Social networks

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: How does PriorityQueue using heap differ from array-based implementation?

A: Array-based priority queue mein insertion par sorting hoti thi ($O(N \log N)$ or $O(N)$ shift). Heap-based mein insert $O(\log N)$ aur deleteMin $O(\log N)$ hota hai — much faster. Heap internal structure maintain karta hai, caller ko sorting ki tension nahi.

Q: What is the Selection Problem and how can heap solve it?

A: Selection problem: N elements mein se k th smallest (or largest) find karna. Heap solution: buildHeap $O(N)$ min heap banate hain, phir k deleteMin operations karte hain — $O(N + k \log N)$. Sorting solution $O(N \log N)$ se faster hai for small k .

Q: What is Heap Sort and how does it work?

A: Heap sort sorting algorithm hai — buildHeap se min heap banate hain ($O(N)$), phir N deleteMin operations karte hain aur result array mein store karte hain ($O(N \log N)$). Total $O(N \log N)$ — in-place sorting possible hai.

Q: What are equivalence relations and their properties?

A: Equivalence relation over set S has three properties: (1) Reflexive: xRx for all $x \in S$. (2) Symmetric: if xRy then yRx . (3) Transitive: if xRy and yRz then xRz . Example: "is related to" over people is an equivalence relation.

Q: What is Blob Coloring and how does it use Disjoint Set ADT?

A: Blob coloring computer vision problem hai — black and white image mein connected black regions (blobs) identify karna. Disjoint Set ADT use karte hain — adjacent black pixels ko same set mein union karte hain. Each blob ek disjoint set ban jaata hai.

Q: What are some applications of Disjoint Set ADT?

A: Disjoint Set ADT applications: (1) Blob coloring / image segmentation. (2) Social networks — finding connected people. (3) Medical imaging (MRI, CT Scan) — organ identification. (4) Network connectivity — connected components in graphs.

► QUICK QUIZ (10 QUESTIONS)

1. Priority Queue using heap — insert time complexity? ($O(\log N)$)
2. Selection problem: kth smallest using heap — time complexity? ($O(N + k \log N)$)
3. Heap Sort time complexity? ($O(N \log N)$)
4. Equivalence relation is reflexive, symmetric, and ___? (Transitive)
5. Blob coloring uses which ADT? (Disjoint Set)
6. Heap Sort uses which operations? (buildHeap + deleteMin)
7. Median corresponds to $k =$ ___? ($N/2$)
8. Image segmentation uses which ADT? (Disjoint Set)
9. Priority Queue remove operation calls which heap method? (deleteMin)
10. Transitivity: xRy and $yRz \rightarrow$ ___? (xRz)

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Priority Queue (Heap-based):** Heap use karke priority queue — $O(\log N)$ insert/delete.
- **Selection Problem:** N elements mein se k th smallest find — heap solution $O(N + k \log N)$.
- **Median:** $k = N/2$ — half elements greater, half smaller.
- **Heap Sort:** buildHeap + N deleteMin — $O(N \log N)$ sorting.
- **Disjoint Set ADT:** Elements ko disjoint groups mein partition — equivalence relations.
- **Equivalence Relation:** Reflexive, Symmetric, Transitive properties.
- **Blob Coloring:** Computer vision — connected components identification using disjoint set.

IMPORTANT CONCEPTS

- Heap-based priority queue > array-based — faster insert/delete.
- Selection problem: buildHeap + k deleteMin — efficient for small k .
- Heap Sort: $O(N \log N)$ — in-place sorting possible.
- Equivalence relations: reflexive, symmetric, transitive — equivalence classes.
- Disjoint Set used in blob coloring, image segmentation, social networks.
- Medical imaging (MRI, CT Scan) uses disjoint set for organ identification.

△ IMPORTANT FORMULAS

- **Priority Queue insert/delete:** $O(\log N)$
- **Selection Problem:** $O(N + k \log N)$
- **Median:** $k = N/2$
- **Heap Sort:** $O(N \log N)$
- **buildHeap:** $O(N)$

- **Equivalence Relation: Reflexive + Symmetric + Transitive**

FREQUENTLY ASKED TOPICS

- **Priority Queue implementation using heap.**
- **Selection problem — find kth smallest using heap.**
- **Heap Sort algorithm — buildHeap + deleteMin.**
- **Equivalence relations — reflexive, symmetric, transitive.**
- **Blob coloring and image segmentation — disjoint set applications.**
- **Compare heap-based vs array-based priority queue.**

COMMON MISTAKES

- **Thinking selection problem requires sorting all elements — heap is faster.**
- **Confusing heap sort with selection problem — heap sort sorts all.**
- **Forgetting equivalence relation properties — all three required.**
- **Not understanding blob coloring — each blob is a disjoint set.**
- **Thinking priority queue remove is $O(1)$ — it's $O(\log N)$.**

One-Day-Before-Exam Summary: Priority Queue heap-based implement karte hain — insert/delete $O(\log N)$. Selection problem: buildHeap + k deleteMin = $O(N + k \log N)$. Median = $k = N/2$. Heap Sort: buildHeap + N deleteMin = $O(N \log N)$. Disjoint Set ADT equivalence relations par based hai — reflexive, symmetric, transitive. Applications: blob coloring, image segmentation, social networks, medical imaging. Heap array-based priority queue se better hai.

Lesson 34: Disjoint Sets & Equivalence Relations

Dynamic Equivalence — Union/Find Fundamentals

DEEP EXPLANATION — CHAI-TIME ANALOGY

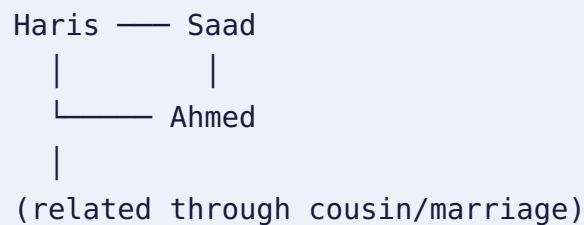
Yaad karo jab ghar mein mehman aate hain aur hum unko rishtedaari ke hisaab se groups mein baithate hain — ek group woh jinka aapas mein rishta hai (bhai, cousin), aur doosre group woh jo kisi aur rishtedaar ke zariye related hain. Agar humein yeh pata karna hai ke do mehman aapas mein related hain ya nahi, toh hum unke group numbers check karte hain. Agar group number same hai toh woh related hain, warna nahi. Isi tarah disjoint sets mein hum elements ko groups (sets) mein divide karte hain, aur union operation do groups ko aapas mein mila deta hai jaise shaadi ke baad do families ek ho jaati hain. Find operation humein batata hai ke koi element kis group mein hai.

KEY TERMS

- 1 **Equivalence Relation** — *Aisa binary relation jo reflexive, symmetric aur transitive ho — jaise family relation ya electrical connectivity.*
- 2 **Disjoint Sets** — *Aise sets jin kaa intersection null ho — ek element sirf ek hi set mein ho sakta hai.*
- 3 **Equivalence Class** — *Equivalence relation ki wajah se jo sets bante hain, unme se har ek set ek equivalence class kehlata hai.*
- 4 **Find Operation** — *find(element) batata hai ke yeh element kis set (equivalence class) mein hai.*
- 5 **Union Operation** — *Do sets ko aapas mein merge karke ek naya set banata hai — $S_k = S_i \cup S_j$.*
- 6 **Dynamic Equivalence Problem** — *Woh problem jahaan humein inputs (relations) milte rehte hain aur humein find aur union operations fast perform karne hote hain.*

DIAGRAM TIME

► EQUIVALENCE RELATION – FAMILY TREE



↗ Ek equivalence class (family) jisme Haris, Saad aur Ahmed sab aapas mein related hain.

► DISJOINT SETS & UNION/FIND

Initial: {1} {2} {3} {4} {5}

Input: 1 R 2 → union(1,2) → {1,2} {3} {4} {5}

Input: 3 R 4 → union(3,4) → {1,2} {3,4} {5}

Input: 5 R 1 → union(5,1) → {1,2,5} {3,4}

Input: 4 R 2 → union(4,2) → {1,2,3,4,5}

↗ Har union operation do sets ko merge karta hai, aur find check karta hai ke do elements ek hi set mein hain ya nahi.

AVL Connection: Is lecture mein trees (AVL) nahi hain, lekin disjoint sets ki representation bhi tree ki form mein ho sakti hai (parent pointers). Union/Find algorithms fast ho jaate hain agar hum tree ki height ko balance rakhein — isi liye "union by size" aur "path compression" jaise techniques use hoti hain.

CODE EXPLANATION

»»» **Is lecture mein actual C++ code nahi diya gaya** — yeh ek conceptual lecture hai jisme disjoint sets, equivalence relations, aur Union/Find operations ki definitions aur mathematical foundation cover ki gayi hai. Agli lectures mein hum in operations ka implementation (array-based parent representation, union by size, path compression) dekhenge.

Agli lecture mein code aayega — typical skeleton:

```
class DisjointSet { public: int find(int x); void  
unionSets(int a, int b); private: int* parent; int* rank; //  
or size };
```

EXAM FOCUS POINTS

MCQ

Equivalence relation ke 3 properties: Reflexive, Symmetric, Transitive — inme se koi ek missing ho toh woh equivalence nahi hai.

FORMULA

$\text{find}(x) = \text{find}(y) \Leftrightarrow x \text{ aur } y \text{ same set mein hain. Union operation: } S_k = S_i \cup S_j.$

CONFUSION

Equivalence relation ka matlab yeh nahi ke elements ki koi "value" compare ho rahi hai — sirf grouping/ relation important hai.

PRACTICE

Diye gaye relations ko use karke disjoint sets banayein aur check karein ke $\text{find}(3)$ aur $\text{find}(5)$ same set return karte hain ya nahi.

MCQ

Electrical connectivity ek equivalence relation hai kyunki yeh reflexive, symmetric aur transitive hai.

CONFUSION

$\leq$ (less-than-or-equal) symmetric nahi hai, is liye equivalence relation nahi hai — reflexive aur transitive hona kaafi nahi.

MIND MAP

Equivalence → Disjoint Sets → Union/Find

Core Definitions

- Equivalence Relation (R)
- Disjoint Sets ($S_i \cap S_j = \emptyset$)
- Equivalence Class

The Problem

- 2D array solution (n^2) memory
- Need fast find/union
- Dynamic relations aate rehte hain

The Solution

- find(a) → set name
- union(a,b) → merge sets
- Tree representation (parent)

Example

- 5 elements: {1}..{5}
- Input: 1R2, 3R4, 5R1, 4R2
- Final: all in one set

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: $\leq$ (less-than-or-equal) equivalence relation kyun nahi hai?

A: $\leq$ reflexive aur transitive to hai, lekin symmetric nahi hai — agar $x \leq y$ hai toh $y \leq x$ zaroori nahi. Equivalence ke liye teeno properties honi chahiye.

Q: Disjoint sets mein ek element do sets mein kyun nahi ho sakta?

A: Kyunki disjoint sets ki definition yeh hai ke unka intersection null hai — ek element sirf ek hi equivalence class mein ho sakta hai, warna equivalence relation violate ho jaye.

Q: find operation ka kya faida hai?

A: find humein batata hai ke koi element kis set mein hai. Is se hum check kar sakte hain ke do elements already related hain ya nahi, aur union karne se pehle duplicate relations avoid kar sakte hain.

Q: Electrical connectivity equivalence relation kyun hai?

A: Kyunki yeh reflexive (component apne aap se connected), symmetric (agar a b se connected toh b a se), aur transitive (a - b aur b - c connected toh a - c connected) — teeno properties satisfy hoti hain.

Q: Agar hum 1000 elements ke liye 2D boolean array use karein toh kya problem hai?

A: $1000 \times 1000 = 10^6$ entries, lekin agar 100,000 ya 1 million elements ho toh array bahut bada ho jata hai (10^{10} ya 10^{12} entries) — memory inefficient. Is liye disjoint sets aur Union/Find better approach hai.

► QUICK QUIZ (10 QUESTIONS)

1. Equivalence relation ki kaunsi property missing hai $\leq$ **(Symmetric)**
mein?
2. find(a) kya return karta hai? **(Set name / class)**
3. Union operation ka output kya hota hai? **(Naya merged set)**
4. Kya family relationship equivalence relation hai? **(Haan)**
5. Disjoint sets ka matlab? **(Intersection null)**
6. Equivalence class kya hoti hai? **(Ek partition set)**
7. Electrical connectivity kis property ki wajah se **(Teeno: R,S,T)**
equivalence hai?
8. Agar find(3) = find(7) hai toh kya matlab? **(Same set mein hain)**
9. Reflexive property ka matlab? **(x R x har x ke liye)**
10. Union/Find algorithm kis problem ko **(Dynamic Equivalence)**
solve karta hai?

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Equivalence Relation:** Binary relation jo reflexive, symmetric aur transitive ho.
- **Disjoint Sets:** Aise sets jin ka intersection empty ho ($S_i \cap S_j = \emptyset$).
- **Equivalence Class:** Partition ka ek set, jis mein sab elements aapas mein related hain.
- **Dynamic Equivalence Problem:** Sequence of find aur union operations ko efficiently handle karna.

IMPORTANT CONCEPTS

- Har element exactly ek equivalence class mein aata hai.
- $\text{find}(x) = \text{find}(y)$ iff x aur y same set mein hain.
- Union do sets ko merge karta hai — n-1 unions tak possible hain n elements ke liye.
- 2D boolean array $O(n^2)$ memory leta hai — inefficient for large n.

△ IMPORTANT FORMULAS

- Equivalence class partition: $S = S_1 \cup S_2 \cup \dots \cup S_k$, $S_i \cap S_j = \emptyset$
- $\text{find}(a)$ returns set identifier
- $\text{union}(a,b)$: $S_k = S_i \cup S_j$

FREQUENTLY ASKED TOPICS

- Equivalence relation ki properties pe MCQ
- Disjoint sets aur equivalence classes ka relation
- Union/Find operations ki definitions aur usage
- 2D array solution vs disjoint sets approach comparison

△ COMMON MISTAKES

- $\leq$ ko equivalence samajh lena (symmetric nahi)
- Union ko commutative samajhna (hota toh hai, lekin implementation important hai)
- find aur union ko ek hi operation samajh lena

- Equivalence class ko set ke equivalent samajh lena (haan, lekin partition zaroori)

One-Day-Before-Exam Summary: Equivalence relation woh hota hai jo reflexive, symmetric aur transitive ho — jaise family relation ya electrical connectivity. Is relation ki wajah se set partitions (disjoint sets) mein divide ho jata hai, har partition ek equivalence class kehlata hai. Disjoint sets data structure do operations support karta hai: find (element ka set batana) aur union (do sets ko merge karna). Dynamic equivalence problem mein humein relations aate rehte hain aur humein fast find aur union karni hoti hai — 2D array inefficient hai, is liye tree-based representation use karte hain.

Lesson 35: Disjoint Sets

Implementation

Parent Array · Union & Find · Uptree Representation

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke ek school mein bachon ko groups mein baitha diya gaya hai, har group ka ek "leader" (root) hai. Agar humein kisi bache ka group pata karna hai, toh hum us se poochte hain "tumhara leader kaun hai?" — yeh find operation hai. Agar do groups ko ek karna hai, toh hum ek group ke leader ko doosre group ke leader ke neechay laga dete hain — yeh union operation hai. Is lecture mein hum yeh sikh rahe hain ke is system ko array ke zariye kaise implement karein, jahan har bache ke paas apne leader (parent) ka pointer hota hai, aur leader khud -1 rakhta hai.

KEY TERMS

- 1 **Forest** — *Uptrees ka collection — har tree ek disjoint set represent karta hai.*
- 2 **Uptree** — *Aisi tree structure jahan pointers parent ki taraf (upward) point karte hain, children ki taraf nahi.*
- 3 **Parent Array** — *Array jisme har index ke liye uske parent ka index stored hai; root ke liye -1.*
- 4 **find(i)** — *i ke root tak traverse karta hai (parent pointers follow karte hue) aur root (set name) return karta hai.*
- 5 **union(i,j)** — *find(i) aur find(j) karke roots nikalta hai, agar alag hain toh ek root ko doosre ka parent bana deta hai.*
- 6 **Running Time** — *union $O(1)$ (constant time) hai; find $O(\text{height})$ hai — worst case $O(n)$ ho sakta hai.*

DIAGRAM TIME

► INITIAL STATE (8 ELEMENTS)

Array: [-1, -1, -1, -1, -1, -1, -1, -1]

Index: 1 2 3 4 5 6 7 8

Forest: {1} {2} {3} {4} {5} {6} {7} {8}

↗ Har element apni root hai (parent = -1), 8 alag sets.

► AFTER UNION(5,6) & UNION(7,8)

Array: [-1, -1, -1, -1, -1, 5, -1, 7]

Index: 1 2 3 4 5 6 7 8

Trees:

```
    5        7
   /        /
  6        8
```

↗ 6 ka parent 5, 8 ka parent 7 — sets {5,6} aur {7,8}.

► AFTER UNION(5,7) & UNION(3,4)

Array: [-1, -1, -1, 3, -1, 5, 5, 7]

Index: 1 2 3 4 5 6 7 8

Trees:

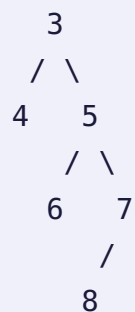
```
    5        3
   / \      /
  6  7    4
   /
  /
```

↗ 7 ka parent 5 ban gaya, 4 ka parent 3.

► FINAL: AFTER UNION(4,5)

Array: [-1, -1, -1, 3, 3, 5, 5, 7]
 Index: 1 2 3 4 5 6 7 8

Final Tree:



Roots: 1, 2, 3 (three sets)

↗ union(4,5) ne pehle 4 ka root (3) find kiya, phir 5 ko 3 ka child bana diya.

AVL Connection: Is lecture mein tree height control nahi ki gayi — agar union hamesha ek root ko doosre ke neechay laga de, toh worst-case height $O(n)$ ho sakti hai (jaise ek lambi chain). Agli lectures mein hum "union by size/rank" aur "path compression" seekhenge jo height ko short rakhte hain.

CODE EXPLANATION

```
// Parent array initialization for (int i = 0; i < n; i++)  
parent[i] = -1; // find operation – returns root (set name)  
int find(int i) { int j = i; while (parent[j] >= 0) j =  
parent[j]; return j; } // union operation – merges two sets  
void unionSets(int i, int j) { int root_i = find(i); int  
root_j = find(j); if (root_i != root_j) parent[root_j] =  
root_i; }
```

- 1 **parent[i] = -1** — Initialization: har element root hai, is liye parent -1 set kiya.
- 2 **while(parent[j] >= 0) j = parent[j];** — find: parent pointer ko follow karte hue root (jis ka parent -1 ho) tak pahunchte hain.
- 3 **root_i = find(i); root_j = find(j);** — union: dono elements ke roots nikalte hain.
- 4 **if (root_i != root_j) parent[root_j] = root_i;** — agar roots alag hain toh doosre root (root_j) ko pehle root (root_i) ka child bana dete hain.

EXAM FOCUS POINTS

MCQ

Parent array mein root ki value -1 hoti hai, aur har non-root index apne parent ka index store karta hai.

FORMULA

find ki running time = $O(\text{height of tree})$. Worst-case height n ho sakti hai (agar union improper ho).

CONFUSION

union operation mein hum sirf roots ko merge karte hain, elements ko nahi — find pehle roots nikalta hai.

PRACTICE

Given parent array: $[-1, -1, 3, -1, 3, 5, 5, 7]$ — find(6) ka result kya hoga? (Answer: 3)

MCQ

Forest mein trees ki total number = number of -1 in parent array.

CONFUSION

Uptree binary tree nahi hota — ek node ke 2 se zyada children ho sakte hain.

MIND MAP

Disjoint Sets — Parent Array Implementation

Core Idea

- Each set = a tree (uptree)
- Root = set name
- Parent array stores pointers

Operations

- $\text{find}(i)$: follow parents to root
- $\text{union}(i,j)$: merge roots
- Initialization: all -1

Array Example

- Index = element
- Value = parent index
- -1 = root

Complexity

- union: $O(1)$ (after find)
- find: $O(\text{height})$
- Worst-case: $O(n)$

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Uptree ko "up" kyun kehte hain?

A: Kyunki isme pointers parent ki taraf (upward) jaate hain, children ki taraf nahi. Is liye hum root tak pahunchne ke liye parent pointers follow karte hain.

Q: find operation mein loop kab rukta hai?

A: Jab $\text{parent}[j] < 0$ ho — yani jab hum root par pahunch jayein. Root ki value -1 hoti hai.

Q: union(4,5) ne 4 aur 5 ko kaise merge kiya?

A: union ne pehle $\text{find}(4) = 3$ aur $\text{find}(5) = 5$ kiya. Phir $\text{parent}[5] = 3$ set kar diya — is tarah 5, 3 ka child ban gaya.

Q: Parent array ka size kya hota hai?

A: Size = number of elements (n). Har index ek element ko represent karta hai, aur us index par parent store hota hai.

Q: Worst-case find kyun $O(n)$ ho sakti hai?

A: Agar union hamesha ek tree ko doosre ke neechay laga de (without balancing), toh ek lambi chain ban sakti hai jiski height n tak pahunch sakti hai — find ko n steps lagenge.

► QUICK QUIZ (10 QUESTIONS)

1. Parent array mein root ki value kya hoti hai? (-1)

2. find(8) kya return karega agar $\text{parent}[8]=7$, $\text{parent}[7]=5$, $\text{parent}[5]=3$, $\text{parent}[3]=-1$? (3)

3. union(i,j) pehle kya karta hai? (find(i) aur find(j))

- | | |
|--|-------------------------|
| 4. Forest kya hai? | (Uptrees ka collection) |
| 5. union ki running time (without find) kya hai? | (O(1)) |
| 6. find ki worst-case complexity? | (O(n)) |
| 7. Agar parent[5]=3 hai toh 5 ka parent kaun? | (3) |
| 8. Initialization mein parent[i] = ? | (-1) |
| 9. Uptree mein pointers kis taraf hain? | (Parent/upward) |
| 10. union(4,5) ne 5 ko kis ka child banaya? | (3 ka) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Parent Array:** 1D array jisme har index ka parent index store hai; root ke liye -1.
- **Uptree:** Tree jisme pointers parent ki taraf hain (children nahi).
- **Forest:** Uptrees ka collection — har tree ek disjoint set.

IMPORTANT CONCEPTS

- find root tak parent pointers follow karta hai.
- union roots ko merge karta hai ($\text{parent}[\text{root}_j] = \text{root}_i$).
- Parent array memory efficient hai ($O(n)$ vs $O(n^2)$ for matrix).
- Number of -1 = number of sets in forest.

△ IMPORTANT FORMULAS

- $\text{find}(i)$ = root of i (jaha parent = -1)
- $\text{union}(i, j)$: if $\text{find}(i) \neq \text{find}(j) \rightarrow \text{parent}[\text{find}(j)] = \text{find}(i)$
- Worst-case find: $O(n)$ (chain)

FREQUENTLY ASKED TOPICS

- Parent array initialization aur updation
- find aur union ka code (pseudocode)
- Array se tree structure ka interpretation
- Running time analysis of find and union

△ COMMON MISTAKES

- union mein find karna bhool jana — direct parent set kar dena (galat)
- Root ki value 0 samajh lena (actually -1 hai)
- find ko $O(1)$ samajh lena (height par depend karta hai)
- Uptree ko binary tree samajh lena (children unlimited)

One-Day-Before-Exam Summary: Is lecture mein humne disjoint sets ko parent array ke zariye implement kiya — har element ka parent store hota hai, root ke liye -1. find operation

parent pointers follow karke root tak jaata hai (jo set ka name hai), aur union do roots ko merge kar deta hai. Yeh array-based approach memory efficient hai ($O(n)$), lekin find ki speed tree ki height par depend karti hai — worst-case $O(n)$ ho sakti hai agar union proper balancing na kare. Agli lecture mein hum union by size/rank aur path compression seekhenge jo height ko control karte hain.

Lesson 36: Disjoint Sets

Optimizations

Union by Size · Path Compression · Near-Linear Performance

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke ek bari family mein sab se bade bhai (leader) ko sab recognise karte hain. Agar humein kisi chhote bache ka leader pata karna hai, toh hum us se poochte hain "tera leader kaun hai?" — lekin agar woh bacha bar-bar poochta hai toh time waste hota hai. Is liye hum ek baar pooch kar us bache ko seedha leader se jod dete hain (path compression) — taake agli baar jawaab jaldi mile. Aur union karte waqt hum chhote group ko bade group mein shaamil kar dete hain (union by size) — taake family tree zyada lamba na ho. Is lecture mein hum yahi dono tricks seekh rahe hain — union by size aur path compression — jo disjoint sets ko bahut fast bana deti hain.

KEY TERMS

- 1 **Union by Size (Weight)** — *Union operation mein chhoti tree ko badi tree ka child bana dena — is se height kam rehti hai.*
- 2 **Size (Weight)** — *Tree mein nodes ki total number; root ke parent mein -size store hota hai (e.g., -5 for 5 nodes).*
- 3 **Union by Height (Rank)** — *Chhoti height wali tree ko badi height wali tree ka child banana — union by size ki alternate technique.*
- 4 **Path Compression** — *find operation ke dauran path ke sab nodes ko seedha root se jod dena — is se future finds fast ho jaate hain.*
- 5 **Inverse Ackermann Function ($\alpha(n)$)** — *Bahut slow growing function; practical purposes mein $\alpha(n) \leq 4$ — is liye union-find practically linear hai.*
- 6 **Amortized Analysis** — *Operations ka average time over sequence — union-find ki amortized complexity almost $O(1)$ hai.*

DIAGRAM TIME

► UNION BY SIZE – STEP-BY-STEP

```
Initial:  [ -1, -1, -1, -1, -1, -1, -1, -1 ]
           1  2  3  4  5  6  7  8
```

```
union(4,6):  [ -1, -1, -1, -2, -1,  4, -1, -1 ]
              1  2  3  4  5  6  7  8
```

Tree: 4 → 6 (size = 2)

```
union(2,3):  [ -1, -2,  2, -2, -1,  4, -1, -1 ]
              1  2  3  4  5  6  7  8
```

Trees: $2 \rightarrow 3$, $4 \rightarrow 6$

```
union(1,4):  [ -3, -2,  2, -3, -1,  4, -1, -1 ]
              1  2  3  4  5  6  7  8
```

(1 joined to 4 because size 4 is larger)

```
union(2,4):  [  4,  4,  2, -5, -1,  4, -1, -1 ]
              1  2  3  4  5  6  7  8
```

(2's tree size 2 joined to 4's tree size 3)

```
union(5,4):  [  4,  4,  2, -6,  4,  4, -1, -1 ]
              1  2  3  4  5  6  7  8
```

(5 joined to 4 – now size = 6)

↗ Root par -size store hai, aur chhoti tree badi tree mein merge ho rahi hai — height kam rehti hai.

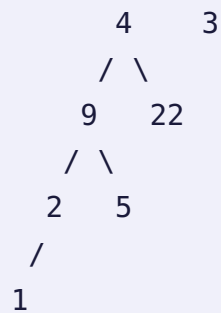
► PATH COMPRESSION – BEFORE & AFTER

```
Before find(1):
```

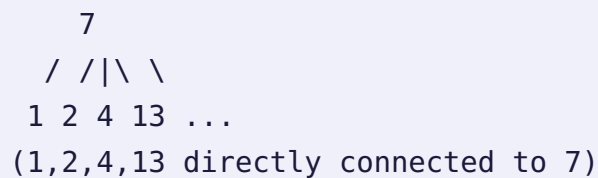
```

      7
     / \
    13  8
   /  \

```



After find(1):



↗ find(1) ne path (1→2→4→13→7) compress kar diya — sab nodes ab seedha root 7 se jude hain.

AVL Connection: Jaisay AVL trees height balance karte hain, yahan union by size height ko log n tak rakhta hai. Path compression extra optimization hai jo find ko aur fast kar deta hai — dono mil kar union-find ko practically constant time banate hain.

CODE EXPLANATION

```
// Union by Size
void unionBySize(int i, int j) {
    int root1 = find(i);
    int root2 = find(j);
    if (root1 == root2) return;
    // parent[root] = -size
    if (parent[root1] <= parent[root2]) {
        // root1 has more nodes (more negative)
        parent[root1] += parent[root2];
        parent[root2] = root1;
    } else {
        parent[root2] += parent[root1];
        parent[root1] = root2;
    }
}

// Path Compression Find (recursive)
int find(int i) {
    if (parent[i] < 0) return i;
    return parent[i] = find(parent[i]);
}
```

- 1 **parent[root1] <= parent[root2]** — Comparison on negative sizes; e.g., $-5 \leq -2$ means root1 tree larger.
- 2 **parent[root1] += parent[root2];** — Add sizes: $-5 + (-2) = -7$ (new total size).
- 3 **parent[root2] = root1;** — Smaller tree ka root ab larger tree ka child.
- 4 **return parent[i] = find(parent[i]);** — Path compression: recursively find root, aur har node ko directly root se jod do.

EXAM FOCUS POINTS

FORMULA

Union by size ki guarantee: depth of any node $\leq \log_2 n$.
(n = total elements)

MCQ

Path compression find recursive call $\text{parent}[i] = \text{find}(\text{parent}[i])$ — yeh root tak recursion karta hai aur path compress karta hai.

CONFUSION

$\text{parent}[\text{root}] = -\text{size}$ — matlab root ki value negative hai, lekin find loop/recursion sirf negative check karta hai, -1 ya -6 dono root indicate karte hain.

PRACTICE

Given parent array: [-5, 4, 4, -6, 4, -1, -1] — $\text{find}(1)$
path compression ke baad $\text{parent}[1]$ kya hoga?
(Answer: 3)

MCQ

Inverse Ackermann function $\alpha(n)$ practically ≤ 4 , is liye union-find amortized complexity $\approx O(1)$ per operation.

CONFUSION

Union by height aur union by size dono similar hain, lekin size wali technique zyada common hai kyunki height maintain karna thoda mushkil hai.

MIND MAP

Union-Find Optimizations

Goal

- Keep tree heights small
- Make find fast
- Avoid worst-case $O(n)$

Union by Size

- Root stores -size
- Smaller tree $\rightarrow$ larger
- Height $\leq \log_2 n$

Path Compression

- Find ke dauran path compress
- Nodes directly to root
- Future finds faster

Complexity

- m operations, n finds
- $O(m \cdot \alpha(n))$
- $\alpha(n) \leq 4$ practically
- $\sim$ Linear time

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Union by size se height kyun kam rehti hai?

A: Kyunki jab bhi kisi node ki depth increase hoti hai, woh node chhoti tree mein thi aur us tree ka size at least double ho jata hai. Is liye depth $\log_2 n$ se zyada nahi ho sakti.

Q: Path compression ka kya faida hai?

A: Path compression find operation ke dauran path ke sab nodes ko seedha root se jod deta hai — is se future find operations bahut fast ho jaate hain kyunki height effectively reduce ho jati hai.

Q: parent array mein root par -size store karne ka kya logic hai?

A: Is se ek hi array mein do kaam ho jaate hain: (1) root identify ho jati hai (negative value), (2) tree ka size bhi pata chal jata hai (absolute value).

Q: Union by height aur union by size mein kya farq hai?

A: Dono similar hain — union by height tree ki height use karta hai, union by size nodes ki count use karta hai. Dono ka result almost same hai, lekin size wali technique implement karna thoda easy hai.

Q: Inverse Ackermann function $\alpha(n)$ kya hai aur kyun important hai?

A: $\alpha(n)$ ek bahut slow growing function hai — practical values mein ≤ 4 . Is ka matlab hai ke union-find with both optimizations practically linear time mein kaam karta hai ($O(m)$ for m operations).

► QUICK QUIZ (10 QUESTIONS)

1. Union by size mein root par kya store hota hai?

(-size)

2. Path compression kaunsi operation mein hoti hai? **(find)**
3. Union by size ki height guarantee kya hai? **($\leq \log_2 n$)**
4. Inverse Ackermann function $\alpha(n)$ ki practical value? **(≤ 4)**
5. Path compression find kaisa hai? **(Recursive)**
6. Union by size mein chhoti tree kahan merge hoti hai? **(Badi tree mein)**
7. $\text{parent}[\text{root}] = -5$ ka kya matlab? **(Size 5, root)**
8. Union by height ka alternative kya hai? **(Union by size)**
9. Union-Find complexity (amortized) kya hai? **($O(\alpha(n))$)**
10. $\text{find}(1)$ path compression ke baad 1 ka parent kya ho ga? **(Root)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Union by Size:** Chhoti tree ko badi tree mein merge karna; root par -size store.
- **Path Compression:** find ke dauran path ke nodes ko seedha root se jodna.
- **Inverse Ackermann Function:** $\alpha(n)$ — bahut slow growing, practically ≤ 4 .

IMPORTANT CONCEPTS

- Union by size height ko $\log_2 n$ tak limit karta hai.
- Path compression future finds ko fast karta hai.
- Dono optimizations mil kar union-find ko practically linear banate hain.
- $\text{parent}[\text{root}] = -\text{size}$ is a clever trick — ek value se root aur size dono pata chal jaate hain.

△ IMPORTANT FORMULAS

- Depth with union by size $\leq \lceil \log_2 n \rceil$
- Amortized complexity: $O(m \cdot \alpha(n))$ where $\alpha(n) \leq 4$
- Path compression recursive: $\text{parent}[i] = \text{find}(\text{parent}[i])$

FREQUENTLY ASKED TOPICS

- Union by size vs union by height comparison
- Path compression code aur us ka effect
- Inverse Ackermann function aur us ki practical significance
- Worst-case analysis vs amortized analysis

△ COMMON MISTAKES

- $\text{parent}[\text{root}] = -\text{size}$ ko -1 samajh lena (size bhi store ho raha hai)
- Path compression ko union operation mein apply karna (yeh find mein hota hai)
- $\alpha(n)$ ko complex function samajh lena (practically constant hai)

- Union by size aur union by height ko ek hi cheez samajh lena (similar hain lekin different metrics)

One-Day-Before-Exam Summary: Is lecture mein humne disjoint sets ko aur optimize kiya — union by size se height $\log_2 n$ tak rehti hai, aur path compression find ko fast kar deta hai. Union by size mein root par -size store hota hai, aur union ke dauran chhoti tree badi tree mein merge hoti hai. Path compression find mein recursive call ke zariye path ke sab nodes ko seedha root se jod deta hai. In dono optimizations ki wajah se union-find practically linear time mein kaam karta hai (amortized $O(\alpha(n))$), aur $\alpha(n) \leq 4$ hai — is liye is data structure ko bahut efficient maana jaata hai.

Lesson 37: Applications of Disjoint Sets

Image Segmentation · Maze Generation · Practical Union-Find

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aap ek bade ghar mein kamron (cells) ke beech deewarein hain. Aapko ek aisa raasta banana hai ke entrance se exit tak jaaya ja sake — maze generation. Har baar jab aap do kamron ke beech ki deewar girate hain, woh kamre aapas mein "related" ho jaate hain (union). Aap tab tak deewarein girate rahte hain jab tak entrance aur exit ek hi "group" (set) mein na aa jaayein. Isi tarah image segmentation mein pixels ko unki intensity ke hisaab se groups mein baantna — yeh dono practical examples hain disjoint sets aur union-find ke.

KEY TERMS

- 1 **Image Segmentation** — *Image ko regions/parts mein divide karna based on pixel intensity, color, ya texture.*
- 2 **Pixel** — *Picture element — image ka smallest unit; grayscale mein 0-255 intensity.*
- 3 **Thresholding** — *Pixels ko ek threshold value ke basis par binary (0/1) mein convert karna.*
- 4 **8-Neighbors** — *Ek pixel ke surrounding 8 pixels (top, bottom, left, right, aur 4 diagonals).*
- 5 **Maze Generation** — *Random walls remove karte hue ek solvable maze create karna, entrance se exit tak path ho.*
- 6 **Equivalence in Maze** — *Do cells equivalent hain agar unke beech walls remove ho chuki hain aur path exist karta hai.*

DIAGRAM TIME

► IMAGE SEGMENTATION — 5×5 PIXEL MATRIX

Original (0=white, 2=gray, 4=black):

```
0 0 0 4 4
2 0 4 4 0
4 2 2 4 4
4 4 0 4 4
0 2 2 4 0
```

After threshold ≥ 4 (binary):

```
0 0 0 1 1
0 0 1 1 0
1 0 0 1 1
1 1 0 1 1
0 0 0 1 0
```

Segmented regions (two disjoint sets of 1s):

Set A: (0,3),(0,4),(1,2),(1,3),(2,3),(2,4),(3,3),(3,4),(4,3)

Set B: (2,0),(3,0),(3,1)

↗ *Union-Find ne connected 1s ko groups mein divide kar diya — image segmentation ho gayi.*

► MAZE GENERATION — 5×5 GRID

Initial: 25 cells, har cell apni set mein (all walls up)

After random wall removals (example):

```
0   1   2   3   4
|   |   |   |   |
5—6—7   8   9
|   |   |   |   |
10—11—12—13—14
```

```
|   |   |   |   |  
15—16 17—18—19  
|   |   |   |   |  
20—21—22—23—24
```

Walls removed: (5-6), (6-7), (10-11), (11-12), (12-13), (13-14), (1

Entrance: 0, Exit: 24 → Now same set? Yes! Path exists.

↗ *Jab entrance (0) aur exit (24) ek hi set mein aa jaate hain, maze complete ho jata hai — solution path guaranteed.*

AVL Connection: Image segmentation aur maze generation dono mein union-find ka efficient implementation zaroori hai — kyunki hum bahut saare pixels/cells par operations karte hain. Union by size aur path compression (previous lecture) is ko fast banate hain.

CODE EXPLANATION

```
// Maze Generation Algorithm void MakeMaze(int size) { int
entrance = 0; int exit = size - 1; while (find(entrance) !=
find(exit)) { int cell1 = randomCell(size); int cell2 =
randomAdjacentCell(cell1, size); if (find(cell1) !=
find(cell2)) { removeWall(cell1, cell2); union(cell1,
cell2); } } } // Image Segmentation (pseudo) for each pixel
(i,j) in image: for each neighbor (ni,nj) of (i,j): if
(abs(pixel[i][j] - pixel[ni][nj]) < threshold) { if
(find(index(i,j)) != find(index(ni,nj))) union(index(i,j),
index(ni,nj)); }
```

- 1 **while (find(entrance) != find(exit))** — Loop tab tak chalega jab tak entrance aur exit same set mein na aa jayein.
- 2 **cell1 = randomCell(); cell2 = randomAdjacentCell();** — Randomly ek cell aur us ka koi neighbor choose karo.
- 3 **if (find(cell1) != find(cell2))** — Agar dono alag sets mein hain toh wall remove karo aur union karo.
- 4 **removeWall(cell1, cell2); union(cell1, cell2);** — Wall hatane ka matlab hai ke ab in cells ke beech path hai — union unhe ek set mein laata hai.

EXAM FOCUS POINTS

MCQ

Image segmentation mein pixels ko groups mein divide karne ke liye union-find use hota hai — similarity threshold ke basis par.

CONFUSION

Maze generation mein union tab tak karte hain jab tak entrance aur exit same set mein aa jayein — is ke baad maze complete hai, chahe aur walls bachi hon.

PRACTICE

5×5 maze mein kitne cells hain? (25). Entrance 0, exit 24. Union-Find loop kab rukega? (Jab $\text{find}(0) == \text{find}(24)$).

MCQ

Thresholding ke baad binary image mein 0 aur 1 ki regions find karne ke liye union-find best suited hai kyunki yeh connected components detect karta hai.

FORMULA

Maze generation complexity: $O(m \cdot \alpha(n))$ where m = number of wall removals until entrance & exit connected.

CONFUSION

Maze mein "remove wall" ka matlab union hai — lekin agar cells already same set mein hain toh wall remove nahi karte (warna cycle ban jayegi).

MIND MAP

Applications of Disjoint Sets

Image Segmentation

- Pixels = elements
- Similar intensity → union
- Threshold based grouping
- Connected components

Maze Generation

- Cells = elements
- Remove walls = union
- Entrance & exit same set
- Randomized algorithm

How It Works

- Initial: all separate sets
- Union merges connected
- Find checks connectivity
- Stop when goal reached

Benefits

- Efficient (near-linear)
- No extra memory
- Simple to implement
- Works for large grids

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Image segmentation mein union-find kaise use hota hai?

A: Har pixel ko ek element maan kar, agar do neighboring pixels ki intensity difference threshold se kam hai toh un par union operation apply karte hain — is se connected regions (segments) ban jaate hain.

Q: Maze generation algorithm kab rukta hai?

A: Jab entrance cell (0) aur exit cell (size-1) ek hi set mein aa jaayein — is ka matlab hai ke un ke beech path exist karta hai.

Q: Maze generation mein agar do cells already same set mein hain toh wall kyun nahi remove karte?

A: Agar already same set mein hain, toh un ke beech pehle se path exist karta hai — wall remove karne se cycle ban jayegi aur maze ki structure kharab ho jayegi.

Q: Thresholding kya hai aur is ka kya faida hai?

A: Thresholding ek specific value (threshold) ke basis par pixels ko binary (0/1) mein convert karta hai — is se image simple ho jati hai aur segmentation easy ho jati hai.

Q: Maze generation ko random kyun kehte hain?

A: Kyunki har baar algorithm randomly cells aur walls choose karta hai — is liye har baar different maze generate hota hai.

► QUICK QUIZ (10 QUESTIONS)

1. Image segmentation mein kya represent karte hain elements? **(Pixels)**

2. Maze generation mein entrance aur exit kab same set mein honge? **(Path exists)**

3. Maze generation mein wall removal ka matlab kya hai? **(Union)**
4. 5×5 image mein kitne pixels hain? **(25)**
5. Thresholding ke baad image kaisi ho jati hai? **(Binary)**
6. Maze generation algorithm kis **(find(entrance)==find(exit))** condition par stop hota hai?
7. Pixel ke kitne neighbors hote hain (8-neighbor)? **(8)**
8. Image segmentation mein similarity kaise **(Intensity difference)** check karte hain?
9. Maze mein cells ka equivalence kya define **(Path between them)** karta hai?
10. Union-Find maze generation ke liye kyun **(Fast connectivity)** useful hai?

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Image Segmentation:** Image ko meaningful regions mein divide karna.
- **Thresholding:** Pixels ko threshold value ke basis par binary mein convert karna.
- **Maze Generation:** Random walls remove karte hue entrance se exit tak path banana.

IMPORTANT CONCEPTS

- Image segmentation mein pixels ko union-find se group karte hain based on similarity.
- Maze generation mein union tab tak karte hain jab tak entrance aur exit connected na ho jaayein.
- Dono applications mein union-find ki efficiency (near-linear) ka faida uthaya jata hai.
- Randomness maze generation mein har baar naya maze banane ke liye zaroori hai.

△ IMPORTANT FORMULAS

- Number of pixels in $n \times n$ image = n^2
- Maze generation complexity = $O(m \cdot \alpha(n))$ where m = wall removals
- Thresholding: if pixel $\geq T \rightarrow 1$, else 0

FREQUENTLY ASKED TOPICS

- Image segmentation ka process (thresholding + union-find)
- Maze generation algorithm steps
- Why union-find is suitable for these applications
- Role of randomness in maze generation

△ COMMON MISTAKES

- Maze generation mein loop condition galat samajhna
(find(entrance) != find(exit))

- Image segmentation mein sirf 4-neighbors use karna (8-neighbors better connected components dete hain)
- Threshold value ko random samajh lena (yeh domain-specific hoti hai)
- Maze mein already same set cells ka wall remove karna (cycle ban jayegi)

One-Day-Before-Exam Summary: Is lecture mein humne disjoint sets ke do practical applications dekhe — image segmentation aur maze generation. Image segmentation mein hum pixels ko unki intensity ke hisaab se groups (segments) mein divide karte hain, aur union-find connected components detect karne mein help karta hai. Maze generation mein hum randomly walls remove karte hain (union) jab tak entrance aur exit ek hi set mein na aa jaayein — is se har baar ek naya solvable maze ban jata hai. Dono applications union-find ki efficiency (near-linear) ka faida uthate hain, aur yeh show karte hain ke disjoint sets sirf theory nahi, real-world problems mein bhi use hote hain.

Lesson 38: Table ADT & Binary Search

Dictionaries · Key-Value Storage · Sorted vs Unsorted Arrays

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aapke paas ek mobile phone directory hai jisme friends ke naam, phone numbers aur addresses hain. Har friend ka record ek "row" hai, aur naam, phone, address "columns" (fields) hain. Jab aap kisi friend ka number dhoondna chahte hain, toh aap naam (key) use karte hain — yeh find operation hai. Agar aap koi naya friend add karte hain, toh woh insert hai. Agar aap phone book ko alphabetically sort rakhein toh search fast ho jati hai (binary search), lekin insert mushkil ho jati hai kyunki order maintain karna padta hai. Is lecture mein hum table (dictionary) ADT ke yehi operations aur unki implementations dekhenge.

KEY TERMS

- 1 **Table (ADT)** — Rows (records) aur columns (fields) ka collection — har row ek entry.
- 2 **Record / Tuple** — Table ki ek row — fields ka set jo ek entity ko represent karta hai.
- 3 **Key / Primary Key** — Ek unique field jo record ko uniquely identify karta hai (e.g., employee ID).
- 4 **Dictionary (ADT)** — Key-value pairs ka collection — table ki abstraction.
- 5 **Unsorted Sequential Array** — Records ko array mein bina kisi order ke store karna — insert fast, find slow.
- 6 **Sorted Sequential Array** — Records ko sorted order mein store karna (e.g., alphabetically) — find fast (binary search), insert slow.
- 7 **Binary Search** — Sorted array mein $O(\log n)$ time mein search karne ka algorithm — har step mein search space half ho jata hai.

DIAGRAM TIME

► TABLE STRUCTURE – RECORDS & FIELDS

+-----+-----+-----+			
Name	Address	Phone	← Fields (Columns)
+-----+-----+-----+			
Sohail	50 Zahoor Elahi Rd	567-4567	← Record 1
Imran	30-T Phase-IV	517-2349	← Record 2
Salman	131-D Model Town	756-5678	← Record 3
+-----+-----+-----+			

Key = Name (unique)

↗ Har row ek record hai, har column ek field. Key unique identifier hai (e.g., Name).

► UNSORTED VS SORTED ARRAY

Unsorted Array (insert $O(1)$, find $O(n)$):

[Salman | Imran | Sohail | ...] ← new record at end

Sorted Array (find $O(\log n)$, insert $O(n)$):

[Imran | Salman | Sohail | ...] ← sorted alphabetically

Binary Search on Sorted Array:

Step 1: Check middle → "Salman"

Step 2: If key < middle → search left half

Step 3: Else → search right half

↗ Unsorted array mein insert fast, find slow. Sorted array mein find fast (binary search), insert slow (shifting).

AVL Connection: Binary search tree (AVL) sorted array ki tarah $O(\log n)$ search deta hai, lekin insert/delete bhi $O(\log n)$ mein ho jate hain — array mein insert/delete $O(n)$ hota hai kyunki shifting zaroori hai. Is liye dynamic data ke liye BST/AVL better hai.

CODE EXPLANATION

```
// Binary Search on Sorted Array
int binarySearch(TableNode arr[], int n, KeyType key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid].key == key) return mid;
        else if (arr[mid].key < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // not found
}

// Insert in Sorted Array ( $O(n)$ )
void insertSorted(TableNode arr[], int &n, TableNode newNode) {
    int pos = n;
    while (pos > 0 && arr[pos-1].key > newNode.key) {
        arr[pos] = arr[pos-1]; // shift right
        pos--;
    }
    arr[pos] = newNode;
    n++;
}
```

- 1 **`mid = (low + high) / 2`** — Binary search ka middle element — search space half kar deta hai.
- 2 **`if (arr[mid].key == key) return mid;`** — Agar key mil gayi toh index return karo.
- 3 **`arr[mid].key < key → low = mid + 1`** — Key middle se badi hai toh right half mein search karo.
- 4 **`while (pos > 0 && arr[pos-1].key > newNode.key)`** — Sorted insert: shift karte hain jab tak correct position nahi mil jati.

EXAM FOCUS POINTS

FORMULA

Binary Search Complexity: $O(\log n)$. Unsorted array find: $O(n)$. Sorted array insert: $O(n)$.

MCQ

Table ADT ke operations: insert, find, remove. Key unique hona chahiye (primary key).

CONFUSION

Sorted array mein binary search $O(\log n)$ hai, lekin insert/delete $O(n)$ hain — shifting ki wajah se.

PRACTICE

Array: [2, 5, 8, 12, 19, 24, 31] mein binary search for 19 — steps? (mid=12 → right, mid=24 → left, mid=19 → found).

MCQ

Telephone directory mein "name" field key hai — is se unique record milta hai.

CONFUSION

Table mein fields ki data type different ho sakti hai (e.g., string, int) — array of objects use karte hain.

MIND MAP

Table ADT (Dictionary)

Core Concepts

- Records (rows) + Fields (cols)
- Key uniquely identifies record
- Also called Dictionary

Operations

- insert(key, entry)
- find(key) → entry
- remove(key)

Unsorted Array

- Insert: $O(1)$ (at end)
- Find: $O(n)$ (linear)
- Remove: $O(n)$

Sorted Array

- Insert: $O(n)$ (shifting)
- Find: $O(\log n)$ (binary)
- Remove: $O(n)$

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Table aur Dictionary mein kya farq hai?

A: Table ek collection hai rows aur columns ka (like database table), jabke Dictionary key-value pairs ka abstract data type hai. Table ko dictionary ki tarah use kar sakte hain jahan key unique field ho.

Q: Unsorted array mein insert fast kyun hai?

A: Kyunki hum naya record array ke end (last position) par daal dete hain — kisi shifting ki zaroorat nahi hoti. Is liye $O(1)$ time lagta hai.

Q: Sorted array mein find fast kyun hai?

A: Kyunki sorted array par binary search apply ho sakta hai — har step mein search space half ho jata hai, is liye $O(\log n)$ time mein find ho jata hai.

Q: Binary search kyun kaam karta hai?

A: Kyunki array sorted hai — agar middle element key se chhota hai toh key right half mein hogi, agar bada hai toh left half mein. Is property ki wajah se hum half elements ignore kar sakte hain.

Q: Sorted array mein remove operation $O(n)$ kyun hai?

A: Remove ke baad array ko sorted rakhne ke liye elements ko left shift karna padta hai — yeh $O(n)$ time leta hai.

► QUICK QUIZ (10 QUESTIONS)

1. Table ki row ko kya kehte hain? **(Record / Tuple)**
2. Table mein unique identifier ka kya naam hai? **(Key / Primary Key)**
3. Unsorted array mein find ki complexity? **($O(n)$)**

4. Sorted array mein find ki complexity? **($O(\log n)$)**
5. Binary search kis condition par kaam karta hai? **(Sorted array)**
6. Sorted array mein insert ki complexity? **($O(n)$)**
7. Binary search har step mein search space kya karta hai? **(Half)**
8. Table ADT ko aur kya kehte hain? **(Dictionary)**
9. Unsorted array mein insert ki complexity? **($O(1)$)**
10. Binary search array ke kis hissay mein search karta hai? **(Middle se start)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Table (ADT):** Records (rows) aur fields (columns) ka collection — har record ek entry.
- **Key:** Unique field jo record ko identify karta hai (e.g., ID, name).
- **Dictionary:** Key-value pairs ka ADT — table is a dictionary.

IMPORTANT CONCEPTS

- Table operations: insert, find, remove — sab key based.
- Unsorted array: insert $O(1)$, find $O(n)$.
- Sorted array: find $O(\log n)$ via binary search, insert $O(n)$.
- Binary search works only on sorted data — $O(\log n)$ time.

△ IMPORTANT FORMULAS

- Binary Search Complexity: $O(\log n)$
- Unsorted Array Find: $O(n)$
- Sorted Array Insert/Delete: $O(n)$

FREQUENTLY ASKED TOPICS

- Table ADT operations aur unki complexities
- Unsorted vs Sorted array comparison
- Binary search algorithm aur us ki working
- Key aur record ka concept

△ COMMON MISTAKES

- Unsorted array ki find complexity $O(1)$ samajh lena (actually $O(n)$ hai)
- Sorted array mein insert $O(\log n)$ samajh lena (shifting ki wajah se $O(n)$ hai)
- Binary search ko unsorted array par apply karna (kaam nahi karega)
- Key ko unique hona zaroori nahi samajh lena (primary key unique hoti hai)

One-Day-Before-Exam Summary: Table (Dictionary) ADT records (rows) aur fields (columns) ka collection hai — har record ko ek unique key identify karta hai. Is par teen operations hote hain: insert, find, aur remove. Unsorted sequential array mein insert fast ($O(1)$) hai lekin find slow ($O(n)$) — kyunki poori array scan karni padti hai. Sorted sequential array mein find fast ($O(\log n)$) hai via binary search — lekin insert slow ($O(n)$) kyunki order maintain karne ke liye shifting karni padti hai. Binary search har step mein search space half kar deta hai, is liye bahut efficient hai.

Lesson 39: Binary Search & Skip Lists

Efficient Search · Linked List Implementations · Introduction to Skip Lists

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aapke paas ek almari hai jisme files alphabetically sorted hain. Agar aapko "S" se shuru hone wali file chahiye, toh aap seedha almari ke beech mein dekhenge — agar wahan "M" hai toh aap jaan gaye ke "S" right side mein hai. Yeh hai binary search — har step mein aap aadhi files ignore kar dete hain. Ab agar yeh files ek zanjeer (linked list) mein hoti, toh aap ko beech mein pahunchne ke liye poori list traverse karni padti. Skip list isi problem ka solution hai — yeh extra "express lanes" (higher level chains) add karta hai taake aap list mein bhi fast jump kar saken, jaise metro mein express train local stations skip karti hai.

KEY TERMS

- 1 **Binary Search** — *Sorted array mein $O(\log n)$ time mein search karne ka algorithm — har step mein search space half.*
- 2 **Bisection** — *Array ko do halves mein divide karna — binary search ka core step.*
- 3 **Linked List Table Implementation** — *Records ko linked list mein store karna — insert $O(1)$, find $O(n)$ — binary search possible nahi.*
- 4 **Skip List** — *Linked list ki extended version with multiple levels — fast search ($O(\log n)$) aur insert/delete bhi efficient.*
- 5 **Levels / Chains** — *Skip list mein multiple linked lists — Level 0 (all elements), Level 1 (every other), Level 2 (every 4th), etc.*
- 6 **Randomization in Skip Lists** — *Bill Pugh's idea: levels randomly assign karo — is se balancing easy ho jati hai.*

DIAGRAM TIME

► BINARY SEARCH – STEP-BY-STEP EXAMPLE

Array: [1, 5, 7, 9, 10, 13, 17, 19, 27]

Search for: 19

Step 1: low=0, high=8, mid=4 → arr[4]=10
10 < 19 → search right half

Step 2: low=5, high=8, mid=6 → arr[6]=17
17 < 19 → search right half

Step 3: low=7, high=8, mid=7 → arr[7]=19
19 == 19 → Found!

Total comparisons: 3 (vs 8 in linear search)

↗ Binary search ne sirf 3 comparisons mein 19 find kar liya — linear search mein 7 comparisons lagte.

► SKIP LIST – MULTIPLE LEVELS

```
Level 2:  head -----> tail
           |
Level 1:  head → 20 -----> 60 → 80 → tail
           |       ↓               ↓
Level 0:  head → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → tail
```

Search for 70:

1. Level 2: head → tail (skip) (70 > tail? no)
2. Level 1: head → 20 → 60 → 80 (70 < 80, so go down)
3. Level 0: from 60 → 70 → found!

Express lanes (higher levels) let you jump over many nodes.

↗ *Skip list mein higher levels "express lanes" ki tarah kaam karte hain — search fast ho jati hai.*

AVL Connection: Skip list AVL tree ka alternative hai — dono $O(\log n)$ search dete hain. Lekin skip list mein balancing random hoti hai (AVL mein deterministic rotations), is liye skip list implement karna easier hai.

CODE EXPLANATION

```
// Binary Search – C++ Implementation
int isPresent(int *arr, int val, int N) {
    int low = 0; int high = N - 1; int mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == val) return 1; // found
        else if (arr[mid] < val) low = mid + 1; // search right half
        else high = mid - 1; // search left half
    }
    return 0; // not found
}
```

- 1 **low = 0; high = N-1;** — Search space ki boundaries initialize.
- 2 **mid = (low + high) / 2;** — Middle index calculate karo — search space ka center.
- 3 **if (arr[mid] == val) return 1;** — Value mil gayi — function return karo.
- 4 **else if (arr[mid] < val) low = mid + 1;** — Value middle se badi hai — right half mein search.
- 5 **else high = mid - 1;** — Value middle se chhoti hai — left half mein search.

EXAM FOCUS POINTS

FORMULA

Binary search complexity: $O(\log n)$. After i bisections: $N/2^i = 1 \rightarrow i = \log_2 N$.

MCQ

Binary search sirf sorted arrays par kaam karta hai — linked list par nahi kyunki random access possible nahi.

CONFUSION

Sorted array mein insert $O(n)$ hai kyunki shifting karni padti hai — binary search sirf find ko fast karta hai, insert ko nahi.

PRACTICE

Array: [2, 5, 8, 12, 19, 24, 31] mein binary search for 24 — steps? (mid=12 → right, mid=24 → found).

MCQ

Skip list kisne propose ki? (Bill Pugh, 1990). Isme randomization use hoti hai balancing ke liye.

CONFUSION

Skip list mein Level 0 chain all elements contain karti hai — higher levels subsets hain.

MIND MAP

Search Implementations for Table ADT

Binary Search

- Sorted array required
- $O(\log n)$ time
- Har step half data
- Random access needed

Linked List

- Unsorted: insert $O(1)$
- Find $O(n)$ — slow
- Sorted bhi $O(n)$ find
- Binary search not possible

Skip List

- Multiple levels/chains
- Level 0 = all elements
- Higher = express lanes
- Search $O(\log n)$ avg

Randomization

- Bill Pugh (1990)
- Random levels assign
- Easy balancing
- Alternative to AVL

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Binary search linked list par kyun kaam nahi karta?

A: Kyunki binary search ko middle element tak pahunchne ke liye random access chahiye ($arr[mid]$) — linked list mein random access possible nahi, humein traverse karna padta hai.

Q: Skip list ka main idea kya hai?

A: Multiple levels ki linked lists banate hain — Level 0 mein sab elements, Level 1 mein every other, Level 2 mein every 4th, etc. — is se search fast ho jati hai kyunki hum higher levels se jump kar sakte hain.

Q: Skip list mein randomization kyun use karte hain?

A: Agar levels fixed (powers of 2) hon toh insert/delete complex ho jate hain — random levels assign karne se balancing easy ho jati hai aur expected $O(\log n)$ time milti hai.

Q: Binary search ke liye array sorted hona zaroori kyun hai?

A: Kyunki hum middle element ko compare karte hain aur decide karte hain ke value left half mein hai ya right half — yeh sirf tab possible hai jab array sorted ho.

Q: Skip list AVL tree se kis tarah better hai?

A: Skip list implement karna easier hai — rotations ki zaroorat nahi, randomization se balance ho jata hai. AVL mein complex rotations karni padti hain.

► QUICK QUIZ (10 QUESTIONS)

1. Binary search ki complexity kya hai? **($O(\log n)$)**

2. Binary search kis type ke array par kaam karta hai? **(Sorted)**

3. Linked list mein binary search possible hai? **(Nahi)**
4. Skip list kisne propose ki? **(Bill Pugh)**
5. Skip list mein Level 0 kya contain karti hai? **(All elements)**
6. Skip list mein balancing kaise hoti hai? **(Randomization)**
7. Binary search har step mein search space kya karta hai? **(Half)**
8. Skip list ki average search complexity? **($O(\log n)$)**
9. Sorted array mein insert ki complexity? **($O(n)$)**
10. Skip list ko tree alternative **(Fast search + easy balancing)**
kyun kehte hain?

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Binary Search:** Sorted array mein $O(\log n)$ search — har step half.
- **Skip List:** Multi-level linked list with express lanes — $O(\log n)$ expected search.
- **Level 0:** All elements; Level i contains every 2^i -th element (or random).

IMPORTANT CONCEPTS

- Binary search requires random access (array) — linked list mein nahi ho sakta.
- Sorted array: find $O(\log n)$, insert/delete $O(n)$.
- Skip list: linked list ki limitation overcome karti hai — fast search + easy updates.
- Randomization skip list ko balanced rakhti hai without complex rotations.

△ IMPORTANT FORMULAS

- Binary search: after i bisections, $N/2^i = 1 \rightarrow i = \log_2 N$
- Skip list expected search: $O(\log n)$
- Linked list find: $O(n)$

FREQUENTLY ASKED TOPICS

- Binary search algorithm aur us ki working
- Why binary search doesn't work on linked list
- Skip list structure aur levels
- Randomization in skip lists

△ COMMON MISTAKES

- Binary search ko unsorted array par apply karna (kaam nahi karega)
- Skip list mein levels ko fixed powers of 2 samajh lena (random hain)
- Linked list mein binary search possible samajh lena (nahi hai)
- Skip list ko sirf linked list samajh lena (multiple levels hain)

One-Day-Before-Exam Summary: Is lecture mein humne table ADT ke liye search operations dekhe. Binary search sorted array par $O(\log n)$ time mein kaam karta hai — har step mein search space half ho jata hai. Linked list mein binary search possible nahi kyunki random access nahi hai — is liye find $O(n)$ hai. Skip list is problem ka solution hai — yeh multiple levels ki linked lists use karta hai (express lanes) jisse search $O(\log n)$ ho jati hai. Bill Pugh ne 1990 mein skip list propose ki — isme randomization balancing ke liye use hoti hai, is liye yeh AVL tree ka easy alternative hai.

Lesson 40: Skip List Operations

Search · Insert · Delete — Randomized Data Structure

DEEP EXPLANATION — CHAI-TIME ANALOGY

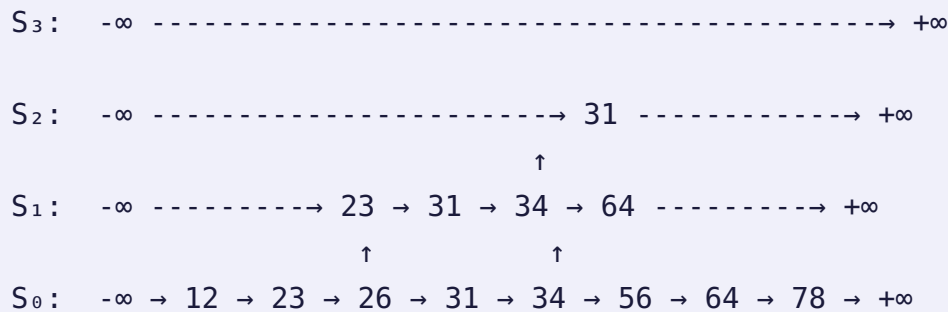
Socho ke aap Lahore se Islamabad ja rahe hain. Agar aap har city mein ruk kar jayein (local train), toh bahut time lagega. Lekin agar aap express train use karein jo sirf bade shehron mein rukti hai, toh aap tez pahunch jayenge. Skip list mein bhi aisa hi hai — Level 0 (local) har node par rukti hai, Level 1 (express) kuch nodes skip karti hai, Level 2 (super express) aur zyada skip karti hai. Search karte waqt hum top level se start karte hain aur "express lane" use karte hain — agar target aage hai toh scan forward, agar peeche hai toh drop down. Insertion mein coin toss decide karta hai ke naya node kitni "express" lanes mein shamil hoga — is liye skip list balanced rehti hai bina rotations ke.

KEY TERMS

- 1 **Skip List** — Series of sorted linked lists $S_0, S_1, \dots, S_h$ — har list apni neechay wali list ka subset.
- 2 **Special Keys** — $+\infty$ aur $-\infty$ — har list mein boundaries ke tor par hote hain.
- 3 **Scan Forward** — Current level par aage move karna jab target key next node se badi ho.
- 4 **Drop Down** — Neechay wali level par jana jab target key next node se chhoti ho.
- 5 **Randomized Algorithm** — Coin toss (random bits) use karta hai execution control ke liye — insertion mein level decide karta hai.
- 6 **Insertion Level (i)** — Heads count until first tail — node is level 0 to level i mein insert hota hai.
- 7 **h (height)** — Skip list ki current maximum level — agar $i > h$ toh new levels add karte hain.

DIAGRAM TIME

► SKIP LIST STRUCTURE – LEVELS & LINKS



Each node has "next" and "down" pointers.

Level 0 = all elements, Level 1 = subset, Level 2 = subset, etc.

↗ Skip list ek hierarchy of linked lists hai — higher levels "express lanes" ki tarah kaam karte hain.

► SEARCH FOR 78 – STEP-BY-STEP

Start at S_3 ($-\infty$):

Step 1: next = $+\infty$, $78 < +\infty \rightarrow$ DROP DOWN to S_2

S_2 ($-\infty \rightarrow 31 \rightarrow +\infty$):

Step 2: next = 31, $78 > 31 \rightarrow$ SCAN FORWARD to 31

Step 3: next = $+\infty$, $78 < +\infty \rightarrow$ DROP DOWN to S_1

S_1 ($-\infty \rightarrow 23 \rightarrow 31 \rightarrow 34 \rightarrow 64 \rightarrow +\infty$):

Step 4: at 34, next = 64, $78 > 64 \rightarrow$ SCAN FORWARD to 64

Step 5: next = $+\infty$, $78 < +\infty \rightarrow$ DROP DOWN to S_0

S_0 ($-\infty \rightarrow 12 \rightarrow 23 \rightarrow 26 \rightarrow 31 \rightarrow 34 \rightarrow 56 \rightarrow 64 \rightarrow 78 \rightarrow +\infty$):

Step 6: at 64, next = 78 $\rightarrow 78 == 78 \rightarrow$ FOUND!

Total steps: 6 (vs 8 nodes in S_0)

↗ Search "drop down" aur "scan forward" ki movement follow karti hai — binary search jaisi.

► INSERTION – COIN TOSS ($i=2$)

Before Insert (15):

S_2 : $-\infty \rightarrow +\infty$

S_1 : $-\infty \rightarrow 23 \rightarrow +\infty$

S_0 : $-\infty \rightarrow 10 \rightarrow 23 \rightarrow 36 \rightarrow +\infty$

Insert 15 with $i=2$ (heads count = 2):

Step 1: Find positions p_0, p_1, p_2

Step 2: Insert 15 in S_0 (after 10), S_1 (after $-\infty$), S_2 (after $-\infty$)

After Insert:

S_3 : $-\infty \rightarrow +\infty$ (new top level, since $i = h = 2$)

S_2 : $-\infty \rightarrow 15 \rightarrow +\infty$

S_1 : $-\infty \rightarrow 15 \rightarrow 23 \rightarrow +\infty$

S_0 : $-\infty \rightarrow 10 \rightarrow 15 \rightarrow 23 \rightarrow 36 \rightarrow +\infty$

15 is now in levels 0, 1, and 2.

↗ Coin toss ne $i=2$ diya — is liye 15 ko 3 levels (0,1,2) mein insert kiya.

► DELETION – REMOVE 34

Before Deletion:

S_3 : $-\infty \rightarrow +\infty$

S_2 : $-\infty \rightarrow 34 \rightarrow +\infty$

S_1 : $-\infty \rightarrow 23 \rightarrow 34 \rightarrow +\infty$

S_0 : $-\infty \rightarrow 12 \rightarrow 23 \rightarrow 34 \rightarrow 45 \rightarrow +\infty$

Remove 34:

Step 1: Find 34 in all levels (S_2 , S_1 , S_0)

Step 2: Remove from S_2 , S_1 , S_0

Step 3: Update pointers to bypass 34

After Deletion:

S_3 : $-\infty \rightarrow +\infty$

S_2 : $-\infty \rightarrow +\infty$ (S_2 becomes empty except boundaries)

S_1 : $-\infty \rightarrow 23 \rightarrow +\infty$

S_0 : $-\infty \rightarrow 12 \rightarrow 23 \rightarrow 45 \rightarrow +\infty$

↗ *Delete operation find karta hai aur har level se node remove karta hai — pointers adjust kar deta hai.*

AVL Connection: Skip list AVL tree ka randomized alternative hai — dono $O(\log n)$ expected search dete hain. AVL mein rotations complex hain, skip list mein coin toss simple hai — is liye skip list implement karna easier aur faster ho sakta hai.

CODE EXPLANATION

```
// Skip List Insertion (Pseudo) void insert(Key x, Value o)
{ int i = 0; while (random() < 0.5) // coin toss: heads i++;
if (i > h) { // add new levels from h+1 to i+1 for (int j =
h+1; j <= i+1; j++) addNewLevel(); h = i + 1; } // find
positions p0, p1, ..., pi Node* p = head; for (int level = h;
level >= 0; level--) { while (p->next[level]->key < x) p = p-
>next[level]; if (level <= i) positions[level] = p; } //
insert x at each level 0..i for (int level = 0; level <= i;
level++) { Node* newNode = createNode(x, o, level); newNode-
>next[level] = positions[level]->next[level];
positions[level]->next[level] = newNode; } }
```

- 1 **while (random() < 0.5) i++;** — Coin toss: jab tak head (0.5 se kam) aata hai, i increment karo — tail par ruko.
- 2 **if (i > h) add new levels** — Agar i current height se zyada hai toh new top levels add karo.
- 3 **while (p->next[level]->key < x) p = p->next[level];** — Har level par scan forward karo jab tak key x se chhoti hai.
- 4 **positions[level] = p;** — Insertion position store karo har level ke liye (p ke baad insert karna hai).
- 5 **newNode->next[level] = positions[level]->next[level];** — Naya node insert karo — pointers adjust karo.

EXAM FOCUS POINTS

FORMULA

Skip list expected search/insert/delete: $O(\log n)$. Height expected: $O(\log n)$.

MCQ

Skip list insertion randomized hai — coin toss decide karta hai ke node kitni levels mein insert hoga.

CONFUSION

Skip list mein "randomized" ka matlab yeh nahi ke search random hai — search deterministic hai, sirf insertion random hai.

PRACTICE

Coin toss se $i=3$ aaya. Node kitni levels mein insert hoga? (Levels 0,1,2,3 — total 4 levels).

MCQ

Skip list search mein "drop down" tab hota hai jab target key next node se chhoti ho.

CONFUSION

Delete mein randomness nahi hai — sirf find + pointer adjustment.

MIND MAP

Skip List Operations

Search

- Start at top-left
- Compare with next node
- If equal $\rightarrow$ found
- If $>$ $\rightarrow$ scan forward
- If $<$ $\rightarrow$ drop down

Insert

- Coin toss $\rightarrow i$
- If $i > h \rightarrow$ add levels
- Find positions $p_0 \dots p_i$
- Insert at levels $0..i$
- Randomized algorithm

Delete

- Search for key x
- Find in all levels
- Remove from each level
- Update pointers
- No randomness

Randomization

- Coin toss (50% head)
- $i =$ heads until tail
- Expected height $O(\log n)$
- Balancing without rotations

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Skip list search mein "scan forward" aur "drop down" ka kya matlab hai?

A: "Scan forward" means current level mein aage move karna jab target key next node se badi ho. "Drop down" means neechay wali level par jana jab target key next node se chhoti ho — is tarah hum binary search ki tarah search space reduce karte hain.

Q: Skip list insertion mein randomness kyun use karte hain?

A: Randomness se levels assign hote hain jo expected $O(\log n)$ height guarantee karte hain — is se balanced tree ki tarah performance milti hai bina complex rotations ke.

Q: Agar coin toss se $i=5$ aata hai aur $h=2$ hai toh kya hoga?

A: $i > h$ hai, is liye hum new levels S_3, S_4, S_5 aur S_6 add karenge ($h+1$ se $i+1$ tak) — phir node ko levels 0 se 5 tak insert karenge.

Q: Skip list delete mein randomness kyun nahi hai?

A: Delete sirf node ko dhoondta hai aur usay har level se remove kar deta hai — isme koi new decision nahi lena, bas pointers adjust karte hain.

Q: Skip list AVL tree se kis tarah better hai?

A: Skip list implement karna simpler hai — AVL mein rotations complex hain, skip list mein randomization balancing sambhal leta hai. Dono expected $O(\log n)$ performance dete hain.

► QUICK QUIZ (10 QUESTIONS)

1. Skip list search kahan se start hoti hai?

(Top-left)

2. Insertion mein coin toss kya decide karta hai?

(Level / i)

3. Agar target $>$ next node hai toh kya karte hain? **(Scan forward)**
4. Agar target $<$ next node hai toh kya karte hain? **(Drop down)**
5. Skip list ki expected search complexity? **($O(\log n)$)**
6. Insertion randomized algorithm kis type ki hai? **(Coin toss based)**
7. Delete mein randomness hai? **(Nahi)**
8. Skip list har level mein kya special keys hain? **($+\infty, -\infty$)**
9. So list kya contain karti hai? **(All elements)**
10. Skip list kiska alternative hai? **(AVL tree / BST)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Skip List:** Series of sorted linked lists $S_0 \supseteq S_1 \supseteq \dots \supseteq S_h$ — har list subset of previous.
- **Special Keys:** $+\infty$ aur $-\infty$ — boundaries in every list.
- **Randomized Insertion:** Coin toss decides insertion level i — node goes to levels $0..i$.

IMPORTANT CONCEPTS

- Search: scan forward + drop down — binary search jaisi movement.
- Insert: coin toss $\rightarrow i \rightarrow$ find positions $\rightarrow$ insert in levels $0..i$.
- Delete: find + remove from all levels + pointer adjustment.
- Expected height $O(\log n)$ — randomization balancing karta hai.

△ IMPORTANT FORMULAS

- Expected search/insert/delete: $O(\log n)$
- Expected height: $O(\log n)$
- Coin toss probability: $P(\text{head}) = 0.5$

FREQUENTLY ASKED TOPICS

- Skip list search algorithm (scan forward + drop down)
- Insertion with randomization (coin toss)
- Deletion process
- Why skip list is alternative to AVL

△ COMMON MISTAKES

- Skip list search mein direct binary search samajh lena (yeh multi-level list hai)
- Insertion mein $i=h$ par new level add karna bhool jana
- Delete mein randomization ki zaroorat samajh lena (nahi hai)
- S_0 ko subset samajh lena (yeh all elements contain karti hai)

One-Day-Before-Exam Summary: Skip list ek randomized data structure hai jo linked list ki limitation ($O(n)$ search) ko overcome

karta hai. Isme multiple levels hain — Level 0 mein sab elements, higher levels subsets. Search top level se start hoti hai: agar target next node se badi hai toh scan forward, chhoti hai toh drop down — is tarah $O(\log n)$ expected time mein search ho jati hai. Insertion mein coin toss decide karta hai ke node kitni levels mein jayega — is randomization ki wajah se skip list balanced rehti hai bina rotations ke. Delete simple hai — node dhoondho aur har level se remove karo. Skip list AVL tree ka easy alternative hai.

Lesson 41: Skip List

Implementation & Hashing

Intro

TowerNode · QuadNode · Hash Functions · Constant-Time Operations

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aapke paas ek hostel mein 100 students hain aur har student ka ek unique roll number hai. Agar aap unki files almari mein alphabetical order mein rakhein toh search fast hai (binary search) lekin naya student aane par shifting mushkil. Skip list is ka solution hai — express lanes ki tarah. Ab hashing ka concept dekhte hain: agar aap har student ka roll number directly almari ke shelf number mein convert kar den (hash function), toh aap seedha us shelf par ja kar file nikaal sakte hain — ek hi step mein! Hashing woh magic hai jo constant time mein insert, find aur delete kar deta hai — jaise hostel warden ko pata ho ke kaunsa student kis room mein hai, woh direct room number par ja kar milta hai.

KEY TERMS

- 1 **TowerNode** — Skip list node with array of next pointers — dynamic number of levels.
- 2 **QuadNode** — Skip list node with 4 pointers: next, prev, up, down — key copied at each level.
- 3 **MAXLEVEL** — Upper limit on number of levels in a skip list — prevents infinite towers.
- 4 **Hashing** — Technique to map keys to array indices using a hash function — enables $O(1)$ operations.
- 5 **Hash Function** — Mathematical function that takes a key and returns an integer (array index).
- 6 **Associative Array** — Array where indices are keys (strings) instead of integers — implemented via hashing.
- 7 **Table Size (T)** — Internal array size — should ideally be prime to avoid clustering.

DIAGRAM TIME

```
TowerNode
key = 40
next[0] → 50
next[1] → 57
next[2] → tail
next[3] → NULL
```

```
TowerNode
key = 40
next[0] → 50
next[1] → 57
next[2] → tail
next[3] → NULL
```

Each TowerNode has dynamic array of next pointers.
Level decided by coin toss at insertion time.

↗ *TowerNode* mein *next pointers* ka array hai — har level ke liye ek pointer.

QuadNode	
key=31	
prev	→
next	→
up	→
down	→

QuadNode
key=31
prev →
next →
up →
down →

$$S_3: \quad -\infty \quad \longleftarrow \quad \longrightarrow \quad +\infty$$

$$S_2: \quad -\infty \longleftrightarrow 31 \longleftrightarrow +\infty$$

$$S_1: \quad -\infty \leftrightarrow 23 \leftrightarrow 31 \leftrightarrow 34 \leftrightarrow +\infty$$

$S_0: -\infty \leftrightarrow 12 \leftrightarrow 23 \leftrightarrow 31 \leftrightarrow 34 \leftrightarrow +\infty$

Key 31 is copied at S_1, S_2, S_3 (3 copies).

↗ *QuadNode mein key copy hoti hai har level par — up/down pointers help navigate.*

► HASHING — FRUIT EXAMPLE

Hash Function $h(\text{key}) = \text{sum}(\text{ASCII}) \% \text{TABLESIZE}$

Keys:

"apple" → 5
"watermelon" → 3
"grapes" → 8
"cantaloupe" → 7
"kiwi" → 0
"strawberry" → 9
"mango" → 6
"banana" → 2

Hash values:

Array (size 10):

Index:	0	1	2	3	4	5	6	7
Value:	kiwi	(empty)	banana	watermelon	(empty)	apple	mango	canta

$\text{find}(\text{"apple"}) \rightarrow h(\text{"apple"})=5 \rightarrow \text{table}[5] = \text{"apple"} \rightarrow O(1)!$

↗ *Hash function key ko array index mein convert karta hai — direct access $O(1)$ time mein.*

AVL Connection: AVL tree $O(\log n)$ search deta hai, lekin hashing $O(1)$ mein kaam karta hai — lekin hashing mein collisions handle karna padta hai (next lecture mein). Skip list AVL ka

randomized alternative hai — hashing ek naya paradigm hai jo constant time operations provide karta hai.

CODE EXPLANATION

```
// Simple Hash Function for Strings
int hashCode(char* s)
{ int sum = 0; for (int i = 0; i < strlen(s); i++) sum = sum + s[i]; // ASCII value
return sum % TABLESIZE; } //
Alternative Polynomial Hash Function
int hashCodePoly(char* s) { int sum = 0; int b = 7; // base (prime)
for (int i = 0; i < strlen(s); i++) sum = sum * b + s[i]; return sum %
TABLESIZE; } // Hash Function for Integer Keys
int hashCodeInt(int key) { return key % TABLESIZE; // T should be prime }
```

- 1 **sum = sum + s[i];** — ASCII values add karo — simple hash function.
- 2 **return sum % TABLESIZE;** — Modulo operation ensures index is within array bounds.
- 3 **sum = sum * b + s[i];** — Polynomial hash — base b (prime) use karta hai, collisions kam hote hain.
- 4 **TABLESIZE should be prime** — Prime table size collisions reduce karta hai, especially for integer keys.

EXAM FOCUS POINTS

MCQ

Skip list implementation: TowerNode (array of next pointers) ya QuadNode (4 pointers with key copying).

FORMULA

Skip list expected space: $O(n)$ with constant factor $\sim 15-20n$. Expected search/insert/delete: $O(\log n)$.

CONFUSION

QuadNode mein key copy hoti hai har level par — TowerNode mein key ek baar store hoti hai.

MCQ

Hashing mein insert/find/remove all $O(1)$ expected time — key ko hash function se index mein convert karte hain.

FORMULA

Simple hash: $h(s) = \sum \text{ASCII}(s[i]) \% T$. Polynomial hash: $h(s) = \sum s[i] * b^i \% T$.

CONFUSION

Hash function ko "random" nahi hona chahiye — deterministic hona chahiye (same key $\rightarrow$ same index).

MIND MAP

Skip List Implementation → Hashing Intro

Skip List Impl

- TowerNode: array of next
- QuadNode: 4 pointers
- MAXLEVEL upper bound
- Randomized levels

Performance

- Space: $O(n) \sim 15-20n$
- Search: $O(\log n)$
- Insert: $O(\log n)$
- Delete: $O(\log n)$

Hashing

- Key → hash function → index
- Array storage
- Non-consecutive placement
- Associative array concept

Hash Functions

- Sum of ASCII % T
- Polynomial: $\sum s[i] * b^i$
- Integer: key % T
- T should be prime

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: TowerNode aur QuadNode mein kya farq hai?

A: TowerNode mein next pointers ka array hai — key ek baar store hoti hai. QuadNode mein 4 pointers (prev, next, up, down) hain — key har level par copy hoti hai. TowerNode memory efficient hai, QuadNode traversal easy (up/down available).

Q: Skip list mein MAXLEVEL kyun zaroori hai?

A: Random coin toss se kabhi kabhi bahut zyada heads aa sakte hain — is se tower bahut lambi ho jati hai aur memory issue ho sakta hai. MAXLEVEL upper limit lagata hai.

Q: Hashing insert/find/remove $O(1)$ kyun hain?

A: Hash function key ko seedha array index mein convert kar deta hai — humein search ya traverse nahi karna, direct index par access karte hain. Is liye constant time.

Q: Simple hash function mein kya problem ho sakti hai?

A: "ABC" aur "CBA" ka sum same hoga ($65+66+67$) — is liye different keys ka same index aa sakta hai (collision). Is liye polynomial hash better hai.

Q: Hash function ko deterministic kyun hona chahiye?

A: Agar hash function random hota toh same key har baar different index deta — phir hum data dhoond nahi sakte. Is liye hash function deterministic hona chahiye.

► QUICK QUIZ (10 QUESTIONS)

1. TowerNode mein next pointers kaise store hain?

(Array)

2. QuadNode mein kitne pointers hain?

(4)

- | | |
|---|---------------------------------|
| 3. Skip list expected search complexity? | ($O(\log n)$) |
| 4. Hashing insert/find/delete complexity? | ($O(1)$) |
| 5. Simple hash function kya karta hai? | (Sum ASCII % T) |
| 6. Table size (T) kya hona chahiye? | (Prime) |
| 7. Associative array kya hai? | (String indices) |
| 8. Hash function ka output kya hota hai? | (Array index) |
| 9. QuadNode mein key kahan copy hoti hai? | (Har level par) |
| 10. Skip list kiska alternative hai? | (AVL tree) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **TowerNode:** Skip list node with array of next pointers — dynamic levels.
- **QuadNode:** Skip list node with 4 pointers (prev, next, up, down) — key copied per level.
- **Hashing:** Key to array index mapping via hash function — $O(1)$ operations.
- **Hash Function:** Deterministic function that returns index for a given key.

IMPORTANT CONCEPTS

- Skip list: TowerNode (efficient memory) vs QuadNode (easy traversal).
- Expected performance: $O(\log n)$ for all operations.
- Hashing: insert/find/delete all $O(1)$ expected — array with non-consecutive storage.
- Simple hash: sum of ASCII codes; polynomial hash: $\sum s[i] * b^i$.
- Table size should be prime to reduce collisions.

△ IMPORTANT FORMULAS

- Simple hash: $h(s) = (\sum \text{ASCII}(s[i])) \% T$
- Polynomial hash: $h(s) = (\sum s[i] * b^i) \% T$
- Integer hash: $h(k) = k \% T$ (T prime)
- Skip list expected height: $O(\log n)$

FREQUENTLY ASKED TOPICS

- TowerNode vs QuadNode implementation
- Skip list performance analysis
- Hash function design (simple vs polynomial)
- Why hashing gives $O(1)$ operations

△ COMMON MISTAKES

- Skip list ko binary search tree samajh lena (yeh multi-level list hai)

- Hash function ko random samajh lena (deterministic hai)
- QuadNode mein key copy ko memory waste samajh lena (traversal easy karne ke liye hai)
- Table size prime na hona — collisions increase kar sakta hai

One-Day-Before-Exam Summary: Is lecture mein humne skip list ki implementation dekhi — TowerNode (array of next pointers) aur QuadNode (4 pointers with key copying) — dono $O(\log n)$ expected performance dete hain. Phir humne hashing ka concept introduce kiya — jo constant time ($O(1)$) insert, find aur delete provide karta hai. Hash function key ko array index mein convert karta hai — simple hash sum of ASCII codes use karta hai, polynomial hash base b (prime) use karta hai. Table size prime hona chahiye to avoid collisions. Hashing associative arrays (string indices) ki bhi basis hai.

Lesson 42: Collision Resolution

— Linear Probing

Open Addressing · Clustering · Deletion with States

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aap ek hostel ke rooms mein students ko allocate kar rahe hain. Har student ka ek unique roll number hai, aur aap ek formula (hash function) use karte hain yeh decide karne ke liye ke kaunsa student kis room mein rahega. Lekin kya ho agar formula do students ko ek hi room number de? Yeh hai collision! Ab aap kya karenge? Aap student ko agle khali room mein rakh denge (linear probing) — jaise hostel warden kehti hai "yeh room full hai, agla khali room dekh lo". Agar room khali nahi hai toh aap agla room dekhte rahenge. Agar koi student chala jata hai (delete), toh room "khali" mark kar dete hain taake dobara koi aa sake. Lekin agar bahut saare students ek saath aate hain toh ek "cluster" ban jata hai — jaise hostel ki ek poori corridor full ho jaye — aur efficiency gir jati hai.

KEY TERMS

- 1 **Collision** — *Jab do different keys ka hash function same index return kare.*
- 2 **Open Addressing (Closed Hashing)** — *Collision resolution technique — array mein hi empty location search karte hain.*
- 3 **Linear Probing** — *Open addressing ki technique — collision par $\text{index}+1$, $\text{index}+2$, ... check karte hain.*
- 4 **Probe Sequence** — $h_0(x), h_1(x), h_2(x), \dots$ where $h_i(x) = (\text{hash}(x) + f(i)) \% \text{TableSize}$.
- 5 **$f(i) = i$** — *Linear probing ki collision resolution strategy — $i = 0, 1, 2, 3, \dots$*
- 6 **Cluster** — *Occupied locations ka group jisme koi empty slot nahi — linear probing ki wajah se banta hai.*
- 7 **States (Occupied/Empty/Deleted)** — *Deletion handle karne ke liye har location ka status — Empty, Occupied, Deleted.*
- 8 **Chaining (Separate Chaining)** — *Collision resolution with linked lists — har array position list ka head.*

DIAGRAM TIME

► LINEAR PROBING – INSERTION EXAMPLE

Array (size 10, prime $T=11$ used for example):

Initial state:

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

Hash values: $h(\text{"apple"})=5$, $h(\text{"mango"})=6$, $h(\text{"grapes"})=8$, $h(\text{"banana"})$

After inserting apple, mango, grapes, banana:

[0] [1] [2]banana [3] [4] [5]apple [6]mango [7] [8]grapes

Insert "honeydew" with $h(\text{"honeydew"}) = 6$:

Step 1: check index 6 → occupied (mango)

Step 2: check index 7 → empty → insert here!

After insertion:

[0] [1] [2]banana [3] [4] [5]apple [6]mango [7]honeydew [8]

Collisions: honeydew collided once (at index 6).

↗ *Linear probing collision par agla khali index dhoondhta hai — wrap around bhi ho sakta hai.*

► SEARCH & DELETION – THREE STATES

Array with states:

Index: 142 143 144 145 146 147 148

State: Occupied Occupied Occupied Occupied Empty Occupied Occupied

Value: robin sparrow hawk seagull -- bluejay owl

$h(\text{"hawk"}) = 143$:

Step 1: index 143 → Occupied, value $\neq$ hawk → continue

Step 2: index 144 → Occupied, value $=$ hawk → found!

Delete hawk:

Step 1: Find hawk at 144

Step 2: Mark index 144 as "Deleted" (not Empty)

Search seagull after hawk deleted:

$h(\text{"seagull"}) = 143$

Step 1: index 143 → Occupied, value $\neq$ seagull

Step 2: index 144 → Deleted (not Empty) → continue!

Step 3: index 145 → Occupied, value $=$ seagull → found!

If we had marked 144 as Empty, search would stop at 144 and fail.

↗ Delete karne par state "Deleted" set karte hain — taake chain toot na jaye.

► CLUSTERING – PROBLEM

After many insertions with linear probing:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	A	B	C	D	E	F	--	G	H	I
State:	0	0	0	0	0	0	E	0	0	0

Cluster of size 6 (index 0-5).

Insert new key with $h(\text{key}) = 2$:

Step 1: index 2 → Occupied

Step 2: index 3 → Occupied

Step 3: index 4 → Occupied

Step 4: index 5 → Occupied

Step 5: index 6 → Empty → insert!

Cluster now size 7 (index 0-5 + 7? Actually hole at 6 breaks cluster)
But big clusters degrade performance – probe length increases.

➤ *Clusters linear probing ki performance ko degrade karte hain — quadratic probing is ka solution hai.*

AVL Connection: Hashing AVL tree se bhi fast hai ($O(1)$ vs $O(\log n)$), lekin collisions handle karna padta hai. Linear probing simple hai lekin clustering ki problem hai — quadratic probing aur chaining better alternatives hain.

CODE EXPLANATION

```
// Linear Probing Insert void insert(Key key, Value val)
{ int index = hashCode(key); int i = 0; while (state[(index +
i) % TABLE_SIZE] == OCCUPIED) { if (table[(index + i) %
TABLE_SIZE].key == key) { // update existing table[(index +
i) % TABLE_SIZE].value = val; return; } i++; } // insert at
empty/deleted slot int pos = (index + i) % TABLE_SIZE;
table[pos] = {key, val}; state[pos] = OCCUPIED; } // Linear
Probing Find Value* find(Key key) { int index =
hashCode(key); int i = 0; while (state[(index + i) %
TABLE_SIZE] != EMPTY) { int pos = (index + i) % TABLE_SIZE;
if (state[pos] == OCCUPIED && table[pos].key == key) return
&table[pos].value; i++; } return NULL; // not found } //
Delete void remove(Key key) { int index = hashCode(key); int
i = 0; while (state[(index + i) % TABLE_SIZE] != EMPTY) { int
pos = (index + i) % TABLE_SIZE; if (state[pos] == OCCUPIED &&
table[pos].key == key) { state[pos] = DELETED; // mark
deleted return; } i++; } }
```

- 1 **while (state[pos] == OCCUPIED)** — Jab tak occupied hai, aage badho (linear probing).
- 2 **state[pos] == DELETED** — Deleted slot bhi empty ki tarah use ho sakta hai — is liye loop sirf EMPTY par rukta hai.
- 3 **state[pos] = DELETED;** — Delete karte waqt state DELETED set karo — chain intact rehti hai.
- 4 **(index + i) % TABLE_SIZE** — Wrap around — end par pahunch kar start se continue.

EXAM FOCUS POINTS

FORMULA

Linear probing: $h_i(x) = (\text{hash}(x) + i) \% \text{TableSize}$, where $f(i) = i$.

MCQ

Open addressing = closed hashing. Linear probing open addressing ki ek technique hai.

CONFUSION

Delete karte waqt state EMPTY nahi, DELETED set karte hain — warna chain toot jayegi aur find fail ho jayega.

PRACTICE

Table size 10, $\text{hash}(x) = x \% 10$. Insert 5, 15, 25 — linear probing se $5 \rightarrow 5$, $15 \rightarrow 6$, $25 \rightarrow 7$ (collisions: 15 collides at 5, 25 collides at 5 and 6).

MCQ

Clusters linear probing ki performance degrade karte hain — quadratic probing is ka solution hai.

CONFUSION

Chaining (linked list) mein array size limitation nahi hai — open addressing mein array full ho sakta hai.

MIND MAP

Collision Resolution

Collision

- Two keys same hash
- Hash function limitation
- Inevitable with large data

Linear Probing

- $f(i) = i$
- Check index+1, +2, ...
- Wrap around at end
- Three states required

Problems

- Clustering
- Performance degrades
- Deletion needs states
- Array can become full

Alternatives

- Quadratic probing ($f(i)=i^2$)
- Double hashing
- Separate chaining
- Linked lists at each slot

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Linear probing mein "wrap around" kya hai?

A: Jab array ke end par pahunch jayein aur empty slot na mile, toh start (index 0) se continue karte hain — array ko circular treat karte hain.

Q: Delete karte waqt state DELETED kyun set karte hain, EMPTY kyun nahi?

A: Agar EMPTY set kar denge toh search chain toot jayegi — jo elements baad mein insert hue hain (due to collision) woh nahi milenge. DELETED state chain continue karne deti hai.

Q: Cluster kya hai aur kyun problem hai?

A: Cluster occupied locations ka continuous group hai jisme koi empty slot nahi. Naya element agar cluster mein hash ho toh probe length badh jati hai — performance degrade hoti hai.

Q: Open addressing aur chaining mein kya farq hai?

A: Open addressing mein data array mein hi store hota hai (probing se empty slot dhoondte hain). Chaining mein har array position ek linked list ka head hai — collision par list mein add karte hain.

Q: Linear probing ki probe sequence kya hai?

A: $h_0(x) = \text{hash}(x)$, $h_1(x) = \text{hash}(x)+1$, $h_2(x) = \text{hash}(x)+2$, ... — $f(i) = i$.

► QUICK QUIZ (10 QUESTIONS)

1. Collision kya hai? **(Same hash for two keys)**
2. Linear probing mein $f(i)$ kya hai? **(i)**
3. Delete karte waqt kaunsi state set karte hain? **(DELETED)**

- | | |
|---|---------------------------|
| 4. Cluster kya hai? | (Group of occupied slots) |
| <hr/> | |
| 5. Open addressing ka doosra naam? | (Closed hashing) |
| <hr/> | |
| 6. Linear probing mein wrap around kyun? | (Empty slot at start) |
| <hr/> | |
| 7. Chaining mein array mein kya store hota hai? | (Linked list head) |
| <hr/> | |
| 8. Chaining ka advantage over open addressing? | (No array full) |
| <hr/> | |
| 9. Linear probing ki probe sequence? | (hash+i) |
| <hr/> | |
| 10. Clustering kis technique mein hoti hai? | (Linear probing) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Collision:** Different keys same hash index.
- **Linear Probing:** Collision resolution by checking consecutive slots ($f(i)=i$).
- **Open Addressing:** Data array mein hi store hota hai — empty slot search.
- **Chaining:** Har slot linked list hai — collision par list mein add.

IMPORTANT CONCEPTS

- Linear probing: $h_i(x) = (\text{hash}(x) + i) \% \text{TableSize}$
- Three states: OCCUPIED, EMPTY, DELETED — deletion ke liye zaroori.
- Clustering linear probing ki major problem — performance degrade.
- Chaining advantage: array size limitation nahi, simple insertion/deletion.

△ IMPORTANT FORMULAS

- Linear probing: $f(i) = i$
- Quadratic probing: $f(i) = i^2$ (next lecture)
- Probe sequence: $h_0, h_1, h_2, \dots$

FREQUENTLY ASKED TOPICS

- Linear probing algorithm (insert/find/delete)
- Why three states are needed
- Clustering problem
- Chaining vs open addressing comparison

△ COMMON MISTAKES

- Delete karte waqt EMPTY set karna (galat — DELETED karna hai)
- Linear probing ko quadratic probing samajh lena
- Chaining mein array size limitation samajh lena (nahi hai)
- Cluster ko problem nahi samajh lena (performance degrade karta hai)

One-Day-Before-Exam Summary: Collision tab hoti hai jab do different keys ka hash function same index return kare. Linear probing open addressing ki ek technique hai — collision par index+1, index+2, ... check karte hain (wrap around bhi). Delete karte waqt state "DELETED" set karte hain — "EMPTY" nahi — taake search chain toot na jaye. Linear probing ki major problem "clustering" hai — occupied slots ka group jisme koi empty nahi — is se performance degrade hoti hai. Alternatives: quadratic probing aur chaining (linked lists). Chaining mein array size limitation nahi aur insertion/deletion simple hai.

Lesson 43: Hashing

Applications & Sorting Intro

Hash Table Uses · When to Hash · Selection Sort

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aapke paas ek library hai jahan books unke titles ke hisaab se rakhi hain. Hashing ki tarah, aap title ka ek formula (hash function) use karte hain yeh decide karne ke liye ke book kis shelf par rakhi jaye — direct access, $O(1)$! Lekin agar do books ka formula same shelf number de (collision), toh aap unhe aglay khali shelf par rakh dete hain (linear probing) ya ek hi shelf par list bana dete hain (chaining). Hashing bahut useful hai jab aapko fast search chahiye — jaise compiler symbol table, spell checker, ya chess game positions. Lekin agar aapko books ko alphabetical order mein arrange karna hai (sorting), toh hashing kaam nahi aati — is liye sorting algorithms (jaise selection sort) use karte hain. Sorting 80-90% applications mein use hoti hai — jaise students ki list ko marks ke hisaab se arrange karna.

KEY TERMS

- 1 **Symbol Table** — *Compiler ka data structure jo variables, functions, types track karta hai — hashing use karta hai.*
- 2 **Spell Checker** — *Dictionary ko hash table mein store karta hai — words $O(1)$ mein check hote hain.*
- 3 **Hash Table Stability** — *Hash table tab best hai jab searches zyada hain, insert/delete kam.*
- 4 **Sorting** — *Data ko ascending/descending order mein arrange karna — 80-90% applications mein use.*
- 5 **Selection Sort** — *Elementary sorting algorithm: har iteration mein smallest element select karo aur correct position par rakho.*
- 6 **Elementary Sorting Algorithms** — *Selection, Insertion, Bubble sort — simple baseline algorithms.*

DIAGRAM TIME

► HASHING COLLISION STRATEGIES – SUMMARY

Linear Probing ($f(i) = i$):

Insert 506 (hash=6), 206 (hash=6), 806 (hash=6), 807 (hash=7):

Index: 6 7 8
506 → 206 → 806 → 807
(clustering effect)

Quadratic Probing ($f(i) = i^2$):

Insert with hash=6:

$6 \rightarrow 6+1^2=7 \rightarrow 6+2^2=10 \rightarrow 6+3^2=15 \dots$
(scatters better, reduces clustering)

Chaining (Linked List):

Index 0 → [num1] → [num2] → [num3] → ...
Each array slot is head of a linked list.
No array full problem.

↗ *Teeno collision strategies ka comparison — linear probing clusters banata hai, quadratic better scatter karta hai, chaining list use karti hai.*

► SELECTION SORT – STEP-BY-STEP

Array: [20, 8, 5, 10, 7]

Step 1: Find smallest (5) → swap with index 0
[5, 8, 20, 10, 7]

Step 2: Find next smallest (7) → swap with index 1
[5, 7, 20, 10, 8]

Step 3: Find next smallest (8) → swap with index 2
[5, 7, 8, 10, 20]

Step 4: Find next smallest (10) → already at index 3
[5, 7, 8, 10, 20]

Sorted! Complexity: $O(n^2)$

↗ *Selection sort har position ke liye minimum element dhoondta hai aur swap karta hai — $O(n^2)$ time.*

AVL Connection: AVL tree in-order traversal sorted data deta hai — $O(n)$ time. Hashing sorted data nahi deti — is liye jab sorting requirement ho, AVL tree ya sorting algorithms use karte hain.

CODE EXPLANATION

```
// Selection Sort Implementation
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Find minimum element in unsorted part
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        // Swap found minimum with first element
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}
```

- 1 **for (int i = 0; i < n-1; i++)** — Har position *i* par correct element laane ke liye loop.
- 2 **minIndex = i;** — Maan lo current position ka element minimum hai.
- 3 **for (int j = i+1; j < n; j++)** — Unsorted part mein minimum dhoondho.
- 4 **swap(arr[i], arr[minIndex]);** — Minimum ko current position par laao.

EXAM FOCUS POINTS

MCQ

Symbol table compiler mein variables track karta hai — hashing use hoti hai.

MCQ

Spell checker dictionary ko hash table mein store karta hai — $O(1)$ lookup.

CONFUSION

Hashing sorted data nahi deti — agar min/max chahiye toh hashing suitable nahi.

FORMULA

Selection Sort complexity: $O(n^2)$ — har position ke liye $n-i$ elements scan.

PRACTICE

Array [20,8,5,10,7] selection sort ke steps likhein — first pass 5, second 7, third 8.

CONFUSION

Hash table stable honi chahiye — frequent insert/delete ke liye AVL tree better.

MIND MAP

Hashing Applications → Sorting Intro

Hashing Applications

- Compiler symbol table
- Spell checker
- Game positions (chess)
- Inequality check

When to Use

- Many searches
- Stable table
- Insert/delete few
- No sorting needed

Sorting Intro

- 80-90% applications
- Sort + search go together
- Elementary algorithms

Selection Sort

- Find smallest → swap
- Find next smallest → swap
- $O(n^2)$ complexity
- Simple baseline

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Compiler symbol table mein hashing kyun use karte hain?

A: Compiler ko variables ki quick search chahiye — hashing $O(1)$ lookup deta hai. Variable name key hai, aur attributes (type, scope) value hain.

Q: Spell checker mein dictionary ko hash table mein kyun store karte hain?

A: Har word check karte waqt $O(1)$ time mein dictionary mein lookup ho jata hai — agar word nahi milta toh misspelling hai.

Q: Hashing kab suitable nahi hai?

A: Jab frequent insert/delete ho, ya sorted data (min/max) chahiye ho — tab AVL tree ya other data structures better hain.

Q: Selection Sort ka basic idea kya hai?

A: Har position ke liye unsorted part mein minimum element dhoondho aur us position par swap karo — is tarah array gradually sorted ho jata hai.

Q: Sorting itni important kyun hai?

A: 80-90% applications mein sorting use hoti hai — searching, data analysis, reporting sab mein sorted data helpful hai.

► QUICK QUIZ (10 QUESTIONS)

1. Compiler symbol table kis data structure se implement hoti hai? **(Hash table)**

2. Spell checker dictionary kahan store karta hai? **(Hash table)**

3. Hash table kab best hai? **(Many searches, stable)**

4. Hashing sorted data deti hai? **(Nahi)**
-
5. Sorting kis percentage applications mein use hoti hai? **(80-90%)**
-
6. Selection Sort ka pehla step kya hai? **(Find smallest)**
-
7. Selection Sort complexity? **($O(n^2)$)**
-
8. Elementary sorting algorithms **(Selection, Insertion, Bubble)** examples?
-
9. Hash table ko stable kyun hona chahiye? **(Insert/delete kam)**
-
10. Chess game positions store karne ke liye kya use **(Hash table)** hota hai?

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Symbol Table:** Compiler ka hash table — variables track karta hai.
- **Spell Checker:** Dictionary hash table — $O(1)$ word lookup.
- **Selection Sort:** Elementary sort — har iteration mein minimum select karo.

IMPORTANT CONCEPTS

- Hashing best for: stable table, frequent searches, no sorting needed.
- Hashing not suitable for: frequent insert/delete, sorted output, min/max queries.
- Selection sort: $O(n^2)$ — simple but slow for large data.
- Sorting is fundamental — 80-90% applications mein use.

△ IMPORTANT FORMULAS

- Selection Sort: $O(n^2)$ time, $O(1)$ space.
- Hash table lookup: $O(1)$ expected.

FREQUENTLY ASKED TOPICS

- Hashing applications (symbol table, spell checker, games)
- When to use hashing vs AVL tree
- Selection sort algorithm and complexity
- Elementary sorting algorithms overview

△ COMMON MISTAKES

- Hashing ko sorting ke liye use karna (nahi karte)
- Selection sort ko $O(n)$ samajh lena ($O(n^2)$ hai)
- Hash table stable nahi honi chahiye samajh lena (stable honi chahiye)
- Symbol table ko compiler ki memory samajh lena (data structure hai)

One-Day-Before-Exam Summary: Hashing ke practical applications hain — compiler symbol table, spell checker, game positions. Hash table tab best hai jab searches zyada hain aur table stable hai (few insert/delete). Agar sorting ya min/max chahiye toh hashing suitable nahi — AVL tree use karte hain. Sorting 80-90% applications mein use hoti hai. Selection sort elementary algorithm hai — har iteration mein unsorted part se minimum select karo aur correct position par rakho — $O(n^2)$ complexity.

Lesson 44: Elementary Sorting Algorithms

Selection · Insertion · Bubble Sort — $O(n^2)$ Complexity

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aapke paas chai ki 4 cups hain jin par alag-alag temperature likha hai: 19°C , 12°C , 5°C , 7°C . Aap inhe ascending order mein arrange karna chahte hain (coldest se hottest). Selection sort: aap sab se thandi cup dhoondte hain (5°C) aur pehle rakhte hain, phir agli thandi (7°C) aur aise hi — jaise khazana dhoondna. Insertion sort: aap pehle do cups sort karte hain (12°C , 19°C), phir teesri cup (5°C) ko correct position par insert karte hain — jaise cards sort karna. Bubble sort: aap neighboring cups ko compare karte hain aur garam cup ko neechay le jaate hain — jaise bubbles upar uthte hain. Teenon algorithms simple hain lekin slow — $O(n^2)$ — agar 1 million cups hain toh bahut time lagega. Is liye next lecture mein hum fast algorithms (merge sort, quick sort) dekhenge jo $O(n \log n)$ hain.

KEY TERMS

- 1 **Selection Sort** — *Har iteration mein unsorted part ka minimum element dhoond kar correct position par swap karo.*
- 2 **Insertion Sort** — *Array ko left part sorted rakho, har naya element right se lekar left part mein correct position par insert karo (shifting).*
- 3 **Bubble Sort** — *Pair-wise swapping — smaller elements upar (bubble) uthte hain, larger neechay jaate hain.*
- 4 **In-Place Algorithm** — *Sorting ke liye extra array/memory nahi chahiye — original array mein hi swap/shift hota hai.*
- 5 **$O(n^2)$ Complexity** — *N elements ke liye time proportional to n^2 — large data ke liye slow.*
- 6 **Divide and Conquer** — *Problem ko chhote parts mein divide karo, solve karo, phir merge karo — Merge Sort, Quick Sort, Heap Sort.*

DIAGRAM TIME

► SELECTION SORT – STEP-BY-STEP

Array: [19, 12, 5, 7]

Step 1: Find smallest (5) → swap with index 0
[5, 12, 19, 7]

Step 2: Find smallest in remaining (7) → swap with index 1
[5, 7, 19, 12]

Step 3: Find smallest in remaining (12) → swap with index 2
[5, 7, 12, 19]

Sorted! [5, 7, 12, 19]

↗ *Selection sort har position ke liye minimum dhoondta hai aur swap karta hai.*

► INSERTION SORT – STEP-BY-STEP

Array: [19, 12, 5, 7]

Step 1: Sort first 2 elements: [19, 12] → [12, 19]
[12, 19, 5, 7]

Step 2: Insert 5 into [12, 19] → [5, 12, 19]
[5, 12, 19, 7]

Step 3: Insert 7 into [5, 12, 19] → [5, 7, 12, 19]

Sorted! [5, 7, 12, 19]

↗ Insertion sort left part sorted rakhta hai — naya element correct position par shift karke insert hota hai.

► BUBBLE SORT — STEP-BY-STEP

Array: [19, 12, 5, 7]

Pass 1 (compare adjacent, swap if out of order):

19 > 12 → swap → [12, 19, 5, 7]

19 > 5 → swap → [12, 5, 19, 7]

19 > 7 → swap → [12, 5, 7, 19] ← 19 is now at correct position

Pass 2 (up to index 2):

12 > 5 → swap → [5, 12, 7, 19]

12 > 7 → swap → [5, 7, 12, 19] ← 12 is now at correct position

Pass 3 (up to index 1):

5 > 7? No → [5, 7, 12, 19] ← sorted!

Sorted! [5, 7, 12, 19]

↗ Bubble sort neighboring elements swap karta hai — largest element end par pahunch jata hai.

► $O(N^2)$ VS $O(N \log N)$ — COMPARISON

N	N^2	$N \log_2 N$
10	100	33
100	10,000	664
1,000	1,000,000	9,966
10,000	100,000,000	132,877
100,000	10,000,000,000	1,660,964

For $N=1,000,000$:

$N^2 = 1,000,000,000,000$ (1 trillion)

$N \log_2 N \approx 20,000,000$ (20 million)

$O(n^2)$ is 50,000 times slower than $O(n \log n)$!

↗ $O(n^2)$ algorithms large data ke liye bahut slow hain — $O(n \log n)$ algorithms much faster.

AVL Connection: AVL tree in-order traversal sorted data deta hai — $O(n)$ time. Sorting algorithms mein divide and conquer (merge sort, quick sort) AVL ki tarah $O(n \log n)$ performance dete hain — elementary sorts ($O(n^2)$) se better.

CODE EXPLANATION

```
// Selection Sort
void selectionSort(int *arr, int N) {
    for (int i = 0; i < N; i++) {
        int minIndex = i;
        for (int j = i+1; j < N; j++) {
            if (arr[j] < arr[minIndex]) minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }
}

// Insertion Sort
void insertionSort(int *arr, int N) {
    for (int i = 1; i < N; i++) {
        int val = arr[i];
        int j = i-1;
        while (j >= 0 && arr[j] > val) {
            arr[j+1] = arr[j]; // shift right
            j--;
        }
        arr[j+1] = val;
    }
}

// Bubble Sort (optimized)
void bubbleSort(int *arr, int N) {
    int bound = N-1;
    while (swapped > 0) {
        swapped = 0;
        for (int i = 0; i < bound; i++) {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                swapped = i;
            }
        }
        bound = swapped;
    }
}
```

- 1 **selectionSort:** *i loop har position ke liye, j loop minimum dhoondta hai — $O(n^2)$.*
- 2 **insertionSort:** *i loop har element ko left sorted part mein insert karta hai — shifting $O(n^2)$.*
- 3 **bubbleSort (optimized):** *swapped variable track karta hai last swap position — bound decreases.*
- 4 **All are in-place:** *Koi extra array nahi — sorting original array mein hoti hai.*

EXAM FOCUS POINTS

FORMULA

Selection Sort: Total comparisons = $N(N+1)/2 = O(n^2)$.

FORMULA

Insertion Sort worst-case: $O(n^2)$ — shifting sum = $(2+N)(N-1)/2$.

FORMULA

Bubble Sort: $O(n^2)$ — nested loops sum = $1+2+\dots+N = O(n^2)$.

MCQ

All three elementary sorts are in-place algorithms — no extra array needed.

CONFUSION

Selection sort mein minimum find karte hain, insertion sort mein shifting hoti hai, bubble sort mein pair-wise swapping.

PRACTICE

Array [20,8,5,10,7] selection sort steps likhein — first pass 5, second 7, third 8, fourth 10.

MCQ

$N \log_2 N$ algorithms: Merge Sort, Quick Sort, Heap Sort — divide and conquer.

MIND MAP

Elementary Sorting Algorithms ($O(n^2)$)

Selection Sort

- Find minimum $\rightarrow$ swap
- $N + (N-1) + \dots + 1$
- In-place
- Unstable sort

Insertion Sort

- Left part sorted
- Insert new element
- Shifting overhead
- Stable sort

Bubble Sort

- Pair-wise swap
- Larger sink down
- Optimized with bound
- Stable sort

Better Algorithms

- Merge Sort ($O(n \log n)$)
- Quick Sort ($O(n \log n)$)
- Heap Sort ($O(n \log n)$)
- Divide and Conquer

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Selection sort aur insertion sort mein kya basic farq hai?

A: Selection sort har iteration mein unsorted part se minimum dhoond kar swap karta hai. Insertion sort left part ko sorted rakhta hai aur naya element right se lekar left mein correct position par insert (shift) karta hai.

Q: Bubble sort ko "bubble" kyun kehte hain?

A: Kyunki smaller elements (lighter) pair-wise swapping ki wajah se upar (start) ki taraf uthte hain — jaise pani mein bubbles upar uthte hain.

Q: Teenon elementary sorts ki complexity kya hai?

A: Teenon $O(n^2)$ hain — selection, insertion, aur bubble sort. Large data ke liye slow hain.

Q: "In-place" algorithm ka kya matlab hai?

A: Sorting original array mein hi hoti hai — extra array ya memory nahi chahiye. Teenon sorts in-place hain.

Q: $O(n^2)$ aur $O(n \log n)$ mein kya farq hai?

A: $O(n^2)$ bahut slow hai large data ke liye — $N=1,000,000$ par $O(n^2)$ 1 trillion comparisons, $O(n \log n)$ sirf 20 million — 50,000 guna fast.

► QUICK QUIZ (10 QUESTIONS)

1. Selection sort har iteration mein kya dhoondta hai? **(Minimum)**
2. Insertion sort mein left part kaisa hota hai? **(Sorted)**
3. Bubble sort mein kya swap hota hai? **(Neighboring pairs)**

4. Teenon sorts ki complexity? **($O(n^2)$)**
5. In-place algorithm ka matlab? **(No extra array)**
6. $O(n \log n)$ algorithms examples? **(Merge, Quick, Heap)**
7. Selection sort total comparisons formula? **($N(N+1)/2$)**
8. Bubble sort ko optimized kyun kehte hain? **(swapped variable)**
9. Divide and conquer strategy kya hai? **(Divide → solve → merge)**
10. Sorting kitne percentage applications mein use hoti hai? **(80-90%)**

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Selection Sort:** Minimum select $\rightarrow$ swap — $O(n^2)$.
- **Insertion Sort:** Left part sorted, insert new element with shifting — $O(n^2)$.
- **Bubble Sort:** Pair-wise swap — smaller bubble up — $O(n^2)$.
- **In-Place:** No extra memory — all three are in-place.

IMPORTANT CONCEPTS

- Selection sort: minimum dhoondna — $N + (N-1) + \dots + 1$ comparisons.
- Insertion sort: shifting overhead — worst-case $O(n^2)$.
- Bubble sort: optimized with swapped variable — best-case $O(n)$ for sorted array.
- $O(n \log n)$ algorithms: Merge Sort, Quick Sort, Heap Sort — divide and conquer.

△ IMPORTANT FORMULAS

- Selection Sort: $N(N+1)/2 = O(n^2)$
- Insertion Sort worst-case: $(2+N)(N-1)/2 = O(n^2)$
- Bubble Sort: $\text{sum } 1+2+\dots+N = O(n^2)$
- $O(n \log n)$ is much faster than $O(n^2)$ for large n .

FREQUENTLY ASKED TOPICS

- Selection sort algorithm steps
- Insertion sort with shifting
- Bubble sort optimization (swapped variable)
- $O(n^2)$ vs $O(n \log n)$ comparison

△ COMMON MISTAKES

- Selection sort ko insertion sort samajh lena
- Insertion sort ki shifting ko bhool jana
- Bubble sort ko optimized nahi samajh lena (swapped variable use karte hain)

- Teenon sorts ko stable/unstable confuse karna

One-Day-Before-Exam Summary: Teen elementary sorting algorithms hain — Selection, Insertion, aur Bubble Sort. Selection sort har iteration mein minimum dhoond kar swap karta hai. Insertion sort left part sorted rakhta hai aur naya element shift karke insert karta hai. Bubble sort neighboring pairs swap karta hai — smaller elements upar uthte hain. Teenon $O(n^2)$ complexity ke hain — simple hain lekin large data ke liye slow. In-place hain — extra array nahi chahiye. Next lecture mein hum $O(n \log n)$ algorithms (Merge, Quick, Heap Sort) dekhenge — jo divide and conquer use karte hain aur bahut faster hain.

Lesson 45: Merge Sort, Quick Sort & Course Overview

Divide and Conquer · $O(n \log n)$ Algorithms · The Final Lecture

DEEP EXPLANATION — CHAI-TIME ANALOGY

Socho ke aapko 8 cups chai ko temperature ke hisaab se sort karna hai. Divide and conquer: aap cups ko do groups mein divide karte hain (4 aur 4), har group ko alag sort karte hain, phir dono groups ko merge karte hain — jaise do sorted packets ko milana. Merge sort exactly yahi karta hai — recursively divide karo, sort karo, merge karo. Quick sort: aap ek cup ko "pivot" chunte hain (jaise beech wala temperature), aur baaki cups ko pivot se chhote aur bade mein divide karte hain — phir dono groups ko recursively sort karte hain. Dono algorithms $O(n \log n)$ hain — 1000 cups ke liye $\sim 10,000$ comparisons vs $O(n^2)$ mein 1,000,000. Yeh hamara aakhri lecture hai — ab aap data structures aur algorithms ki duniya mein apna safar shuru kar sakte hain!

KEY TERMS

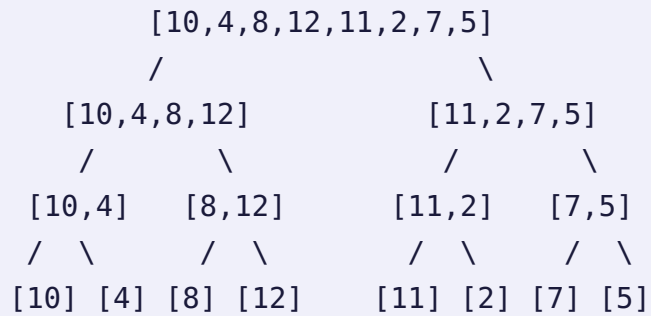
- 1 **Divide and Conquer** — *Problem ko chhote parts mein divide karo, solve karo, phir merge karo — Merge Sort, Quick Sort, Heap Sort.*
- 2 **Merge Sort** — *Recursively split array in half, sort halves, merge sorted halves — $O(n \log n)$ time, $O(n)$ extra space.*
- 3 **Merge Operation** — *Do sorted arrays ko ek sorted array mein combine karna — single loop, $O(n)$ time.*
- 4 **Quick Sort** — *Pivot element choose karo, partition karo (smaller left, larger right), recursively sort — $O(n \log n)$ average, in-place.*
- 5 **Pivot** — *Quick sort mein selected element jo array ko two parts mein divide karta hai.*
- 6 **Partition** — *Array ko pivot ke around rearrange karna — smaller elements left, larger right.*
- 7 **In-Place Algorithm** — *Quick sort in-place hai (extra array nahi), Merge sort in-place nahi (extra $O(n)$ space).*

DIAGRAM TIME

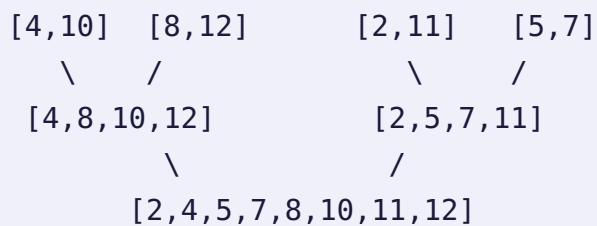
► MERGE SORT – RECURSIVE DIVIDE & CONQUER

Array: [10, 4, 8, 12, 11, 2, 7, 5]

Divide:



Merge (bottom-up):



Sorted! $O(n \log n)$ time.

↗ Merge sort recursively divides until single elements, then merges back up — 3 levels for 8 elements.

► MERGE OPERATION – TWO SORTED ARRAYS

```
Left:  [3, 5, 15, 28, 30]      Right: [6, 10, 14, 22, 43, 50]
      ↑                        ↑
      i                        j
```

Compare 3 and 6 → 3 smaller → tmp[0]=3, i++

Left: [3, 5, 15, 28, 30] Right: [6, 10, 14, 22, 43, 50]

Merged: [3, 5, 6, 10, 14, 15, 22, 28, 30, 43, 50]

Recursively sort left and right.

➤ *Quick sort pivot ko correct position par laata hai — fir left aur right recursively sort hote hain.*

AVL Connection: Merge Sort aur Quick Sort dono $O(n \log n)$ hain — AVL tree ki tarah efficient. Quick sort in-place hai (AVL ki tarah extra memory nahi), Merge sort stable hai lekin extra $O(n)$ memory leta hai.

CODE EXPLANATION

```
// Merge Sort (Recursive) void mergeSort(int arr[], int size)
{ int* tmp = new int[size]; if (tmp != NULL)
{ mergeSortRec(arr, size, tmp); delete[] tmp; } } void
mergeSortRec(int arr[], int size, int tmp[]) { if (size > 1)
{ int mid = size / 2; mergeSortRec(arr, mid, tmp);
mergeSortRec(arr + mid, size - mid, tmp); merge(arr, mid, arr
+ mid, size - mid, tmp); for (int i = 0; i < size; i++)
arr[i] = tmp[i]; } } // Quick Sort (Recursive) void
quickSort(int arr[], int size) { if (size > 1) { int
pivotIndex = partition(arr, size); quickSort(arr,
pivotIndex); quickSort(arr + pivotIndex + 1, size -
pivotIndex - 1); } } int partition(int arr[], int size) { int
mid = size / 2; swap(arr[0], arr[mid]); // pivot to front int
i = 0; for (int j = 1; j < size; j++) { if (arr[j] < arr[0])
{ i++; swap(arr[i], arr[j]); } } swap(arr[0], arr[i]); return
i; }
```

- 1 **mergeSortRec(arr, mid, tmp)** — Left half recursively sort karo.
- 2 **mergeSortRec(arr+mid, size-mid, tmp)** — Right half recursively sort karo.
- 3 **merge(arr, mid, arr+mid, size-mid, tmp)** — Do sorted halves ko merge karo.
- 4 **quickSort: pivotIndex = partition(arr, size)** — Partition karo, pivot ki position lao.
- 5 **quickSort(arr, pivotIndex)** — Left part recursively sort karo.
- 6 **quickSort(arr+pivotIndex+1, size-pivotIndex-1)** — Right part recursively sort karo.

EXAM FOCUS POINTS

FORMULA

Merge Sort Complexity: $O(n \log n)$ time, $O(n)$ extra space.

FORMULA

Quick Sort Average Complexity: $O(n \log n)$, Worst-case: $O(n^2)$ (if pivot bad).

CONFUSION

Merge Sort is NOT in-place (needs $O(n)$ extra space).
Quick Sort IS in-place (no extra array).

MCQ

Divide and conquer algorithms: Merge Sort, Quick Sort, Heap Sort, Binary Search.

PRACTICE

Array [10,4,8,12,11,2,7,5] merge sort steps likhein — divide into halves, sort, merge.

CONFUSION

Quick sort partition pivot ko correct position mein laata hai — left side smaller, right side larger.

MIND MAP

$O(n \log n)$ Sorting Algorithms

Merge Sort

- Divide → sort → merge
- $O(n \log n)$ time
- $O(n)$ extra space
- Stable sort

Quick Sort

- Pivot → partition → recurse
- Avg $O(n \log n)$
- In-place (no extra array)
- Unstable sort

Heap Sort

- Build heap → extract min
- $O(n \log n)$ time
- In-place
- Unstable sort

Course Overview

- Arrays, Linked Lists
- Stacks, Queues
- Trees, AVL, Union-Find
- Hashing, Sorting

CONCEPTUAL QUESTIONS & QUIZ

► CONCEPTUAL QUESTIONS

Q: Merge Sort aur Quick Sort mein kya basic farq hai?

A: Merge Sort array ko halves mein divide karta hai, sort karta hai, phir merge karta hai — $O(n)$ extra space. Quick Sort pivot choose karta hai, partition karta hai, recursively sort karta hai — in-place (no extra array).

Q: Quick Sort ki worst-case $O(n^2)$ kab hoti hai?

A: Jab har baar pivot smallest ya largest element ho (sorted array) — partition unbalanced ho jata hai. Is liye random pivot choose karte hain.

Q: Merge Sort ko in-place kyun nahi kehte?

A: Kyunki merge operation ke liye temporary array ($O(n)$ size) chahiye — original array mein merge nahi kar sakte.

Q: Divide and conquer ki strategy kya hai?

A: Problem ko chhote parts mein divide karo, har part ko solve karo, phir results ko merge karo — wars mein bhi yahi strategy use hoti hai.

Q: Course mein sab se important data structure kaun sa hai?

A: Koi ek nahi — har problem ke liye different data structure suitable hai. Arrays, Linked Lists, Stacks, Queues, Trees, AVL, Union-Find, Hash Tables — sab apni jagah useful hain.

► QUICK QUIZ (10 QUESTIONS)

1. Merge Sort ki complexity? **($O(n \log n)$)**
2. Quick Sort average complexity? **($O(n \log n)$)**
3. Merge Sort extra space? **($O(n)$)**

- | | |
|---------------------------------------|--------------------------|
| 4. Quick Sort in-place hai? | (Haan) |
| 5. Merge Sort ka pehla step? | (Divide in half) |
| 6. Quick Sort mein pivot kya hai? | (Selected element) |
| 7. Partition operation kya karti hai? | (Pivot around rearrange) |
| 8. Divide and conquer example? | (Merge Sort, Quick Sort) |
| 9. Quick Sort worst-case? | ($O(n^2)$) |
| 10. Merge Sort stable hai? | (Haan) |

EXAM REVISION SHEET

IMPORTANT DEFINITIONS

- **Merge Sort:** Divide and conquer — split, sort recursively, merge — $O(n \log n)$, $O(n)$ space.
- **Quick Sort:** Divide and conquer — pivot, partition, recursively sort — avg $O(n \log n)$, in-place.
- **Divide and Conquer:** Problem ko chhote parts mein divide karo, solve karo, merge karo.

IMPORTANT CONCEPTS

- Merge Sort: stable, not in-place, good for linked lists.
- Quick Sort: in-place, unstable, fastest average case.
- Both are $O(n \log n)$ — much faster than $O(n^2)$ for large data.
- Course covered: Arrays, Linked Lists, Stacks, Queues, Trees, AVL, Union-Find, Hashing, Sorting.

△ IMPORTANT FORMULAS

- Merge Sort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$
- Quick Sort average: $T(n) = T(n/2) + T(n/2) + O(n) \rightarrow O(n \log n)$
- Quick Sort worst-case: $O(n^2)$ (when pivot is min/max)

FREQUENTLY ASKED TOPICS

- Merge Sort algorithm steps (divide, sort, merge)
- Quick Sort partition mechanism
- $O(n \log n)$ vs $O(n^2)$ comparison
- In-place vs extra space

△ COMMON MISTAKES

- Merge Sort ko in-place samajh lena (nahi hai)
- Quick Sort ki worst-case bhool jana ($O(n^2)$ ho sakti hai)
- Merge Sort aur Quick Sort dono ko same samajh lena (different approaches)
- Partition aur Merge operations ko confuse karna

One-Day-Before-Exam Summary: Merge Sort aur Quick Sort dono divide and conquer algorithms hain — $O(n \log n)$ time. Merge Sort array ko halves mein divide karta hai, recursively sort karta hai, phir merge karta hai — $O(n)$ extra space leta hai, stable hai. Quick Sort pivot choose karta hai, partition karta hai (smaller left, larger right), recursively sorts — in-place hai, unstable hai, average $O(n \log n)$ but worst-case $O(n^2)$. Course mein humne Arrays, Linked Lists, Stacks, Queues, Trees, AVL, Union-Find, Hashing, aur Sorting cover kiye — yeh foundation hai data structures aur algorithms ki duniya mein. Mubarak ho! Aap ab CS301 complete kar chuke hain.